



Aras Innovator 33

TDF Administrator Guide

Document #: D-008159

Last Modified: 10/02/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments	6
Document Conventions	7
1 Overview	8
1.1 Anatomy of a Technical Document	9
1.1.1 Tech-Doc Enabled Item	9
1.1.2 Structured Content	10
1.1.3 Document Elements	11
1.1.4 Use of XML Schema and Schema Validation	11
1.1.5 Content Generators	11
1.1.6 Item Referencing	11
1.1.7 Property References	12
1.1.8 Filters	13
1.1.9 Document Modules	14
1.1.10 Document Content Styling	14
1.1.11 Metadata	14
1.1.12 Publication and Data Export	14
1.1.13 Templates	15
1.1.14 Images / Graphics	15
1.2 Terms	15
2 Document Type Configuration	18
2.1 Overview	18
2.2 Document Schema	18
2.3 Standard XML Schema	18
2.3.1 XML Schema	19
2.3.2 Document Element Types	20
2.3.3 Schema Validation	30
2.3.4 Document Element Configuration	31
2.3.5 Editing Referenced Technical Documents	31
2.4 CSS Styles	33
2.4.1 CSS Example	33
2.4.2 Defining a Cascading Stylesheet	36
2.4.3 CSS Formatting Examples	37
2.5 Publishing Technical Documents	41
2.5.1 Required Installations	41
2.5.2 Managing Conversion Rules	42
2.5.3 Output Settings	42
2.5.4 Style Settings	43
2.6 Key Handling	44

2.6.1	Enabling Extended Key-Handlers	44
3	Customizing Technical Documents.....	46
3.1	Enabling the Ability to Edit Child Documents.....	47
3.1.1	Edit Modes.....	48
3.1.2	Configuring an Edit Mode.....	48
3.1.3	Update Presentation Configuration	49
4	Content Generation.....	50
4.1	Overview	50
4.2	Content Generator Example	50
4.3	Content Generator Configuration	53
4.3.1	Content Generator Parameters	54
4.3.2	Content Generator Example – Name / Value Pair	54
4.3.3	Content Generator Example – Lists	55
4.4	Content Generator API.....	56
5	Item Reference Configuration	57
5.1	Overview	57
5.2	Configuration	58
5.2.1	onBeforeInit Method	60
i)	onAfterInit Method	60
6	Document Filters	62
6.1	Creating the Filter List	62
7	Appendix.....	64
7.1	Content Generator API.....	64
7.1.1	Table Document Element.....	64
7.1.2	List Document Element	65
7.1.3	Text Document Element.....	65
7.1.4	Image Document Element.....	66
7.1.5	Item Document Element.....	67
7.1.6	Item Property Document Element	67
7.2	Configuration Examples	68
7.2.1	Overview.....	68
7.2.2	Example – Auto-creating hidden elements from Tech Doc Item Properties	68
7.2.3	Example – Auto-creating inline link and text to external references:	71
7.2.4	Example – Using Attributes to apply styling	74
7.2.5	Example – Populating PDF header content from attribute data.....	76
7.2.6	Example – Creating a Table, in which each row points to individual Parts.....	77
7.3	Configuring a Table of Contents	82
7.4	Managing Languages.....	83
7.5	Attributes Validation Support.....	85
7.5.1	String Data Types.....	85
7.5.2	Date and Time Data Types.....	86

7.5.3	<i>Numeric Data Types</i>	87
7.5.4	<i>Miscellaneous Data Types</i>	88
7.6	File Import API.....	88
7.6.1	<i>Overview</i>	88
7.6.2	<i>Creating a File Importer</i>	89
7.7	Client API	94
7.7.1	<i>Overview</i>	94
7.7.2	<i>Scope</i>	94
7.7.3	<i>Customization Examples</i>	94
7.7.4	<i>Client API Definition</i>	99

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support/>

Document Conventions

The following table highlights the document conventions used in the document:

Convention	Description
Bold	This shows the names of menu items, dialog boxes, dialog box elements, and commands. Example: Click OK .
Code	Code examples appear in <code>courier</code> font. It may represent text you type or data you read.
Yellow highlight	Code highlighted in yellow draws attention to the code that is being indicated in the content.
Yellow highlight with red text	Red text highlighted in yellow indicates the code parameter that needs to be changed or replaced.
<i>Italics</i>	Reference to other documents.
Note:	Notes contain additional useful information.
Warning	Warnings contain important information. Pay special attention to information highlighted this way.
Successive menu choices	Successive menu choices may appear with a greater than sign (-->) between the items that you will select consecutively. Example: Navigate to File --> Save --> OK .

1 Overview

The Technical Document *Framework* (TDF) provides the configuration tools and authoring User Interface for the Technical Documentation *Application* (Tech Docs). It provides the core functionality to enable the creation of different types of Document content. As compared to documents that store data in individual files, 'Document' in this context refers to a collection of text and graphic (image) data, that conforms to a well-defined document *structure*, which is stored as XML with an associated Item. This content can then be *published* into one or more files in a file system, but that is not its native storage format.

The Technical Documentation Application is a content authoring environment, fully integrated into Aras Innovator PLM that collectively provides component content management capability for topics-based, modular documentation. Tech Docs enables users of various disciplines to author, visualize, share, and publish information all within a central, controlled, and collaborative environment. Unlike other systems that provide Document-level Management functions for content produced by external applications, Aras Tech Docs leverages Innovator's core PLM functionality (Access Control, Workflow/Lifecycle, Change Notification, etc.) and provides a web-based authoring tool for creating content directly, embedding shared document components, or referencing other elements managed by the PLM system. So as related Documents, Parts, Project Tasks, Requirements, or any other information managed by Aras Innovator evolve through subsequent revisions, authors of technical documentation can be alerted and navigated to pending changes, or the system can update the related documentation automatically.

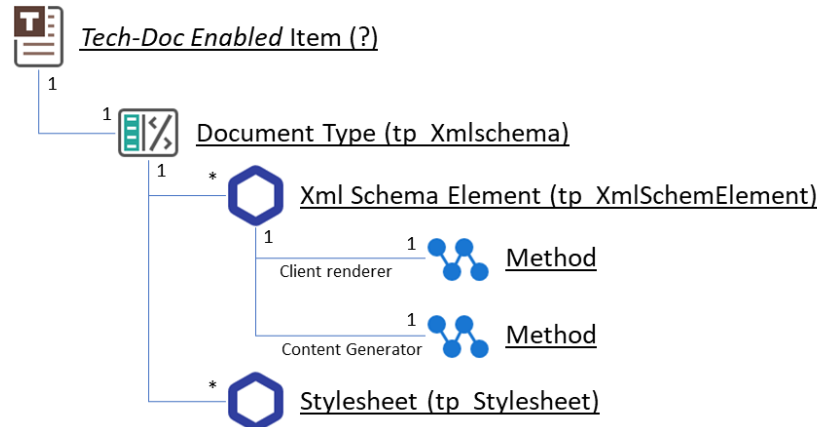
Technical Documents can be composed of individual components (sections, paragraphs, illustrations, tables, etc.), where each component can change independently; and optionally be created/managed by multiple users. The aggregated information forms the complete published document. This Structured Content is specified by a user-defined data model that identifies the elements of each component, their relationships and cardinality, and any restrictions on data type and value limitations. The built-in authoring User Interface directs the author as to the types of document elements that can be inserted, constantly validating the document as it is created. Candidate documents for release can be routed through workflow and published on final acceptance.

This document describes the functions and features of the Technical Documentation Framework, which is used to configure Technical Documentation. It is meant to be a reference for the Technical Document Administrator for configuring technical content.

- *Section 1* provides a brief explanation of the major components of a Technical Document and the Technical Document Framework.
- *Section 2* describes how to configure a Document Type.
- *Section 3* provides a brief overview of configuring the **tp_Block** ItemType, which is the Item used for the Technical Documentation Application
- *Section 4* describes Content Generators and how to create them.
- *Section 5* discusses Item Reference Configuration and how/when to use them.
- *Section 6* shows how to configure the Filters/Options for a Technical Document
- *Section 7* is an Appendix with subsections describing additional useful information for configuring a Technical Document and customizing the environment.

1.1 Anatomy of a Technical Document

In its most basic instance configuration, a Technical Document-enabled Item is a single entity that contains formatted text, stored as XML in a single Property, and one or more associated graphics. Each of these Items references a Document Type that specifies the XML Schema, CSS Styling, and any configured content renderers/generators for automating the creation of XML content.



Tech-Doc Enabled Items can also reference other Tech-Doc Enabled Items and Business Objects creating a rich Digital Thread. The following sub-sections will provide a brief overview of each of the main features of the Technical Documentation Framework with links to associated sections for more information.

1.1.1 Tech-Doc Enabled Item

This document will refer to an ItemType, or an instance thereof, as being *Tech Doc Enabled* if this ItemType that has been configured to use the Technical Document Editor and store the XML Content in a configured Property. This document will also refer to Items of this type as simply a 'Technical Document'. For the Technical Documentation Application, the `tp_Block` ItemType is an instance of a Tech-Doc Enabled Item.

1.1.2 Structured Content

Technical Documents have content maintained as XML, based on an XML Schema provided by the user. As authors add text and graphics to a document, XML Elements are generated and added by the system and stored in a single Property. The XML provides additional metadata about each Document Element and the XML Schema ensures that the structure of the document is consistent and compliant.

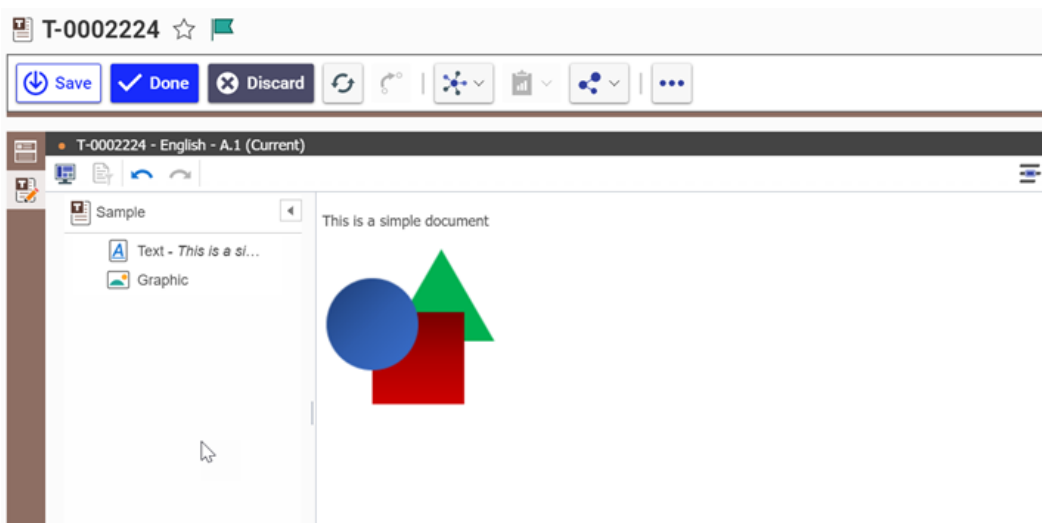


Figure 1 Sample Document - Showing Editor View

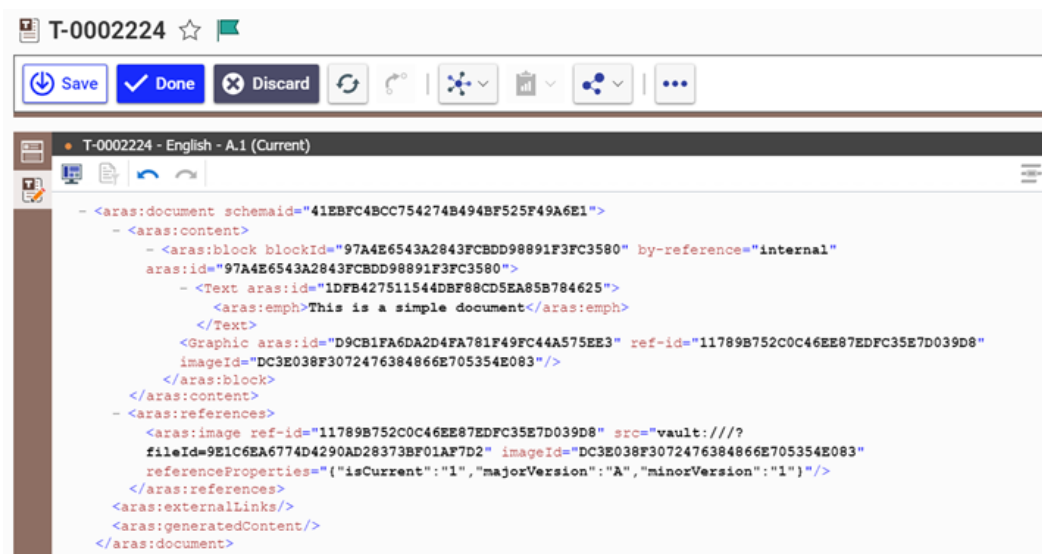


Figure 2 Sample Document - Showing XML Content View

1.1.3 Document Elements

In XML vernacular, the term 'Element' refers to the markup tag (e.g., <ElementName>) and everything that is included – as child Elements. This document uses the term '**Document Element**' to identify the types of content that can be added to a Technical Document. Document Elements are stored as individual XML Elements with a name and optional Attributes, which provide additional metadata to be used to further characterize the Document Element, distinguish its style, etc. Document Elements derive from one of the following Types:

- **Container** – Use to *contain* other Document Elements – e.g., Chapter, Section.
- **Formatted Text** – Text that can have basic formatting applied by the author – bold, italic, underline.
- **Unformatted Text** – Text without author-specified formatting
- **List** – Container for List Item Document Types
- **List Item** – Container for content to be displayed in a List.
- **Image** – References a 2D graphic stored by a `tp_Image` ItemType.
- **Table** – Container for Row Document Types
- **Row** – Container for Cell Document Types
- **Cell** – Container for content to be displayed in a cell of a Table.
- **Item** – Container Element that can be associated directly with an Item.
- **Item Property** – Unformatted Text Document Element that references a specific Property on an Item referenced by a parent (or ancestor) Item Document Element.
- **Embedded Document Element** – Refers to any Image, Text, Item, or Item Property Document Element that exists within a parent *Text* Document Element. These *embedded* Document Elements appear in line with surrounding text.

1.1.4 Use of XML Schema and Schema Validation

The authoring process is guided by XML Schema Validation. The Document Elements that can be added at any place in the document is dictated entirely by the XML Schema contained in the associated Document Type and enforced by the Schema Validator. XML Schemas specify the XML Elements, Attributes associated with each, the hierarchy of these elements, and the cardinality. Note that not all the XML Schema *schema* is supported.

1.1.5 Content Generators

In addition to manual addition and removal of Document Elements, a configuration mechanism can be used to execute Content Generators – Method code – to automate the creation/update of Document Elements. This method is typically used when there has been association/reference set between a Document Element and a selected Item Instance whereby Property values from these referenced Items are used specifically to create text or graphics content in the document.

1.1.6 Item Referencing

Technical Documents can have explicit relationships created with other Items maintained in Innovator. These Items are referred to as 'Business Objects'. This relationship establishes a direct binding between a Document Element and the associated Business Object, although the relationship is maintained by a

RelationshipType between the Technical Document and the Business Object. In addition, property data from these Business Objects can be queried and used within the Technical Document. When the Business Object property data is updated, the associated Technical Documents are updated accordingly. This binding helps maintain data integrity and identify a link between Objects and the documentation that was created specifically for them.

1.1.6.1 Item Reference Configuration

Item Reference Configuration provides a mechanism to automate the creation of values for Properties of Referenced Items and validate content added by the author. Configured JavaScript Methods are used to apply custom business logic and set and/or verify Property Data.

1.1.6.2 Pick vs Create

When an Item Document Element is created in the Technical Document Editor, the author has the choice of choosing an existing Item to reference or create one at that moment. If an Item is created, the Item Reference Configuration (if defined) is used to prepopulate and/or validate Properties of the generated Item.

1.1.6.3 Reference ItemTypes

ItemTypes that can be referenced by a Technical Document must be configured as a PolySource for the `tp_Item` ItemType.

1.1.7 Property References

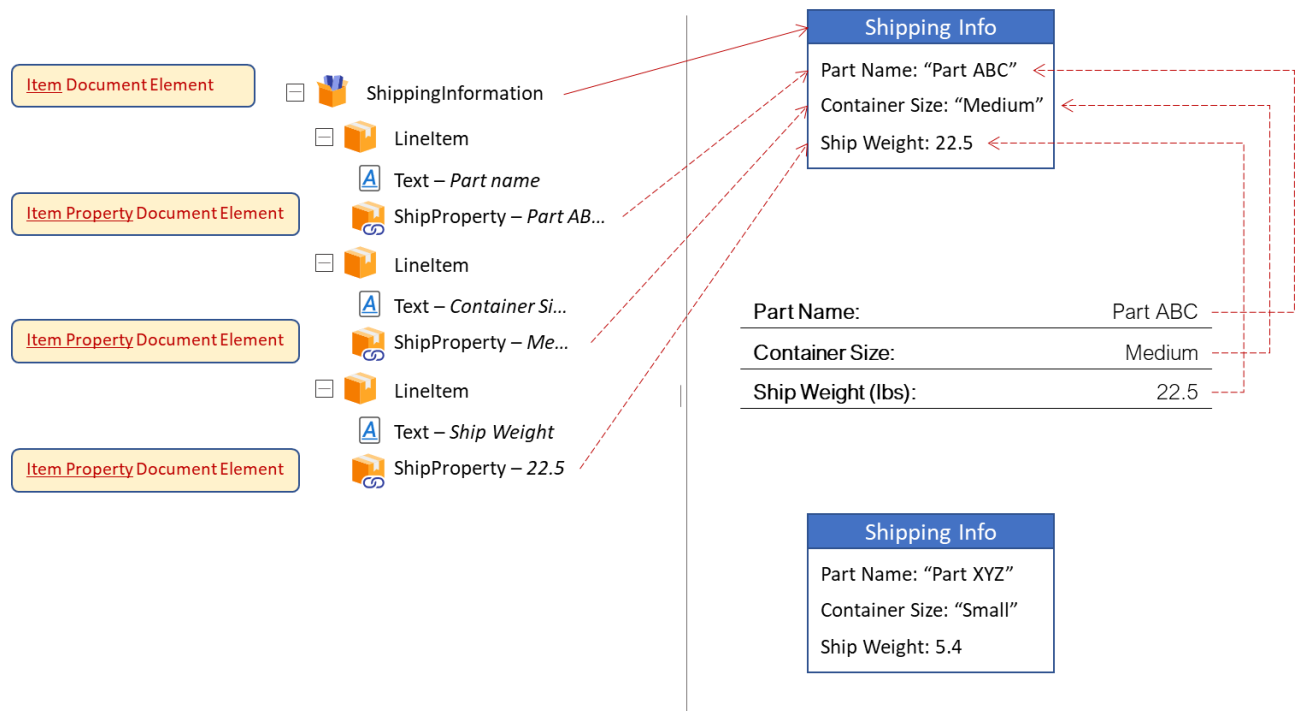
Document Elements can be configured as a 'Mapped Property'. These Item Property Document Elements are associated with Properties based on the Property Name. The Item used specifically for Property content is determined by a Document Reference of some Document Element parent (or grandparent, etc.).

Using the following illustration as an example, assume that there exists an ItemType – *Shipping Info* – that contains the following Properties:

- Name: *part_name*, Label: *Part Name*
- Name: *container_size*, Label: *Container Size*
- Name: *ship_weight*, Label: *Ship Weight*

In addition, there exists a Technical Document with schema elements:

- 'ShippingInformation': Document Element of type 'Item' that references a ShippingInformation ItemType and a container for 'LinItem's
- 'LinItem': Document Element of type 'Container' that is a container for one Text Document Element – used for a Label – and one Item Property Document Element – used to reference an Item Property.
- Name: 'ShipProperty': Item Property Document Element



In the scenario above, the top-level Item Document Element references a specific Shipping Info Item with the Properties and values shown. Switching this top-level Document Element to reference the other Shipping Info Item would cause the ShipProperty Document Elements to update automatically to the values of this Item.

1.1.8 Filters

Filters provide a mechanism to 'tag' individual Document Elements with metadata ('Filters'), which can then be used to hide content when published. Filters consist of a name and one or more values. For example – a document describing technical information related to an automobile may have a 'Model' Filter that can have values for each of *models* for a particular Make. "A4", "A5", "A6" are example Models for Audi. Filtered Document Elements provide a reuse mechanism to isolate (or hide) certain specific content and expose common content. Using the example, an author would be able to 'publish' a Technical Document filtered by a specific Model.

1.1.9 Document Modules

Technical Documents can exist either as a single entity - with all content contained within – or aggregate content from multiple document *modules*. Breaking up documents into smaller components has multiple advantages including the ability to:

1. Reuse each component.
2. Enable concurrent editing.
3. Apportion each component to areas of specific specialty or expertise.

In the Technical Publication domain, technical content is sometimes referred to as ‘Topics-based’ or ‘Modular’. This refers to content that is about a specific topic or area of focus and is, to some degree, context insensitive. This helps to characterize the content and enable it to exist in multiple places within a document. Contrast this with a narrative in other forms of documents where every paragraph builds upon (and thus depends on) previous paragraphs and context.

Each Technical Document instance can evolve independently and be protected by specific access rights. To be reusable, each document component needs to be constructed in a way that allows it to fit within a larger context. To achieve this, the content of each Document / Document Component will be based on an overarching schema or document structure (Section 2.2). This document structure helps ensure consistency and provides a mechanism for determining how individual components can fit into a larger context.

1.1.10 Document Content Styling

The style and layout of Technical Documents is dictated by a separate style configuration that is maintained separately but associated directly with a Technical Document Type definition. Each Document Element defined in the Schema can be referenced and have a unique set of styling parameters that are used to position and render the associated Document Element instances in each document. Style settings can be assigned to HTML output, PDF Output, and the Document Editor.

Referencing centrally controlled style settings ensure consistency in look and feel for published content and provides a convenient mechanism to update style in one location and have it automatically applied to all associated documents.

1.1.11 Metadata

All Document Element content can have additional metadata – configured as XML Attributes - assigned to provide additional semantics for content classification and aid in content search and retrieval. This metadata is provided by the Technical Document Author when the Document Structure is designed and configured.

1.1.12 Publication and Data Export

The Technical Documentation application, together with Aras Innovator, provide an authoring and data management environment. To make use of the content created, it needs to be exposed outside the PLM environment. The publishing/export function (Section 0) provides an ability to convert the content of each Technical Document to a format more suitable to end-user consumption.

1.1.13 Templates

Technical Documents can be created to be used as Templates. These 'Template' documents can be used to create duplicate content in other Technical Documents – template content is inserted as a copy. The root Document Element of the document identifies where the content of the document can be inserted. Templates are useful when some portion of the content should be updated or extended. Authors should create a reference to other Technical Documents when that content will not be changed.

1.1.14 Images / Graphics

Images used within a Technical Document are managed by the ItemType `tp_Image`. This Item has a File Property for the 2D Image file. Tech-Doc Enabled Items will have a Relationship defined to this ItemType.

1.2 Terms

The following Terms are used throughout this document:

Term	Definition
Business Object	Refers to any Item in Innovator that contains information, which may be referred to in a Technical Document.
Cascading Style Sheets (CSS)	Industry standard text-based format for controlling the position, color, size, etc. of web-based content. It is used by the Technical Documentation application to control the format of HTML and PDF content.
Content Filters	Metadata (name/value pairs) that are defined by the Technical Document Administrator and chosen by the Technical Document Author to characterize Document Elements within a Technical Document.
Context Menus	Pop-up menus, used mostly by Document Elements, that contain menu items used to invoke various functions on the Document Element instance. The term 'context' is used because the specific menu items displayed in the menu may be different depending on the specific Document Element instance selected.
Document Element	Refers to the components that make up a Technical Document. Every Document Element is associated with a specific node in the Navigation Pane and XML Element in the resulting content. Document Elements are explicitly defined in the Technical Document Type Definition.
Document Style	Refers to the formatting and positional logic that defines the placement and look-and-feel of the rendered content of a Technical Document. Document Style is controlled using Cascading Style Sheets (CSS), associated with a specific Document Type, and defined by the Technical Document Administrator.
Embedded Document Element	Refers to a either a Text, Image, Item, or Item Property that is a direct child of a Text Document Element. When placed within a Text Element, these <i>embedded</i> Document Elements are placed in line with surrounding text.
Graphic	Any 2D image that can be displayed in a Technical Document.
Mapped Property	Refers to a Document Element with content that is automatically created from a Property of a referenced Item. Also referred to as an Item Property Document Element.
Structured Content	Refers to the general use of an underlying schema to define the content of a document. Structured Content is contrasted with unstructured (or freeform) content in which case there are no rules that help define and control the content.
Tech Doc Enabled	ItemType that has been configured to use the Technical Document Editor and store the XML Content in a configured Property. The term is also applied to Items (instances) of these ItemTypes.

Technical Document	Refers to instances of a Technical Document ItemType. In this case, the term is capitalized. When lower case, the context is more general – any technical document.
--------------------	---

2 Document Type Configuration

2.1 Overview

A Technical Document Configuration defines the XML Schema used for a Technical Document, style configuration using Cascading Stylesheets, Methods used for Content Generation and Editor rendering customizations, and is maintained by a Document Type (tp_XmlSchema). This section provides a description of each of these areas.

2.2 Document Schema

The content of all Technical Documents is defined as XML and the Technical Document Editor generates this XML as authors add Document Elements. The types of Document Elements, along with the number of each allowed (cardinality) is defined by an XML Schema.

Note: XML and XML Schema are ubiquitous for defining digital content and the rules for this information respectively. This document assumes the reader's familiarity with both.

Note: This document will use the terms 'XML Element' and 'Document Element' interchangeably. However, the term 'Document Element' has additional meaning in the way they are handled specifically by the system. Document Element 'Types' are defined in Section 2.3.2.

2.3 Standard XML Schema

The Technical Documentation Application includes a default schema that defines a document structure called **Standard**. It is possible to customize this schema as well as develop additional custom schemas to meet business requirements.

Note: For more information on how to use the Technical Documentation Editor, see Aras Technical Documentation – User Guide for reference.

Note: Administrators should use the *Standard* Document Type for reference only. Modifications / Customizations to a Document Type is best applied to a new copy of the Standard Document Type.

The **Standard** schema implements the following structure:

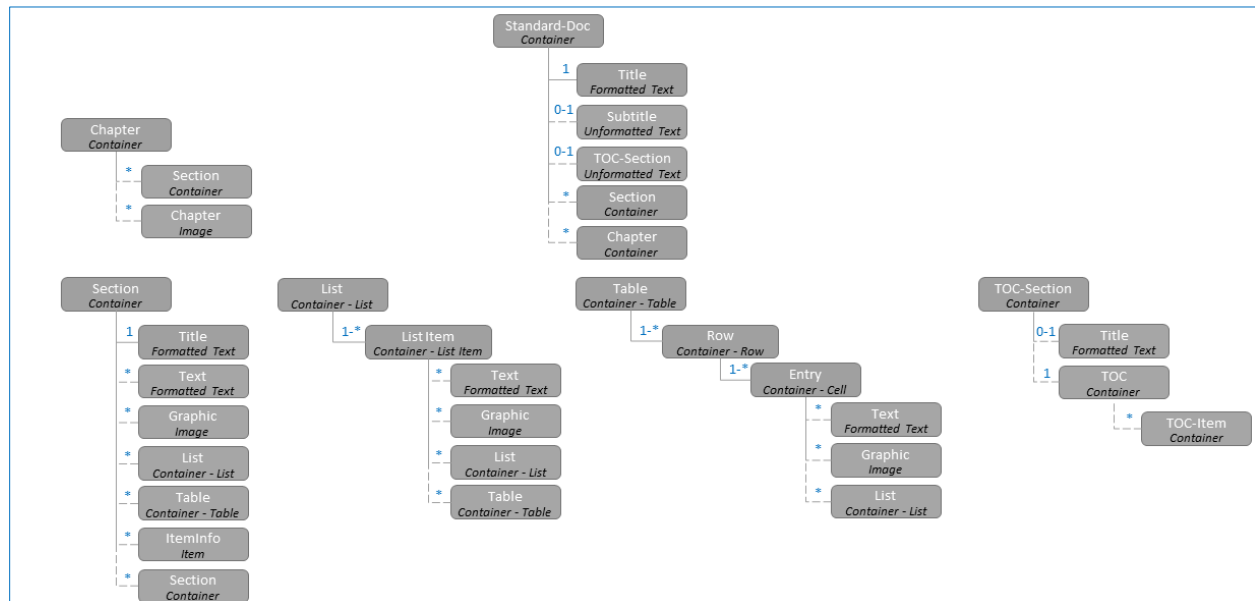


Figure 1. Standard Document Type

2.3.1 XML Schema

To understand how XML schema is used, it is helpful to discuss a portion of the schema provided with the 'Standard' Document Type, which is included by default.

```
<xs:element name="Standard-Doc">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="Title"/>
      <xs:element ref="Subtitle" minOccurs="0" maxOccurs="1"/>
      <xs:element ref="TOC-Section" minOccurs="0" maxOccurs="1"/>
      <xs:choice>
        <xs:element ref="Section" minOccurs="0" maxOccurs="unbounded"/>
        <xs:element ref="Chapter" minOccurs="0" maxOccurs="unbounded"/>
      </xs:choice>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

The top-most schema element with name '**Standard-Doc**' is essentially a *container* for the content defined within it. For Document Types, content within another schema element is either defined as a *sequence* or a *choice*. In the XML Schema definition, a sequence is an ordered list, and a choice is unordered.

Note: There are other restrictions for using either `<xs:sequence>` or `<xs:choice>` that are not covered in this document.

Each schema element for a complex type is defined within either a `sequence` or `choice` and can have specific cardinality defined, which identifies the minimum and maximum number of XML Elements allowed. Minimum and Maximum counts are defined by the `minOccurs` and `maxOccurs` attributes respectively. Undefined `min/maxOccurs` attributes default to the value '1'. In this case, an XML Element with name 'Standard-Doc' can have, as children, the following XML Elements:

- A single, required 'Title' XML Element, followed by
- An optional 'Subtitle' XML Element, followed by
- An optional 'TOC-Section' XML Element (see Section 0), followed by either
- 0 or more 'Section' XML Elements or
- 0 or more 'Chapter' XML Elements

Each of the XML Elements in this example is defined in another portion of the XML Schema. XML Schema 'structures' defined in this manner thus define the structure of the corresponding content in a Technical Document. The following example shows what content using this schema might look like. Note that the content for a 'Chapter' XML Element was not described above but is included in the Standard Document Type.

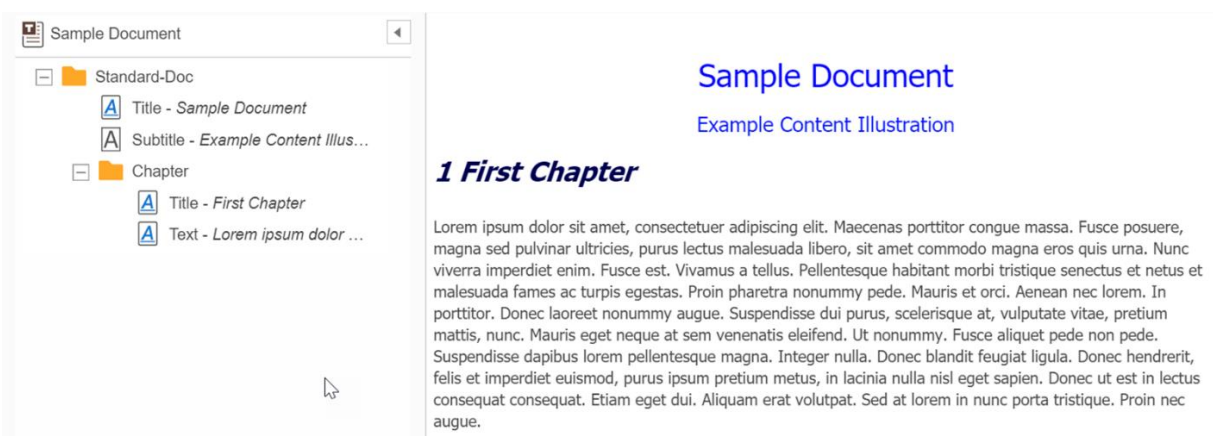


Figure 3: Sample Content from Standard Document Type

2.3.2 Document Element Types

XML Elements defined within the XML Schema for a Document Type can be configured to extend XML Elements defined by the system. In this way these Document Elements are processed differently and provide additional functionality within the Technical Document Editor. This section describes each of the Document Element Types.

2.3.2.1 Container Elements

The simplest Document Element is identified as a 'Container Element' because it is used specifically to *contain* other Document Elements and thus has no other special processing by the Editor. 'Standard-Doc' is an example of this type. 'Section', 'Chapter', and 'Graphic-Block' are also examples of Container Elements defined in the 'Standard' Document Type. Note that many of the other Document Types defined in this section are also used to contain other content, but they have additional qualities and are thus processed differently by the Editor.

2.3.2.2 Item Elements

Item Document Elements are used to associate content within a Technical Document with a separate Item (Business Object). Typically, Item Document Elements also have a Content Generator (Section 4) defined so that the Item Document Element's content can be automatically generated based on information extracted from the associated Item. This type of association is critical to a Digital Thread and helps maintain a link between technical documentation and the business objects it describes.

Once a Document Element is created, the system will prompt the author to select the referenced Item and a relationship will be created between that selected Item and the Technical Document Item in which the Item Document Element was placed. To illustrate this process, the following example of an Item Document Element is described:

```
<xs:element name="PartData">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:itemType">
        <xs:choice maxOccurs="1">
          <xs:element ref="Table" minOccurs="0" maxOccurs="1"/>
        </xs:choice>
        <xs:attribute name="typeId" type="xs:string" fixed="<Part ItemType Id>"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

This example describes a '**PartData**' Item Document Element. Item Document Elements extend the `aras:itemType` XML Element. This example defines a single child Document Element – **Table**. In addition, the XML Attribute `typeId` is used to fix the associated/referenced ItemType to a **Part** ItemType.

Note: Defining the `typeId` is optional. If it is not defined the ItemType of the referenced Item can be any of the PolySources defined within `tp_Item`. The value of the `fixed` attribute must be the ItemType ID of one of the PolySources configured.

When placing an Item Document Element, the author has a choice to either choose an existing Item or create a new one.

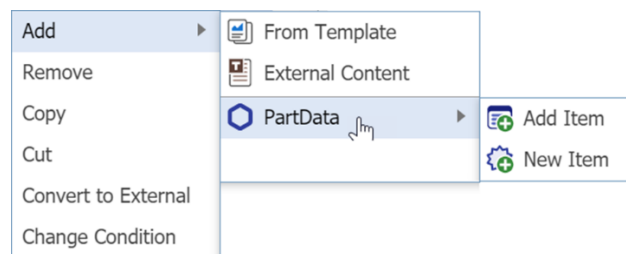


Figure 4: Adding an Item Document Element

If the author chooses '**Add Item**', the system will display a search dialog to choose a specific Item. If the `typeId` is used in the schema, only ItemTypes corresponding to the ItemType id specified will be displayed. Otherwise, the user can select the specific ItemType or search all.

If the author chooses '**New Item**', the system will display the Form of the ItemType so that the Property values can be specified before the Item is created. If the `typeId` is not specified in the schema, the author will first need to select the specific ItemType to be created.

Authors can change the referenced Item for existing Item Document Elements with either a different, existing Item or based on a newly created Item. The functionality is like 'Add' and 'New' as defined above respectively.

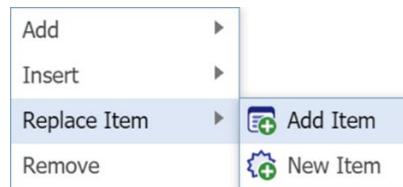


Figure 5: Replacing a Referenced Item

2.3.2.3 Text Elements

Text Document Elements are used for textual data. Text Document Elements can either be formatted or non-formatted. Formatted Text Document Elements can have text content within them formatted with a limited set of options. Unformatted Text Document Element do not have formatting options.

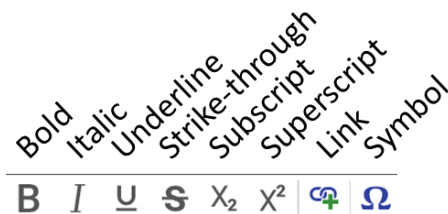


Figure 6: Formatted Text Options

Note: The intent for Technical Documentation is to have the format of content be managed by CSS Styles as much as practical. This establishes consistency across all technical documents that are based on the same Document Type.

The following example shows two types of Text Document Elements.

```
<xs:element name="Subtitle">
  <xs:complexType mixed="true"/>
</xs:element>

<xs:element name="Text">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:text"/>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

In this example the 'Subtitle' Document Element is unformatted, and the 'Text' Document Element is formatted. Unformatted Text Document Elements are defined as a complex type with the `mixed` attribute set to 'true'. Formatted Document Elements extend the `aras:text` XML Element.

Note: The Technical Document Editor will attempt to place a Text Document Element whenever the user has selected some Container Document Element (Container, Cell, List Item, etc.) and types a character. At this point, the system will search for any child Document Element of type Text allowed by the schema, insert a new instance of that Text Document Element, apply the characters typed by the user, and place the cursor at the end for further text input.

2.3.2.4 Image Elements

Image Document Elements are used to place 2D images in a Technical Document. All Image Document Elements use the `tp_Image` ItemType to contain the 2D image file and are thus independently managed.

When a user places an Image Document Element, the system will display a Search Dialog for Items of type `tp_Image` (Graphic). Like Item Document Elements, an author can either select an existing Graphic Item or create a new Item. In the latter case, the system will present a Form for the author to enter necessary Properties before the new Graphic Item is created. Referenced Graphic Items will have a Relationship Item generated between the Graphic Item and the Technical Document Item in which the Image Document Element was placed.

The following example shows an Image Document Element schema definition:

```
<xs:element name="Graphic">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:imageType"/>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

In this example, the '**Graphic**' Image Document Element is created. Image Document Elements must extend the `aras:ImageType` XML Element.

Note: Authors can also insert new Graphic Items by pasting an Image from the clipboard. If a Document is selected in the Technical Document Editor that *allows* a following sibling Image Document Element of any name defined in the schema, the system will automatically create a Graphic Item using the Item Reference Configuration (see Section 5) to automatically populate required Graphic Item Properties and upload the image in the clipboard as File Item to the vault.

2.3.2.5 Mapped Properties

Mapped Property Document Elements are a type of Unformatted Text Document Element that are used to reference a Property of an Item referenced by an ancestor Item Document Element. These are also referred to as *Item* Property Document Elements. When authors place instances of a Mapped Property Document Element, the system will automatically apply the content of the associated Property of the referenced Item.

In the following example, a '**ShippingInformation**' Item Document Element contains '**LinItem**' Container Document Elements, which contain a single '**Text**' (Formatted Text Document Element) and a '**ShipProperty**' Mapped Document Element:

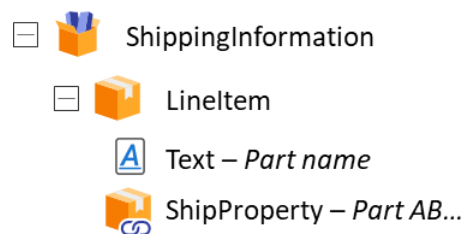


Figure 7: Mapped Property Example

The corresponding XML Schema would be specified as follows:

```
<xs:element name="ShippingInformation">
  <xs:complexType>
    <xs:complexContent>
```

```

<xs:extension base="aras:itemType">
  <xs:choice maxOccurs="1">
    <xs:element ref="LineItem" minOccurs="1" maxOccurs="unbounded"/>
  </xs:choice>
  <xs:attribute name="typeId" type="xs:string" fixed="<ItemType Id>" />
</xs:extension>
</xs:complexContent>
</xs:complexType>
</xs:element>

<xs:element name="LineItem">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="Text" minOccurs="1" maxOccurs="1"/>
      <xs:element ref="ShipProperty" minOccurs="1" maxOccurs="1"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

<xs:element name="ShipProperty">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:itemProperty">
        <xs:attribute name="property" type="xs:string" fixed="name"/>
        <xs:attribute name="mode" type="aras:itemPropertyModeEnum" fixed="read"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

```

The example illustrates a possible scenario in which Shipping Information is maintained within a separate Item (Business Object). In this case, there may be several Properties related to the shipping information managed. Using Item Document Elements with Mapped Property Document Elements in this manner provide a convenient mechanism to automatically populate content in a technical document when these Document Elements are placed manually and update the content dynamically when the referenced Item is changed. See Section 7.1.5 for a description of the Item Document Element.

The **'ShippingInformation'** XML Element extends `aras:itemType`, is *fixed* to refer to an ItemType (presumably containing Shipping Information), and contains a single **'LineItem'** Container Document Element. The **'LineItem'** Container Document Element contains a single Text Document Element - **'Text'** - and a single Mapped Property Document Element – **'ShipProperty'** – which extends `aras:itemProperty`. The **'ShipProperty'** Mapped Property Document Element is configured with two, optional Attributes: **'property'** and **'mode'**.

Note: The **'property'** Attribute specifies the name of the Property to reference. The association between a Mapped Property Document Element and an Item Property is by the Property's name. This association is loose, in that any ItemType with the same Property name will result in a mapping. Since referenced Items are associated via a PolySource, mapping by name allows the reference to 'float' to different ItemTypes.

Note: The **'mode'** Attribute specifies whether the reference Item Property is editable. A Mode value of **'read'** specifies that the Property is read-only. A Mode value of **'write'** specifies that the Property of the referenced Item can be updated based on changes in the technical document.

If the **'property'** Attribute is configured, it can specify the default Property name to map to (using the **'default'** attribute) or fix the value using the **'fixed'** attribute. If the **'property'** Attribute is not configured, authors can manually specify the Property to map using the Attributes Dialog:

The 'Attributes Dialog' window shows two fields: 'Property' with the value 'description' and 'Mode' with a dropdown menu set to 'write'.

Figure 8: Attributes Dialog for Mapped Property Document Elements

If the **'mode'** Attribute is configured, it can specify the default Property mode (using the **'default'** attribute) or fix the value to either **'read'** or **'write'** using the **'fixed'** attribute. If the **'property'** Attribute is not configured, authors can manually specify the Property to map using the Attributes Dialog. Fixed values for these of these two Attributes will disable the ability to change them in the Attributes Dialog.

Mapped Property Document Elements are edited using a pop-up Form in the Technical Document Editor. To edit a value, select the Document Element in the Editor or Tree View and/or double-click to display the Property Editor Form. Document Elements with a grey, dotted border can be edited. Document Elements with a red, solid border are read-only. For example:

Part Item Id	mc-19	Read-only
Part Name	Carburetor 1-1	
Part Item Id	mc-19	Editable
Part Name	Carburetor 1-1	

Name

Carburetor 1-1

Figure 9: Mapped Property Editor Form

Values can only be changed using this Form. Selecting the **'Check'** button will update the Mapped Property Document Element in the technical Document only. Authors will see a pencil icon (✎), signifying that the Property of the referenced Item has not been updated, next to the Document Element in the Tree View. The referenced Item will be updated when the technical document is saved, in which case the pencil icon will be hidden. Selecting the **'X'** button will cancel the update without any changes to the technical document.

Note: Selecting the pencil icon (✎) in the Tree View will prompt the author with a Dialog to discard the updated Property. In this case selecting **'OK'** will revert the modified Property(ies) to their previous value.

The following Property Value Types are supported:

- String
- Text
- Numeric (Integer, Decimal, etc.)
- Date
- Boolean
- Item
- List
- Image (Display only - Editor not available)

The Edit Field in the Property Edit Form is based on the Property Type. For Text and Numeric Fields, an attempt to set a value that does not match the type will result in an Error being displayed in a Dialog.

Figure 10: Mapped Property Edit Forms

Modifying a versionable Item vis-à-vis Mapped Property Document Elements will result in the referenced Item to be versioned automatically. Since Technical Documents reference Items via fixed Relationships, the Item Document Element will show as stale since it will refer to the previous generation of the Item.

2.3.2.6 Table Document Element

Table Document Elements are used to group content in a tabular format. Document Elements that are configured as Tables have additional functionality provided in the Technical Document Editor to provide the ability to add columns, rows, merge cells, etc. Thus Tables, Rows, and Cell Document Element are a type of Container Document Element, but with extended capability.

Note: It is possible to display content in a Technical Document as a Table without extending the Table and related Row or Cell XML Element Types using CSS Flex Box for example, but these Elements will not have the additional Table functions.

Configuring a Document Element as a Table requires corresponding Document Elements for Rows and Cells. Table Document Elements can be nested within other Table Document Elements. The following is an example of a Table Configuration in XML Schema:

```
<xs:element name="Table">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:table"/>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:element name="Row" substitutionGroup="aras:tablerow">
```

```
<xs:complexType>
  <xs:complexContent>
    <xs:extension base="aras:tr"/>
  </xs:complexContent>
</xs:complexType>
</xs:element>

<xs:element name="Entry" substitutionGroup="aras:tablecell">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:td">
        <xs:choice minOccurs="0" maxOccurs="unbounded">
          <xs:element ref="Text" minOccurs="0" maxOccurs="unbounded"/>
          <xs:element ref="List"/>
          <xs:element ref="Graphic"/>
          <xs:element ref="Table"/>
        </xs:choice>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

Table Document Elements extend the `aras:table` XML Element. Table Document Elements must contain Row Document Elements as their only child element. Likewise, Row Document Elements must contain Cell Document Elements as their only child element. Cell Document Elements can contain other types of content, including other Tables as specified above. Row Document Elements extend the `aras:tr` XML Element. Cell Document Elements extend the `aras:td` XML Element.

Note: The XML Schema file `Aras.xsd` contains the core XML Schema for Technical Documentation. The schema file defines `aras:tablerow` and `aras:tablecell`, shown as substitution groups in the example above.

Note: The Technical Document Editor will attempt to place a Text Document Element whenever the user has selected some Container Document Element (Container, Cell, List Item, etc.) and types a character. At this point, the system will search for any child Document Element of type Text allowed by the schema, insert a new instance of that Text Document Element, apply the characters typed by the user, and place the cursor at the end for further text input.

Warning There can only be one Table, Row, and Cell Document Element defined in a Document Type.

2.3.2.7 List Document Element

List Document Elements are used to group content in a sequential list format. Document Elements that are configured as Lists have additional functionality provided in the Technical Document Editor to provide the ability to set the individual List Items as bulleted, numeric, or alpha. Thus Lists, and List Item Document Elements are a type of Container Document Element, but with extended capability.

Note: It is possible to display content in a Technical Document as a List without extending the List and related List Item XML Element Types using CSS `display='list-item'` for example, but these Elements will not have the core functionality to set the bullet, numeric or alpha capability.

Configuring a Document Element as a List requires corresponding Document Elements for List Items. There can be multiple List Document Elements defined and it is possible to nest Lists within Lists. The following is an example of a List Configuration in XML Schema:

```
<xs:element name="List">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:list">
        <xs:choice maxOccurs="unbounded">
          <xs:element ref="List-Item" minOccurs="1" maxOccurs="unbounded"/>
        </xs:choice>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:element name="List-Item">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:listitemType">
        <xs:choice maxOccurs="unbounded">
          <xs:element ref="Text" />
          <xs:element ref="Graphic"/>
          <xs:element ref="List"/>
        </xs:choice>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

List Document Elements extend the `aras:list` XML Element. List Document Elements must contain List Item Document Elements as their only child element. List Item Document Elements can contain other types of content, including other Lists as specified above. List Item Document Element extend the `aras:listitemType` XML Element.

Note: The default List Document Element specifies at least one child List Item Document Element, as shown in the example above. When there is a single child Document Element that is required, the system will automatically place the child Document Element. For this List example, when the user places a List, the system will automatically insert the first List Item Document Element.

Note: The Technical Document Editor will attempt to place a Text Document Element whenever the user has selected some Container Document Element (Container, Cell, List Item, etc.) and types a character. At this point, the system will search for any child Document Element of type Text allowed by the schema, insert a new instance of that Text Document Element, apply the characters typed by the user, and place the cursor at the end for further text input.

2.3.2.8 Embedded Document Element

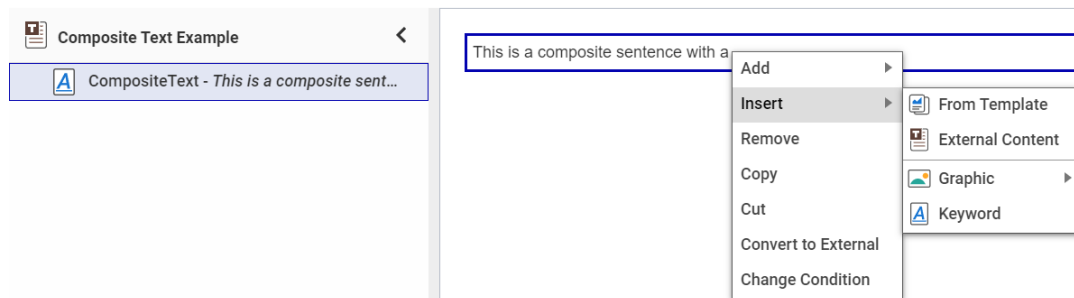
Embedded Document Elements are any Text, Image, Item, or Item Property Document Element that is configured as a direct child of another Text Document Element. The ability to *embed* Document Elements in this manner allows sentences to include other types of content interleaved within the text. The following example show a configuration for a 'CompositeText' Text Document Element, which can contain a 'Keyword' Text Document Element or a 'Graphic' Image Document Element.

```
<xs:element name="CompositeText">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:text">
        <xs:choice minOccurs="0" maxOccurs="unbounded">
          <xs:element ref="Keyword"/>
          <xs:element ref="Graphic"/>
        </xs:choice>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:element name="Keyword">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:text"/>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

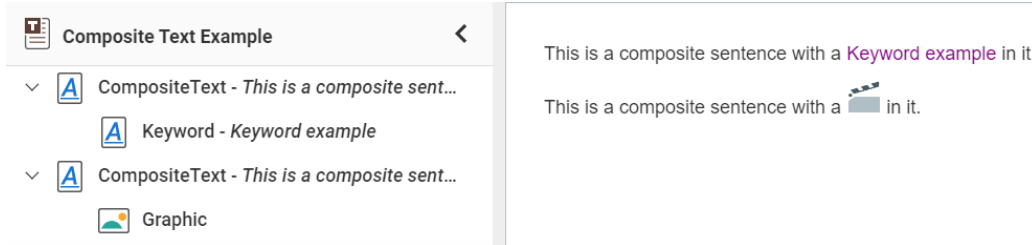
In the example schema above the 'CompositeText' Text Document Element is configured to optionally include either a 'Keyword' Document Element (which is another Text Document Element) or a 'Graphic' (which is explained in other sections of this document). Embedding graphics/images allows text element to contain images within the text. This is useful for symbols that are stored as images for example. When configuring one type of Text that can be included within another the embedded text can be distinguished semantically and/or visually.

When Text Document Elements are configured with Embedded Elements an **Insert** menu option is added, which will display the configured child types. Using the schema sample above would result in the following Insert options:



Inserting Embedded Elements

The following example shows the result of a Keyword and a Graphic being embedded in separate composite sentences. Note that the Keyword is styled to be rendered in purple.



2.3.3 Schema Validation

The Document Type Form uses a Text Editor to display and edit the XML Schema definition. This Editor has been configured specifically for XML Schema to highlight XML keywords and show validation errors/warnings with indicators of such on the corresponding line in the User Interface. The XML Schema is validated as the user edits the content and when the user Saves the Document Type Item. If there are any warnings or errors, the user is prompted with a message. It is not possible to save a Document Type Item if there are schema errors. This is to prevent the inadvertent use of the schema if it is not valid.

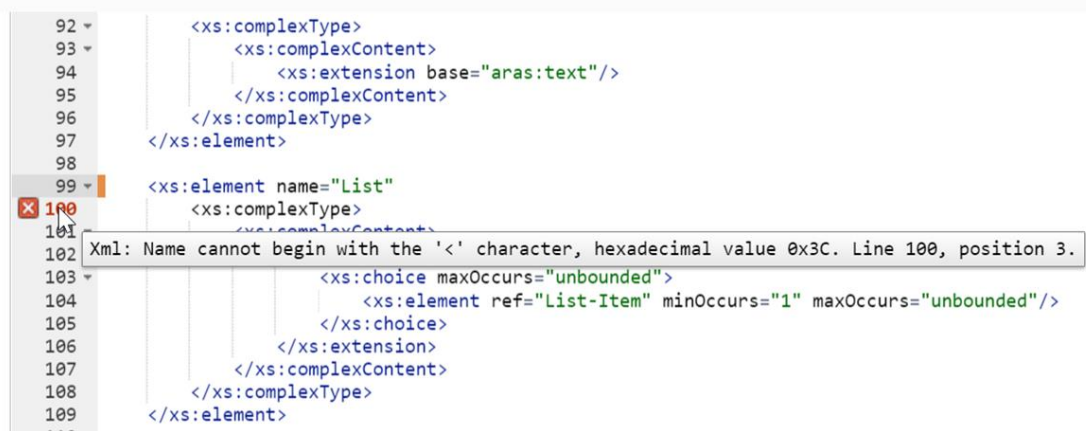


Figure 11: XML Schema Validation Warning

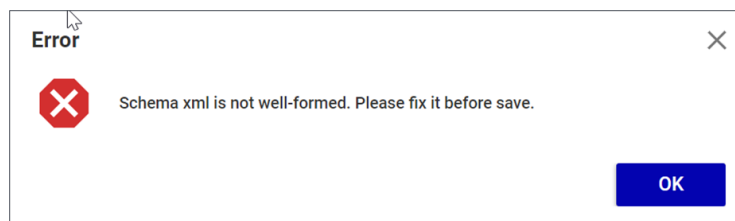


Figure 12: XML Schema Validation Error Dialog

2.3.4 Document Element Configuration

For each Document Element, the Technical Document Framework provides the ability to configure Content Generators (Section 4), Methods to prepopulate and validate Properties on referenced Items (Section 5), custom Tree Node Renders, and the default action for specific Classifications of Technical Document Enabled Items. These configuration options extend the default behavior of Document Element Types.

Name	Renderer [...]	Content Generator [...]	Dynamic Con...	Document Classification
ShippingList		td_ProductInformationCG	☐	
SpecInfo		td_ProductInformationCG	☐	
Product-Data-...			☐	ProductDataSheet
List-Item	td_PDS_ListItemRenderer		☐	

Figure 13: Document Element Configuration User Interface

Document Element configuration is specified using XML Schema Element Items. References to these Items are added in the Document Type Form. For each XML Schema Element Item, the associated Document Element is identified by **Name**. It is important that the spelling of the name match what is defined for the XML Element in the XML Schema. The **Renderer** identifies the JavaScript Method that is used as an alternative to the default logic for populating the Tree Node Icon and Text. The **Content Generator** identifies the C# Server-side Method that is called whenever an instance of the corresponding Document Element is placed in a Technical Document or when it is refreshed. The **Dynamic Content** checkbox determines how often a configured Content Generator is executed. When *False*, the Content Generator is only executed when the Document Element is first created or when it is manually refreshed in the Technical Document Editor. When *True*, the Content Generator is additionally executed whenever the Technical Document is opened or saved. **Document Classification** associates the Document Element with a Classification for the Technical Document Enabled Item. When the Technical Document Enabled Item is first created, with a specified Classification, the system will automatically add the Document Element as the root. In this case, only a single root element is allowed for this technical document.

2.3.5 Editing Referenced Technical Documents

Technical Documents can reference (and thus be composed of an aggregation of) the content of other Technical Documents so that documentation is managed by individual document *modules*. By default, all referenced document content cannot be edited within the context of a Technical Document that references it. Changing this content would thus require opening the associated Technical Document Item and editing the content from there. However, it is possible to enable referenced content editing in one of two modes: Implicit or Explicit.

Note: See Section 3.1 for more information on how to configure external edit mode.

Implicit Edit Mode allows authors to edit referenced content as if the referenced content was part of the opened document without any need to lock this content prior. In this mode, when the parent (or opened) document is saved, the referenced document content is locked, updated, unlocked in a continuous series of steps for any referenced document content that was changed.

Explicit Edit Mode will also allow authors to edit referenced content but will require that the associated Technical Document Item is first locked for Edit. This can be done directly in the Technical Document Editor or separately as is part of normal Lock/Unlock operations in Aras Innovator.

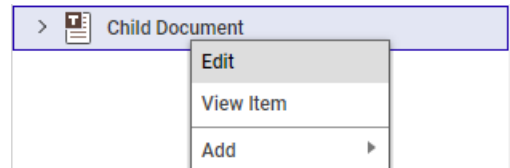


Figure 2. Example Edit Command in Context Popup Menu

Explicit Edit Mode also requires that referenced Technical Document Items be unlocked when the author has completed their updates. This can be done within the Technical Document Editor or separately as part of the normal Item Lock/Unlock operations in Aras Innovator.

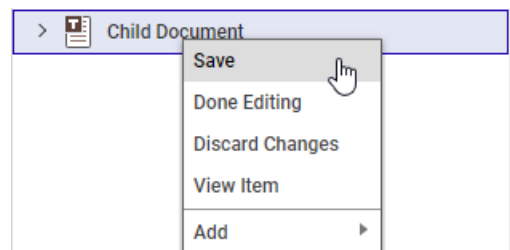


Figure 3. Example Save, Done Editing, Discard Changes Command in Context Popup Menu

In Explicit Mode, referenced and locked Technical Document Items are saved when the parent document is saved. Authors can also save referenced document content directly using the context popup menu.

When a referenced document is claimed or locked by a separate User, the Technical Document Editor will display an icon for the node associated with the referenced document in the Structure Tree.



Figure 4. Example of Locked Referenced Content

Hovering the mouse over the icon will display the name of the User who has the Technical Document claimed or locked. Any referenced document content that has been changed and not saved will also cause the display of an icon in the Structure Tree as an indication that the content has not been saved.

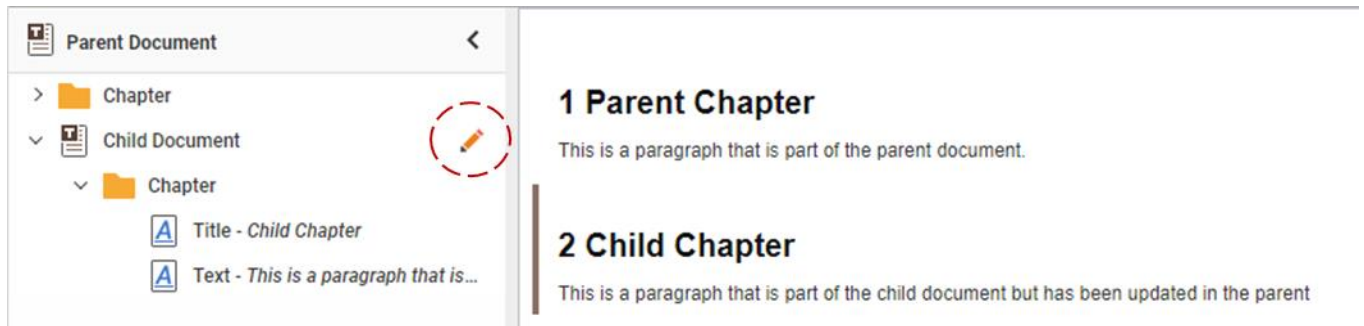


Figure 5. Example of Unsaved Referenced Content

2.4 CSS Styles

The format and placement of Document Element content in a technical document is largely controlled by the CSS settings configured with the associated Document Type.

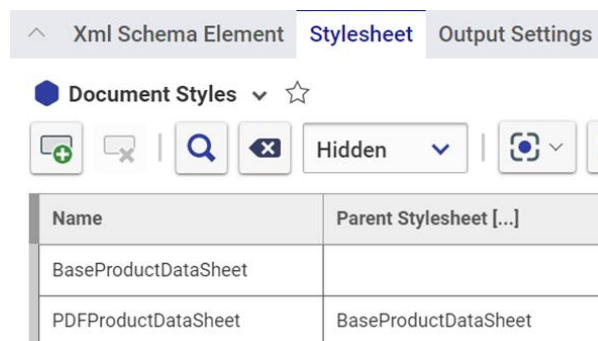


Figure 14: Cascading Stylesheet Items

There can be multiple Stylesheet Items configured and each one can extend (cascade) from others through the **Parent Stylesheet** selection.

When the content a Technical Document is rendered in the Editor or in the HTML published format, each Document Element in the document is converted to a <div> HTML element with a class set equal to the Document Element name. For example: <div class="Text">...</div>.

Note: Table Document Elements are converted to <table>, <tr>, and <td> elements. List Document Element are converted to , , and elements. Image Document Elements use elements.

2.4.1 CSS Example

The example Technical Document:

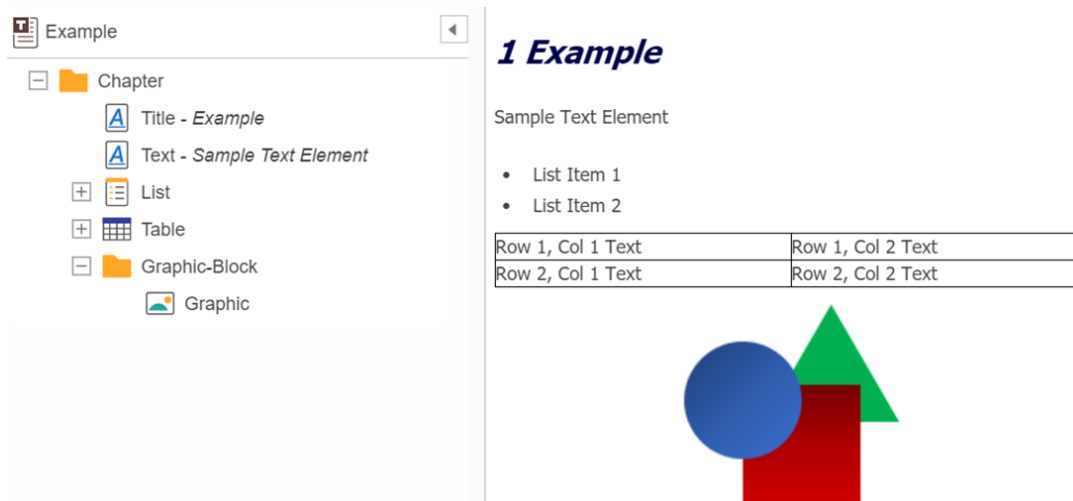


Figure 15: Sample Technical Document

... will result in HTML like:

```
<body>
  <div documentId="...">
    <div class="block ArasXmlElement" blockId="..." by-reference="internal">
      <div class="Chapter ...">
        <div class="Title ... ArasTextElement">
          <span class="emph ... ArasStringElement">Example</span>
        </div>
        <div class="Text ... ArasTextElement">
          <span class="emph ... ArasStringElement">Sample Text Element</span>
        </div>
        <ul class="List ... ArasListElement" type="bullet">
          <li class="List-Item ... ArasListItemElement" bulletListItem">
            <div class="Text ... ArasTextElement">
              <span class="emph ... ArasStringElement">List Item 1</span>
            </div>
          </li>
          <li>...</li>
        </ul>
        <table class="Table ... ArasTableElement" ColWidth="50|50">
          <colgroup><col width="50%" /><col width="50%" /></colgroup>
          <tr class="Row ... ArasTableRowElement">
            <td class="Entry ... ArasTableCellElement" valign="top" align="left">
              <div class="Text ... ArasTextElement">
                <span class="emph ... ArasStringElement">Row 1, Col 1 Text</span>
              </div>
            </td>
            <td>...</td>
          </tr>
          <tr>...</tr>
        </table>
        <div class="Graphic-Block ...">
          
        </div>
      </div>
    </div>
  </body>
```

Figure 16: Sample Generated HTML

The Example above contains instances of common Document Elements (i.e., Text, Table, List, Image). The XML content (not shown) is like the HTML. The top-most HTML element is a <div> that represents the entire document and contains an attribute for the Technical Document Item Id. The second level is a single <div> used to represent the content of the document. It is assigned a class of 'block'.

The Document Elements placed into the Document start at the third level of <div>s. Notice that class value contains the **name** of the Document Element as assigned in the Document schema. It also contains **class names** assigned based on the XML Elements that are extended in the Document Type schema. Also notice that Formatted Text Document Elements ('Text' in this example) have embedded <emph> tags around the text content. These are used to assign the formatted style (Bold, Italic, Underline, etc.) assigned in the Technical Document Editor.

2.4.2 Defining a Cascading Stylesheet

CSS content is defined using the CSS Editor, accessed from the Style Items added to the Document Type:

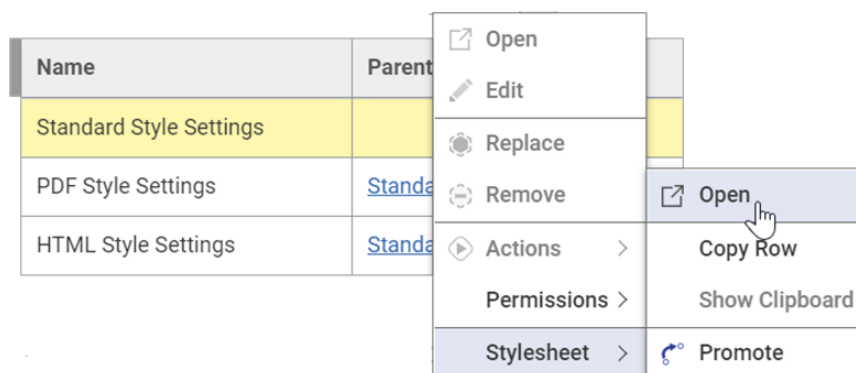


Figure 17: Opening a Stylesheet Item



Figure 18: Stylesheet User Interface

The CSS Form Editor is configured to display CSS content with syntax highlighting to match the elements of the content. Note that CSS selectors using the class name is the preferred method to identify the generated HTML elements.

Note: This document assumes the reader's familiarity with CSS and the methods used to isolate specific elements within HTML.

The editor form element can be extended/resized by dragging the handle at the lower right corner.

2.4.3 CSS Formatting Examples

This section contains examples for setting CSS attributes for various types of Document Elements. It is meant to provide a guide for customizing the layout and style of the reader's content.

2.4.3.1 Identifying a class of Document Element

HTML Elements can be identified in a CSS Selector by the standard element name (e.g., div, p, li, etc.), using an element's ID, or by a class name. The class name selector is the approach used for Technical Documents. The class selector uses a dot '.' followed by the name of the class. As described in this document, the name of the Document Element is stored within the class attribute so it can be used as a mechanism to target the application of CSS parameters.

The following selector is used to declare properties for the entire HTML body:

```
body {
    text-align: left;
    font-family: "Tahoma", Geneva, sans-serif
}
```

The following selector is used to declare properties for the Document Element with name 'Text':

```
.Text {
    color: #505050;
}
```

In the first example, the HTML Element name is used. The second example uses the class style selector is used (not the preceding '.') to select *all* Document Elements with the name 'Text'.

The following selector is used to declare properties for the Document Element with name 'Text' that are within a Table:

```
.Table .Text {
    color: #FF0000;
}
```

In this example above the first part of the Selector identifies the elements with a class of 'Table', followed by a space, followed by the Selector for elements of class 'Text'. In this case, any Text Document that is a descendant of a Table Document Element will use the Property Declaration.

```
.Row:nth-child(1) {
    background-color: #ccc;
    text-align: left;
    font-weight: bold;
}
```

In this example above, any content that is a descendant of the first instance of a Row Document Element will use the Property Declaration. Such a format is used to modify the style for text in the first row of a table.

In this last example, alternating rows in a table will have different background colors:

```
.Row:nth-child(even) {
    background-color: #ddd;
}

.Row:nth-child(odd) {
    background-color: #eee;
}
```

2.4.3.2 Centering Images

Centering images within the Editor, HTML output, and PDF output can be configured with CSS like:

```
.Graphic img {
    display: block;
    margin-left: auto;
    margin-right: auto;
}

img.Graphic {
    display: block;
    margin-left: auto;
    margin-right: auto;
}
```

This example defines a Graphic Document Element, named 'Graphic'. All image content uses the `img` HTML Element for displaying the image. However, the Technical Document Editor uses a format like:

```
<div class="Graphic">
  <img .../>
</div>
```

The HTML output format used for HTML and PDF however uses a format like:

```
<img class="Graphic".../>
```

Therefore, both declaration formats are required to center an image in the Editor and in visual output formats. In the top declaration, the `img` element are targeted as children of other elements with a class = 'Graphic'. The bottom declaration targets `img` elements with a class of 'Graphic'.

2.4.3.3 Using Counters

Technical Documents rely on CSS Counters for automatic numbering used for Chapter, Section, Figures, and Lists. CSS Counters use Properties that are implicitly defined in the CSS content to manage counter values. Accessor functions within the CSS language can then be used to increment and reference Counter Property Values. The following example show the process of declaring, incrementing, and accessing Counter Property values:

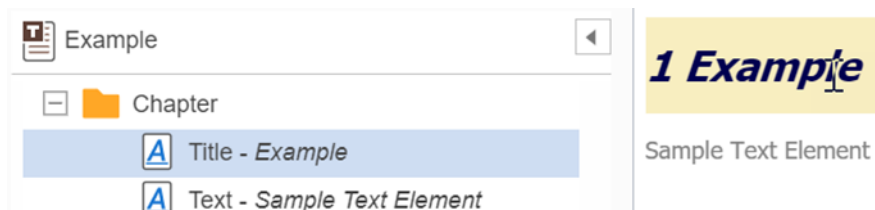


Figure 19: Chapter Numbering Example

```
body {
    counter-reset: sectionNum; /* Counter used for Chapter / Section numbering */
}

.Chapter {
    counter-increment: sectionNum;
}

.Chapter > .Title::before {
    content: counter(sectionNum) ' ';
}
```

In this example a Counter Property – ‘sectionNum’ is declared (using ‘counter-reset’) within the body declaration. It is incremented (using ‘counter-increment’) within the Chapter class declaration and used (via the counter() function) with the ‘before’ pseudo-element for the ‘Title’ Document Element that is an immediate child of a ‘Chapter’ Document Element.

Counter Properties must be declared before they are incremented or used/accessed. In this case, the CSS declaration for body ensures that the Counter is declared before any ‘Chapter’ elements are encountered.¹ Each time the HTML process encounters elements with a class = ‘Chapter’ it will increment the value of the counter (counters start at ‘0’ for numerical counters). Once an element with a class name of ‘Text’ is encountered, that is a direct child of a ‘Chapter’ element, it will precede the content for the ‘Text’ Document element with the value of the Counter followed by a single space.

Note: Using CSS Counters is an advanced topic. Readers should consult with CSS Reference guides for a full description for how to apply this functionality.

2.4.3.4 Adding Attributes to customize Document Element Style

In this example, an Attribute (named ‘size’) is added to an Image Document Element (named ‘Graphic’) in the Document Type schema:

```
<xs:element name="Graphic">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:imageType">
        <xs:attribute name="size" type="ImageSizeAttType"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:simpleType name="ImageSizeAttType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="small"/>
    <xs:enumeration value="medium"/>
    <xs:enumeration value="large"/>
  </xs:restriction>
</xs:simpleType>
```

¹ CSS is processed top down. Thus, the first <body> element will be processed before and descendant <div class='Chapter'> elements.

The values for the 'size' Attribute are restricted to the values: 'small', 'medium', and 'large', as defined by the 'ImageSizeAttType' Simple Type defined in the XML Schema. The values can be set for the size Attribute for any 'Graphic' Document Element using the Attributes Dialog in the Technical Document Editor:

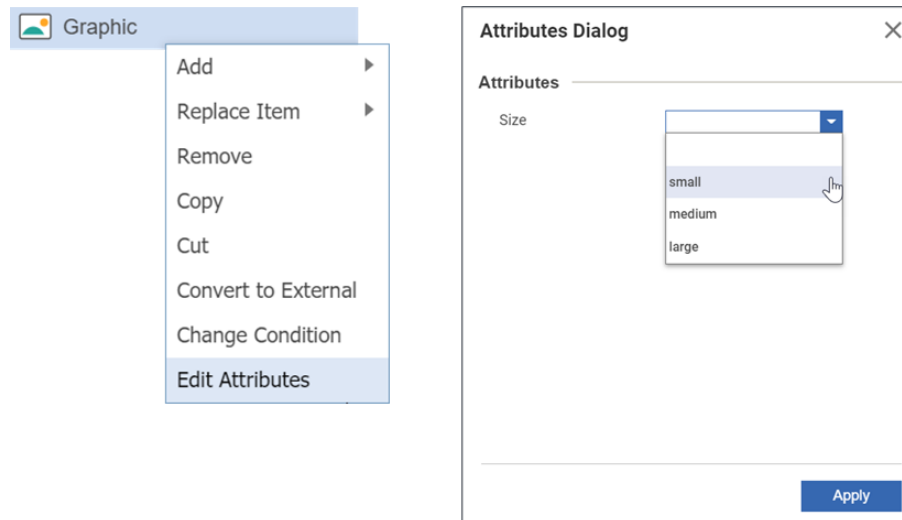


Figure 20: Image Size Attribute Example

CSS selectors can then be defined to isolate 'Graphic' elements with size Attributes with specific values as follows:

```
img[size='small'].Graphic, .Graphic[size='small'] img
{
    max-width: 200px;
}

img[size='medium'].Graphic, .Graphic[size='medium'] img
{
    max-width: 400px;
}

img[size='large'].Graphic, .Graphic[size='large'] img
{
    max-width: 600px;
}
```

In this example the CSS Attribute Selector is defined for each of the Attribute values to set the maximum image width for each. In this manner, when the author selects certain values, the size of the image is updated accordingly. There is no default value specified for the 'size' Attribute so empty values will default to the default size of the image.

Note: HTML content for images is generated differently for the Editor than for the HTML and PDF output.

2.5 Publishing Technical Documents

Aras Technical Documentation solution provides the capability to export the contents of a Technical Document as XML, HTML, or PDF. This part of the solution is available to Aras's subscribers.

Publishing also requires that the Aras Conversion Server and Aras Agent Service be configured properly.

2.5.1 Required Installations

To publish Technical Documents, the following needs to be installed:

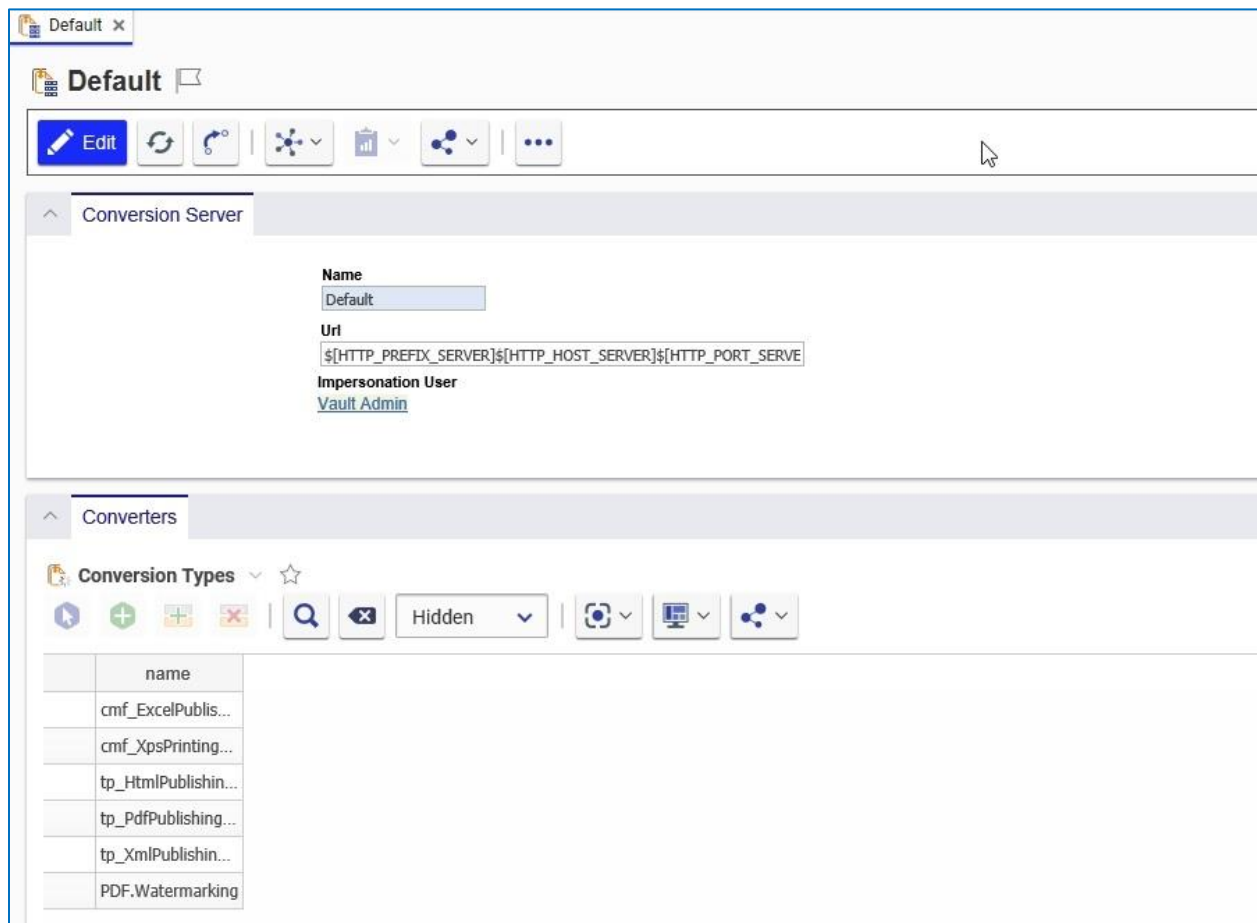
- Aras Conversion Server (See Aras Innovator - Conversion Server Setup Guide)
- Aras Agent Service (See Section 5.13 of the Aras Innovator – Installation Guide)
- Aras Publishing Service (See Aras Innovator – Publishing Service Setup Guide)
- Aras HTML to PDF Converter (See Aras Innovator – HTML to PDF Converter Setup Guide)

Publishing Technical Documents requires two Feature Licenses – Aras.PublishingService and Aras.HTMLtoPDFConverter.

2.5.2 Managing Conversion Rules

By default, the conversion rules required for Aras Technical Documentation Publishing are added to the **Default** Conversion Server. If your Innovator setup requires the use of a different Conversion Server, the following Conversion Rules should be added to the **Converters** tab on the appropriate **Conversion Server** item in the database:

- tp_HtmlPublishingConverter
- tp_PdfPublishingConverter
- tp_XmlPublishingConverter



2.5.3 Output Settings

The Output Settings map a stylesheet, configured in the **Stylesheet** tab (Section 2.4), to a selected Output Type: *Editor*, *HTML*, or *PDF*. Although it may be preferable to have document content to appear the same in all three, there are specific CSS settings that only apply to paged content (e.g, PDF) and Administrators may choose to have certain content styled differently in the Editor.

Note: There are default style settings that are applied for each Output Type. Setting the style for each will apply the configured stylesheet in addition to the default settings.

To create an Output Type, Edit the associate Document Type and add a row in the Output Setting tab:

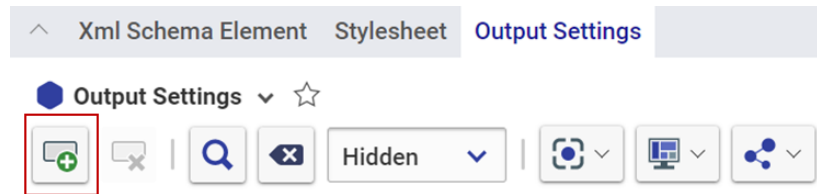


Figure 21: Create Output Setting

Select the Output Type:



Figure 22: Output Type

Note: 'Xml' is not a valid Output Type for the Output Settings. XML content is always displayed as ASCII content with no configurable styling.

Specify a unique Name for the Output Type. Optionally, a Classification for the associated Tech Doc Enabled ItemType (e.g., tp_Block) can be selected. In this case, the selected Stylesheet will only apply for Documents that have this Classification specified. Finally, choose the Stylesheet created in Section 2.4.

2.5.4 Style Settings

For HTML and PDF output formats, and when multiple Output Settings are configured, the user has the ability to select one of the Output Style Settings when publishing. This capability allows a user to publish the same Technical Document with different styles/configuration based on a choice made in the Publishing Dialog.

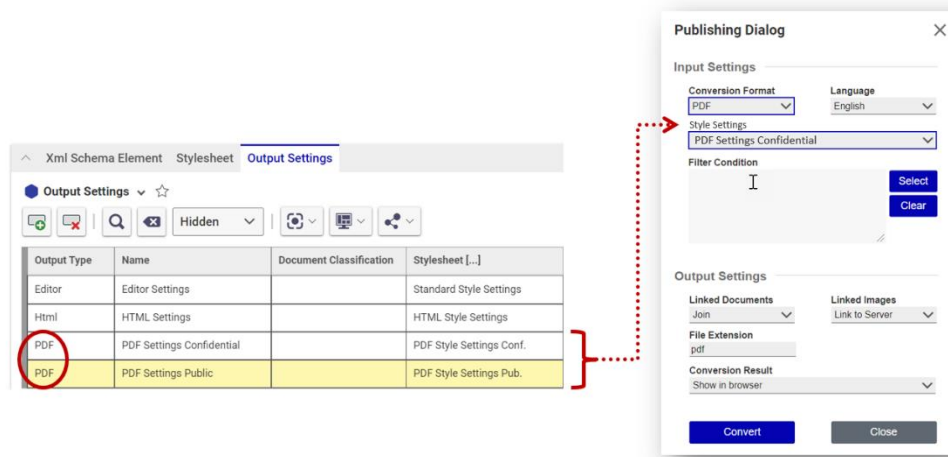


Figure 1. Style Selection for Published Content

The **Style Settings** form field will be enabled, and the drop-down list populated, whenever there has been more than one Output Setting configured for the selected **Conversion Format**. In these cases, the initial/default selection is based on the order of the Output Settings configured for the Document Type. Selecting the drop-down list will expose all other Output Settings. Selecting a specific Style Setting will result in the selected Output Setting to be applied during the publish operation. If there is only one, it will be selected by default and the Style Settings drop-down will be disabled.

2.6 Key Handling

Key handling in the Document Editor refers to how the editor responds to key events (e.g., Enter, Backspace, Delete, etc.). Although an author would seldom notice how key event handling is executed, how these events are processed are important in a structured document editor. This is because how key events are processed can significantly improve the user experience and provide more efficient authoring.

It is possible to enable extended key-handling logic, or create custom key-handlers, using CUI. This section describes the process for enabling extended key-handling logic to improve the authoring user experience by forcing the editor to behave more like common text editing or word-processing applications.

2.6.1 Enabling Extended Key-Handlers

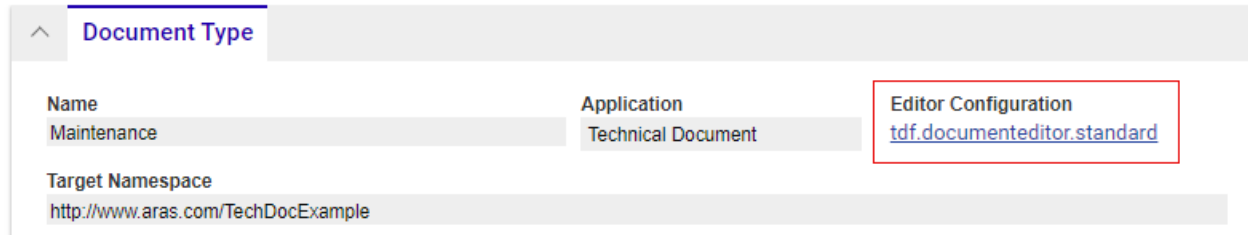
A Document Type can be configured to reference OOTB key-handling logic that alters the standard functionality of the Document Editor for Technical Documents that reference that specific Document Type. This will affect how the Delete, Backspace, and Enter key functions execute for Text and List / List Item Document Elements.

For Text Elements, selecting the Enter Key while editing text within a sentence will automatically create and append a separate Text Document Element and place the cursor at the beginning of the line. Any text that existed after the cursor will be removed from the previous Text Document Element and pasted in the new Text Document Element. Similarly, when a Backspace Key is selected while the cursor is at the start of a Text Element will automatically cut the text from the current Document Element, paste it at the end of the previous Text Document Element, and remove the current Text Document Element. This operation is only done if there exists a preceding Text Document Element. The Delete Key executes in a similar way but only when the cursor is at the end of a Text Document Element and there exists a

following, peer Text Document Element. Thus, these actions execute in a manner similar to how paragraphs are automatically split or merged in a typical text editor.

For Text Document Elements that are the last child of a List Item Document Element, selecting the Enter Key will result in the creation of a new List Item Element, with a Text Document Element child and place the cursor in the next Text Document Element. Any text that was after the previous cursor location will be cut and added to the new Text Document Element. Thus, these actions execute in a manner similar to how items in a list are created in a typical text editor.

These additional key handling features need to be configured for each Document Type as they are not added by default. To do so, add the 'tdf.documenteditor.standard' Presentation Configuration to the Editor Configuration property of the Document Type.



Name	Application	Editor Configuration
Maintenance	Technical Document	tdf.documenteditor.standard

Target Namespace
<http://www.aras.com/TechDocExample>

Note: The key handling logic in the 'tdf.documenteditor.standard' Presentation Configuration will process Text Document Elements and List Item Document Elements only.

Warning When adding the Presentation Configuration, be sure to select the one with the name 'tdf.documenteditor.standard'. There may be a Presentation Configuration instance with the name 'tdf.editorconfiguration.standard [14.0.29 obsolete]'. This uses a version of the TDF Client API that is no longer supported with versions of Aras Innovator 33 and above.

3 Customizing Technical Documents

It is possible to customize Technical Documents to suit business requirements. Most customizations include modifications to the **tp_Block** ItemType.

The screenshot shows the configuration interface for the **tp_Block** ItemType. The interface includes a toolbar with buttons for Edit, Refresh, and other actions. Below the toolbar, there are several sections for configuring the ItemType:

- Name:** tp_Block
- Singular Label:** Technical Document
- Plural Label:** Technical Documents
- Show Parameters Tab:** Never
- Class Structure:** A button to specify the class structure.
- Versioning:** Versionable (checked), Discipline (Manual), Revisions (Default).
- Search:** Auto Search (checked), Default Page Size (50), Max Records.
- Implementation Type:** Single Item (selected), Poly Item, Federated Item.
- Other settings:** Unlock On Logout, Dependent, Is Relationship, Enforce Discovery, Use Src Access, Allow Private Permissions, Enable for Secure Social.

Below these settings is a **Properties** table with columns for Name, Label, Data Type, Data Source, Length, Precision, Scale, Required, Unique, Indexed, Hidden, Hidden2, Alignment, Width, Sort Order, Keyed Name, and Order By.

Name	Label	Data Type	Data Source	Len	Preci	Scale	Requ	Unique	Indexed	Hidden	Hidden2	Alignment	Width	Sort Or	Keyed Name	Order By
classification	Type	String		512			☐	☐	☐	☐	☐	Left	200	3968		
condition	Condition	Text					☐	☐	☐	☒	☒	Left		3328		
config_id		Item	tp_Block				☒	☐	☐	☒	☒	Left		2688		
content	content	Multilingual String		4000			☐	☐	☐	☒	☒	Left		128		
created_by_id		Item	User				☒	☐	☐	☒	☒	Left		640		
created_on		Date					☒	☐	☐	☒	☒	Left		512		

Figure 2.

Possible customizations include:

- **Specifying a class structure:**
Aras Technical Documentation Application includes the ability to specify XML schemas and CSS styles that apply to specific classes. It is possible to specify a class structure for **Technical Documents** to fit business requirements.
- To specify a required class structure, open the **tp_Block** ItemType item for editing and click the **Class Structure** button.
- **Changing Permissions and Life Cycles:**
The Aras Technical Documentation solution comes with default Life Cycle and a set of Permissions. These may be edited to suit business requirements.
- Permissions and Life Cycles can be changed on the **tp_Block** ItemType.
- **Adding properties:**
It is possible to add properties to Technical Documents to suit business requirements.
- Required properties should be added to the **tp_Block** ItemType.
- **Adding items that a Technical Document can reference:**
It is possible to add/edit items that a **Technical Document** can reference. The list of allowed ItemTypes is controlled by the **tp_Item** Poly Item.

To add items that a **Technical Document** can reference, add the appropriate ItemType to **tp_Item** as a Poly Source.

Name	Label	Data Type	Data Source [...]	Len...	Preci...	Scale	Requi...	Unique	Indexed	Hidden	Hidden2	Alignment	Width	Sort Or...	Keyed Name ...	Order By
classification	Classification	String		512			☐	☐	☐	☒	☒	Left		256		
config_id		Item	tp_Item				☒	☐	☐	☒	☒	Left		2816		
created_by_id		Item	User				☒	☐	☐	☒	☒	Left		768		
created_on		Date					☒	☐	☐	☒	☒	Left		640		
css		Text					☐	☐	☐	☒	☒	Left		2432		
current_state		Item	Life Cycle State				☐	☐	☐	☒	☒	Left		1408		
revision_id		Item					☐	☐	☐	☒	☒	Left		768		

Figure 3.

Note: Adding new ItemTypes to **tp_Item** may also require updates to **Content Generator** methods. See Section 4 for more details.

3.1 Enabling the Ability to Edit Child Documents

By default, referenced document content within a *parent* technical document is not editable. However, it is possible to enable editing for this *child* content directly. Doing so will allow authors to edit all document content, that they have the access rights to, within a single editor instance regardless of whether this content exists within a separate Technical Document Item.

Parent Document

- Chapter
 - Title - Parent Chapter
 - Text - This is a paragraph that is part ...
- Child Document
 - Chapter
 - Title - Child Chapter
 - Text - This is a paragraph that is...

1 Parent Chapter

This is a paragraph that is part of the parent document.

2 Child Chapter

This is a paragraph that is part of the child document

Figure 4. Example Technical Document with Referenced Document Content

In the example figure, a Technical Document – Parent Document – includes referenced document content – Child Document. Referenced document content is distinguished visually by a solid bar

displayed to the left of all content within the referenced document. In addition, when referenced document is editable, the content will be displayed in the same font color and opacity as all other content that exists within the opened Document. By contrast, when referenced document content is not editable, it will be displayed in a different opacity, which appears to the user as a slightly different color.

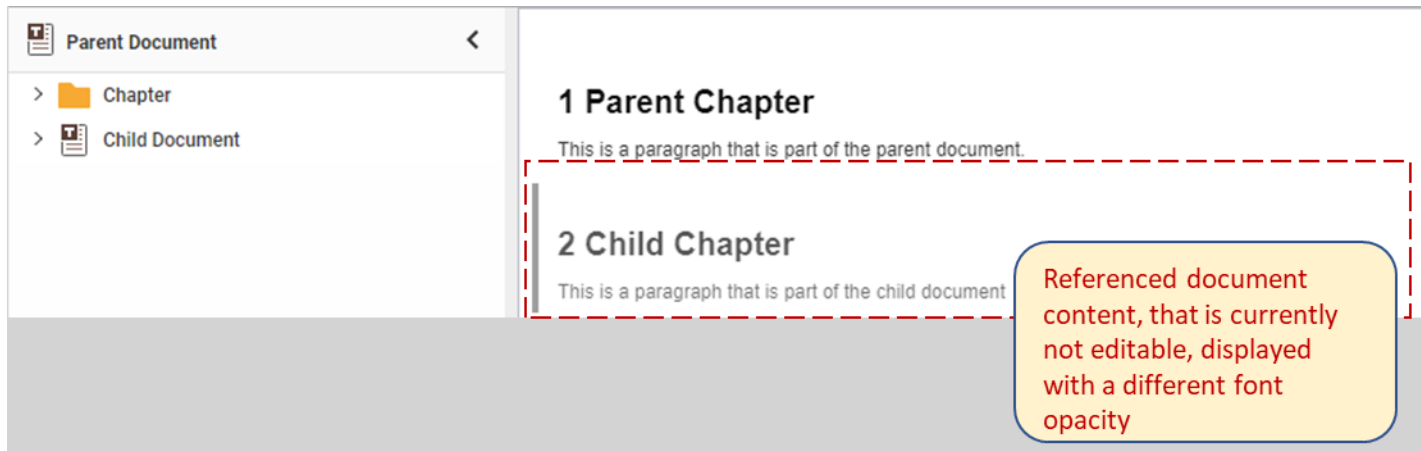


Figure 5. Example Technical Document with Non-editable Referenced Document Content

3.1.1 Edit Modes

Enabling external editing can be done in one of 2 Modes: Implicit or Explicit. Implicit Edit Mode will allow an author to edit referenced content without the need to *explicitly* lock that content for editing. Explicit Edit Mode will require an author to lock that content before any edits are made. Referenced Documents would then need to be unlocked by the author when changes are completed to make that Technical Document Item available for other users to edit. The Edit Mode will apply to all Technical Documents, for any Document Type.

3.1.2 Configuring an Edit Mode

The Edit Mode for a Technical Document is determined by the value of the Variable with name '**tp_ExternalEditMode**'. By default, the value of this Variable is set to '**None**', which means that no referenced document content can be edited. Setting the value to '**Explicit**' enables the Explicit Edit Mode. Setting the value to '**Implicit**' enables the Implicit Edit Mode. Users will need to re-login for any changes to take effect.

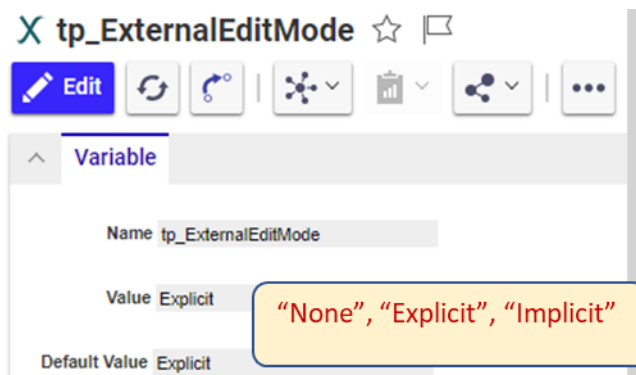


Figure 6. tp_ExternalEditMode Variable Setting

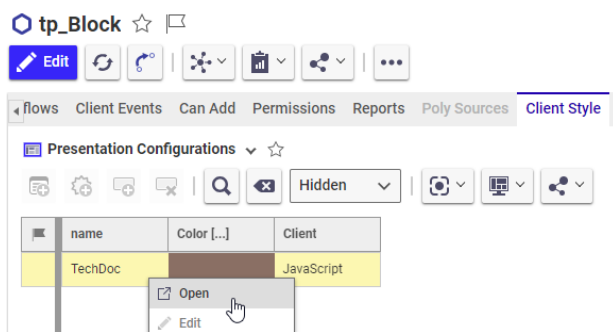
3.1.3 Update Presentation Configuration

In addition to setting the Edit Mode Variable, it's important to check the Presentation Configuration for the `tp_Block` ItemType to ensure that the proper Method Logic is set when saving a Technical Document. This functionality is necessary for Implicit Mode to properly warn the Author when changes have been made to referenced content after the content within the opened document was last opened or saved.

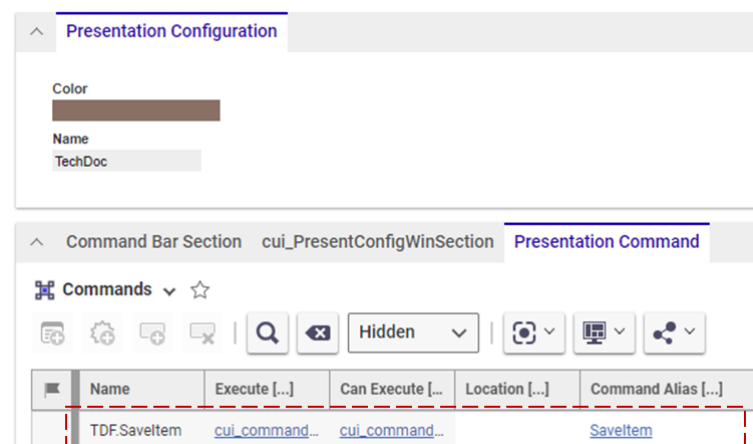
Warning Without the Presentation Configuration settings, Authors who save changes to referenced content in Implicit Mode will overwrite any changes made by another user without any warning.

Changes to the Presentation Configuration must be done by an Administrator. To check or ensure that the proper Presentation Configuration changes are set:

1. Open the `tp_Block` ItemType and select the Client Style Tab



2. Open the related 'TechDoc' Presentation Configuration
3. Set, or ensure, the TDF.Saveltem CUI Command as shown



Make sure that the Name and Command Alias are set as shown.

4 Content Generation

4.1 Overview

The placement of Document Elements within a technical document can trigger the execution of a 'Content Generator'. Content Generators are server-side Methods (C#) that can utilize a software API for creating Document Elements programmatically. Content Generators can apply whatever custom logic is necessary, including information about the document that the Document Elements will be inserted into, to automate the generation of technical document data.

Content Generators are typically configured to execute on the placement of an Item Document Element (See Section 2.3.2.2) to extract information about the selected Item and use that to populate content in the generated Document Elements.

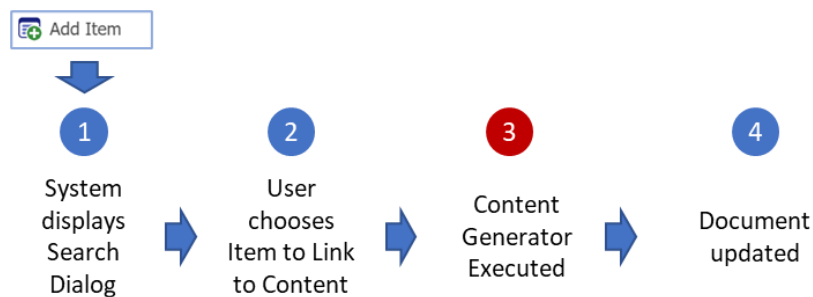


Figure 23: Content Generator Execution

The execution of Content Generators can be parameterized to provide additional flexibility/configurability to Technical Document Administrators. This section will describe the process of configuring a Content Generator.

4.2 Content Generator Example

This section provides an example Content Generator that constructs a table consisting of a row for each child BOM Part of a selected Assembly. The output will be similar to the following:

PartBOMTable

PartBOMTableHeader

Text - Item #

Text - Name

Text - Make/Buy

Text - Type

PartItemRow

Text - mc-27

Text - Intake Valve-1

Text - Make

Text - Component

PartItemRow

PartItemRow

PartItemRow

PartItemRow

Item #	Name	Make/Buy	Type
mc-27	Intake Valve-1	Make	Component
mc-28	Intake Lifter 2 Bottom-1	Make	Component
mc-29	Intake Lifter 1 Bottom-1	Make	Component
mc-30	Intake Camshaft-1	Make	Component
mc-31	Intake Lifter 2 Top-1	Make	Component
mc-32	Intake Lifter 1 Top-1	Make	Component
mc-33	Exhaust Valve-1	Make	Component
mc-34	Exhaust Lifter 2 Top-1	Make	Component
mc-35	Exhaust Lifter 2 Bottom-1	Make	Component
mc-36	Exhaust Camshaft-1	Make	Component

Figure 24: Content Generator Output Example

The top-most Item Document Element, with name 'PartBOMTable' contains two child Document Elements. The first is 'PartBOMTableHeader'; which is a Container Document Element for Text Document Elements that will provide the top-most header row. The second is another Item Document Element, with name 'PartItemRow' that also contains Text Document Elements with values for selected Part Properties. The Document Type XML Schema is as follows:

```
<xs:element name="PartBOMTable">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:itemType">
        <xs:sequence maxOccurs="unbounded">
          <xs:element ref="PartBOMTableHeader" minOccurs="1" maxOccurs="1"/>
          <xs:element ref="PartItemRow" minOccurs="0" maxOccurs="unbounded"/>
        </xs:sequence>
        <xs:attribute name="typeId" type="xs:string" fixed="Part ItemType ID"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:element name="PartBOMTableHeader">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="Text" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

<xs:element name="PartItemRow">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:itemType">
        <xs:choice maxOccurs="unbounded">
          <xs:element ref="Text" minOccurs="0" maxOccurs="unbounded"/>
        </xs:choice>
        <xs:attribute name="typeId" type="xs:string" fixed="Part ItemType ID"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

For more information on how to construct Document Type schema see Section 2.3.

The source code for the Content Generator that produces this output is as follows:

```
//MethodTemplateName=CSharp:Aras.TDF.ContentGenerator(Strict);
ItemDocumentElement targetItem = targetElement as ItemDocumentElement;

if (targetItem != null) {
  targetItem.ClearChilds();

  String AML = @"<AML>"
    + "<Item type='Part BOM' action='get'>"
    + "  <source_id>" + targetItem.ItemId + "</source_id>"
    + "  <related_id>"
    + "    <Item type='Part' action='get' />"
    + "  </related_id>"
```

```

        + "</Item>"
        + "</AML>";

        DocumentSchemaElement headerElement = (DocumentSchemaElement)
this.Factory.NewElement("PartBOMTableHeader");
        TextDocumentElement hd1 = (TextDocumentElement) this.Factory.NewText("Text", "Item #");
        TextDocumentElement hd2 = (TextDocumentElement) this.Factory.NewText("Text", "Name");
        TextDocumentElement hd3 = (TextDocumentElement) this.Factory.NewText("Text", "Make/Buy");
        TextDocumentElement hd4 = (TextDocumentElement) this.Factory.NewText("Text", "Type");
        headerElement.AddChild(hd1);
        headerElement.AddChild(hd2);
        headerElement.AddChild(hd3);
        headerElement.AddChild(hd4);
        targetElement.AddChild(headerElement);

        Item bomItems = this.Factory.InnovatorInstance.applyAML(AML);
        int numItems = bomItems.getItemCount();

        // Loop through each Catalog Entry and fill the remaining table rows
        for(int i=0; i < numItems; i++)
        {
            Item bomItem = bomItems.getItemByIndex(i);
            Item partItem = bomItem.getPropertyItem("related_id");
            ItemDocumentElement itemElement = (ItemDocumentElement)
this.Factory.NewElement("PartItemRow");
            // set item reference
            itemElement.SetItem(partItem);

            TextDocumentElement numText = (TextDocumentElement) this.Factory.NewText("Text",
partItem.getProperty("item_number"));
            TextDocumentElement nameText = (TextDocumentElement) this.Factory.NewText("Text",
partItem.getProperty("name"));
            TextDocumentElement mbText = (TextDocumentElement) this.Factory.NewText("Text",
partItem.getProperty("make_buy"));
            TextDocumentElement typeText = (TextDocumentElement) this.Factory.NewText("Text",
partItem.getProperty("classification"));
            itemElement.AddChild(numText);
            itemElement.AddChild(nameText);
            itemElement.AddChild(mbText);
            itemElement.AddChild(typeText);

            targetElement.AddChild(itemElement);
        } // for() loop over catalog entries
    } // if (targetItem != null)

```

1. The first line of a Content Generator Method must identify the Method Template to use as a comment. In this case, it should be specified as it is in this example –

```
//MethodTemplateName=CSharp:Aras.TDF.ContentGenerator(Strict);
```

2. The Content Generator in this example is configured with an Item Document Element – **PartBOMTable**. The parameter 'targetElement' is passed to the Content Generator Method as a more generic DocumentElement object. This line casts the object to the expected ItemDocumentElement type. The target Element represents the instance of the Document Element that the Content Generator is executed against.

3. The check for null is necessary to assure that the cast on the previous line was successful. Trying to cast to a Document Element Type class that is not represented by the type of Document Element will cause an error and the variable will be assigned `null`.
4. Content Generators can be executed multiple times, either because they are configured as dynamic or because the author explicitly reruns (refreshes) the Document Element. In these cases, the Content Generator Method is executed again on the associated Document Element. Clearing all previously generated content is therefore necessary. Note that the *grammar* of the function is wrong. `'ClearChilds()'` will remove all child content from the document.
5. The Document Element for the first child element for **PartBOMTable** is created using the `Factory Object`. The `DocumentSchemaElement` class is used for Container Document Elements. The name of the Document Element (**'PartBOMTableHeader'**) is passed to the constructor.
6. For this example, the **PartBOMTableHeader** Document Element contains Text Document Elements as its children. The next 4 lines create a new `TextDocumentElement` for the 'labels' of the Part BOM Table. The `TextDocumentElement` constructor is passed the name of the Text Document Element (in this case – 'Text') and the text used for the content.
7. Each Text Document Element is added to the parent **PartBOMTableHeader**
8. The **PartBOMTableHeader** is added to the target Element.
9. An AML Query is executed to extract the Part Items for the Item linked to the target Element.
10. A loop to extract each Part Item.
11. Each Part will be linked to a newly generated Item Document Element with the name **PartItemRow**.
12. The Item object returned from the execution of the AML is used to set the link for the **PartItemRow** Item Document Element. This will create a separate link for each row and likewise, a separate Relationship from the Document to the Part Item.
13. Like the header, new Text Document Elements are added with text extracted from the selected Part properties.
14. These Item Document Elements are added to the target Element.

In summary, all Content Generator Methods should:

1. Set the appropriate Method Template
2. Create a variable that is cast to the appropriate type
3. Clear the content, if necessary
4. Modify the target Element and/or add child content as appropriate

4.3 Content Generator Configuration

The Content Generator Method is specified in the Document Type (see Section 2). Content Generators can be configured as 'dynamic' or 'static'. *Static*, the default, will result in the Content Generator executing when the Document Element is added to the document or when the author explicitly refreshes the Document Element using the context menu for the associated Document Element. *Dynamic* is set if the check box is selected in the Document Type. Dynamic Content Generators are executed when the Document Element is created or refreshed, but also when the Document is opened or saved. Depending on the complexity of the Content Generator, setting the Dynamic flag can affect performance.

4.3.1 Content Generator Parameters

Content Generator Methods can be passed parameters (or any type of input data) defined as a JSON-formatted string. Such an approach can be used to configure the execution of the Content Generator. The JSON is specified on the XML Schema Element for the specific Document Element.

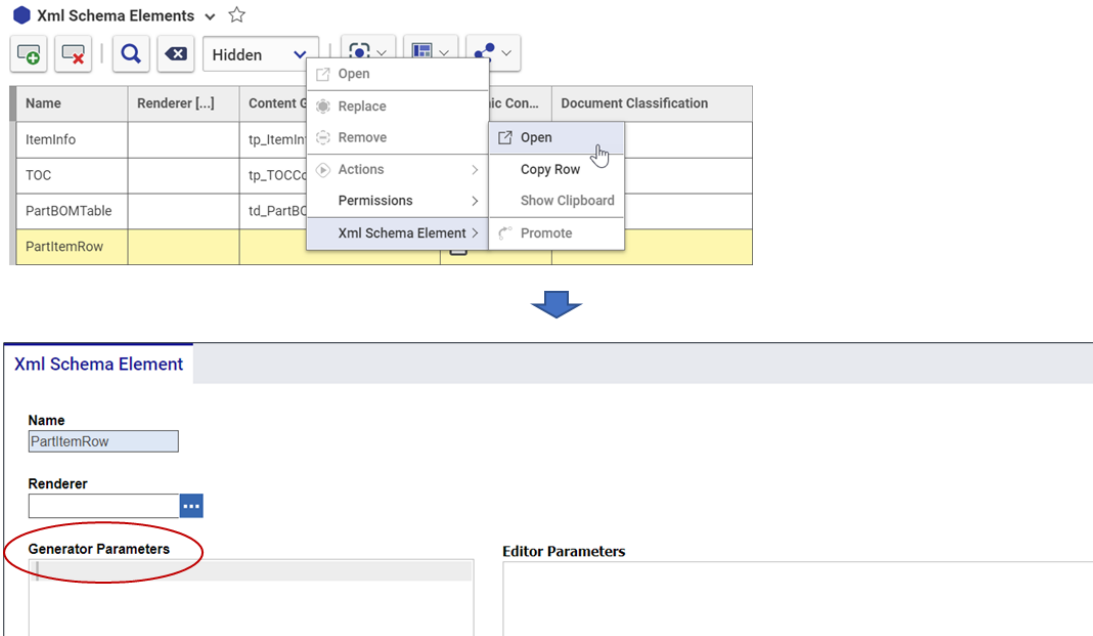


Figure 25: XML Schema Element Form - Content Generator Parameters

The **Generator Parameters** input form is configured for displaying JSON content. Syntactical errors will be displayed if the text content does not conform to JSON formatting. It is important that the content be properly formatted JSON data, or it will not be converted to content that can be accessed in the C# Content Generator Method.

Note: JavaScript Object Notation (JSON) is used to serialize JavaScript Object data. It is used in the Technical Document Framework to enable complex configuration data for parameterizing Methods. A description of JSON, including how JSON can be represented via Collection classes in .Net, is outside the scope of this document.

4.3.2 Content Generator Example – Name / Value Pair

The following example shows how to configure and parse a single name/value pair for a Content Generator. In this example, a parameter 'type' can have one of two values: 'product' or 'shipping'. The JSON formatted string is stored in the UI as follows:

Generator Parameters

```
{
  "type": "product"
}
```

Figure 26: Content Generator Parameter Example - Name / Value Pair

The text is for this must be:

```
{
  "type": "product"
}
```

Or

```
{
  "type": "shipping"
}
```

Note the open and closing brackets ('{' and '}'). The Content Generator Method can retrieve this information using the following example:

```
// The configured parameters should contain a single object with a
// type property with values of either 'product', or 'shipping' like:
//{"type": "product"}
if (!this.GenerationParameters.ContainsKey("type"))
{
    throw new InvalidOperationException("'type' generation parameter was not configured.");
}

string type = (string)this.GenerationParameters["type"];
```

1. Uses the public object `GenerationParameters` to check for the existence of the `type` variable. If it does not exist, it throws an `InvalidOperationException` with an appropriate message. If parameters are required, a similar mechanism should be used to warn the Technical Document Administrator that the Content Generator is not configured properly.
2. The value for the `type` parameter is extracted as a string to be used in the subsequent logic of the Content Generator. Casting is required.

4.3.3 Content Generator Example – Lists

The following example shows how to configure and parse a list of name/value pairs for a Content Generator. In this example, a parameter 'list' can have multiple objects; each consisting of a name and value. The JSON formatted string for this is as follows:

```
{
  "list": [{ "name": "Prop 1", "value": "Prop 1 Value" },
           { "name": "Prop 2", "value": "Prop 2 Value" },
           { "name": "Prop 3", "value": "Prop 5 Value" } ]
}
```

Note the open and closing brackets ('{' and '}') and the use of brackets for each list item. Variable names and values are specified as quoted strings. The Content Generator Method can retrieve this information using the following example:

```
// The configured parameters should contain an array labeled 'list'
// that contains objects with name/value properties, like:
//{
//  "list": [{ "name": "Prop 1", "value": "Prop 1 Value" },
//           { "name": "Prop 2", "value": "Prop 2 Value" },
//           { "name": "Prop 3", "value": "Prop 5 Value" } ]
//}
if (!this.GenerationParameters.ContainsKey("list"))
{
```

```

        throw new InvalidOperationException("'list' generation parameter was not configured.");
    }

    // Create List Items based on given Name/Value pairs
    System.Collections.Generic.List<object> list = (System.Collections.Generic.List<object>)
    this.GenerationParameters["list"];
    foreach (System.Collections.Generic.Dictionary<string, object> listItem in list)
    {
        ...
        string name = (string)listItem["name"];
        string value = (string)listItem["value"];
        ...
    }

```

1. Uses the public object `GenerationParameters` to check for the existence of the `list` variable. If it does not exist, it throws an `InvalidOperationException` with an appropriate message. If parameters are required, a similar mechanism should be used to warn the Technical Document Administrator that the Content Generator is not configured properly.
2. The value for the `list` parameter is extracted as a `Generic.List` to be used in the subsequent logic of the Content Generator. Casting is required.
3. Loop over the contents of the List. Note the use of a `Generic.Dictionary` in this example.
4. The name and value are extracted using array notation.

4.4 Content Generator API

The Content Generator API provides access to the same core classes that the system uses to generate technical document content. It consists of a class hierarchy with extensions for each of the main Document Element Types (See Appendix 7.1).

5 Item Reference Configuration

5.1 Overview

Item Reference Configuration provides the ability to assign JavaScript Methods that get invoked when a referenced Item is created through the process of inserting an instance of an Item Document Element (see Section 2.3.2.2) or via file Drag and Drop (as is the case with Image Files). These JavaScript Methods provide the logic to pre-populate Property values on the newly created Item and validate Property values added/updated by the user in the Item Form.

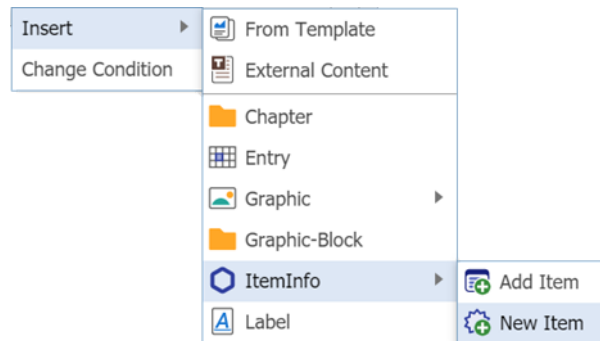
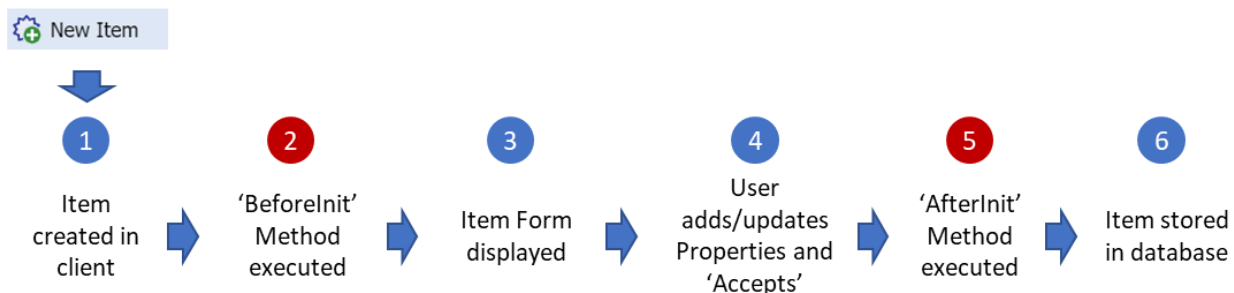


Figure 27: Generating a New Referenced Item

As described in Section 2.3.2.2, Item Document Elements contain a link between a Document Element in a Technical Document and an Item (Business Object) maintained in Innovator. Often, Properties from the associated Item (or related Items) are used to populate content within the Document. In addition to selecting an existing Item, the Technical Document Editor allows the creation of the referenced Item at the time the Item Document Element is placed. Doing so often requires input from the author to populate Properties on the Item before the record is created in the Database. When configuring a Document Element, it is possible to assign a JavaScript Method that will be executed before the Item Form is presented to the author and after the Form is 'Accepted' to generate the Item in the database and establish the reference between it and the Technical Document. The process of creating a referenced Item is as follows:



1. The Authors selects the 'New Item' context menu for the Item Document Element. The system will generate a new referenced Item in the client. Note the same process can be initiated when dragging an Image file from the file system on to the canvas of a Technical Document that is opened for Edit. In this case, an tp_Image ItemType will be created and referenced.
2. The configured JavaScript Method is called to pre-populate Properties on the Item
3. The associated Item Form is presented to the Author/User

4. The author visually validates the content and/or updates as required and selects the 'Accept' Button
5. The configured JavaScript Method is called to validate the Properties on the Item
6. The Item is stored in the database

The reference to the newly created Item is established with the Technical Document once the Technical Document is saved. In the case where an Image File has been dragged onto the Editor canvas, an Image Document Element, of the appropriate type, will be created and added to the document with the image displayed.

5.2 Configuration

The configuration options for Item Reference Configuration allow:

- The identification of an alternate Form to be used for Property Updates
- A Method used to pre-populate Item Properties
- A Method used to validate Item Properties
- The option to bypass the display of the Item Form

This configuration is specified using JSON for the XML Schema Element associated with the Item Document Element.

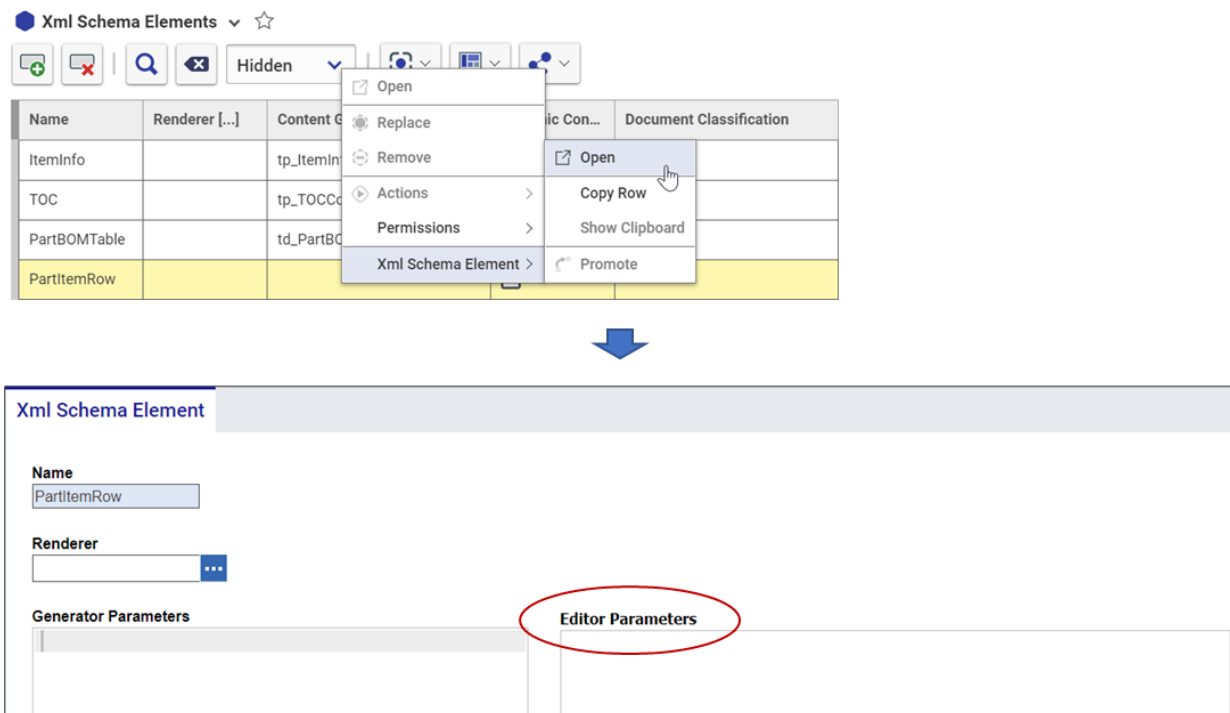


Figure 28: XML Schema Element Form

The **Editor Parameters** input form is configured for displaying JSON content. Syntactical errors will be displayed if the text content doesn't conform to JSON formatting. To set the Item Reference Configuration, a single JSON object is used with the following properties:

```
{
```

```
"itemCreation": {
  "formId": "<id of Form Item to use as alternate>",
  "onBeforeInit": "<beforeInit Method Name>",
  "onAfterInit": "<afterInit Method Name>",
  "initializerType": "<initializer type>",
  "fileAccept": "<true/false>",
  "allowedMIMETypes": "<[comma-delimited array of mime type text]>",
}
```

Note: Additional configuration options may be added in the future. The use of JSON provides a flexible means of specifying configuration options without the need for ItemType modifications in future releases

Table 1: Item Reference Configuration Options

JSON Property	Description
itemCreation	JSON Object Name for Item Creation options.
formId	(Optional): Identifies an alternate Form to use for specifying Item Properties. The default Form is used for the referenced ItemType if this is not used.
onBeforeInit	(Optional): Specifies the name of a JavaScript client Method to be called prior to the Form displayed to the User.
onAfterInit	(Optional): Specifies the name of a JavaScript client Method to be called after the Form displayed to the User.
initializerType	(Optional): Accepts values: 'base' and 'formDialog' when used for non-Image ItemType references. 'formDialog' is the default and is used when a Form should be displayed to the author/user. 'base' is used to suppress the Form display. When 'base' is specified the before <i>and</i> after initialization Method will still be executed if they are specified. When this Property is used for Image Document Elements for file Drag and Drop support, a value of 'coreLocalFileImage' should be used to generate a tp_Image Itemtype.
fileAccept	Used for Drag and Drop of Image Files to auto-create Image Document Elements. When 'true', the system will accept image file drag/drop onto the Editor to auto-create an Image Document Element. 'false' (or not included) to not allow image creation via Drag and Drop
allowedMIMETypes	(Optional): list of image MIME Types that are allowed. If not included, all image types supported by the system will be allowed.

5.2.1 onBeforeInit Method

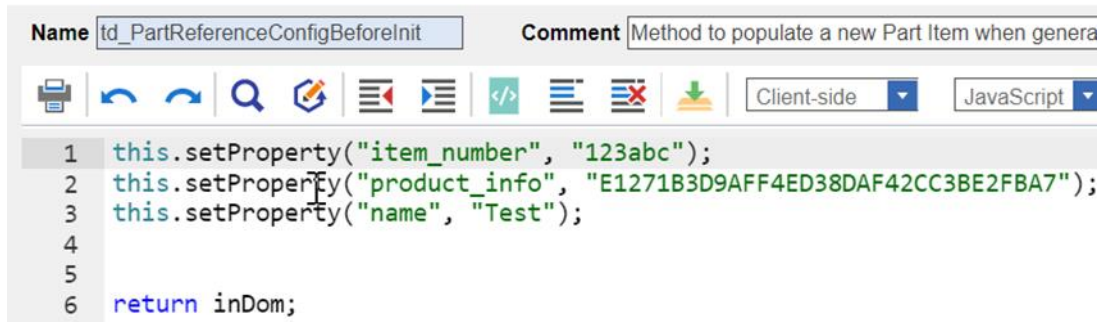


Figure 29: onBeforeInit Method Example

The purpose of the **onBeforeInit** Method is solely to pre-populate selected Properties of the Item being created. It is meant to provide a convenient mechanism to ensure consistent data for Items and should execute quickly, without excessive memory or processing. The **onBeforeInit** Method is a JavaScript Method that will be passed two parameters – **inDom** and **inArgs** – and must return the **inDom** input object. The context object of the Method is the **Item** being created. Thus '*this*' refers to the Item Object and the standard IOM Item API applies. The **inDom** parameter represents the XML content of the Item being created also. As such, methods like `aras.setItemProperty(inDom, ...)` will work as well. The **inArgs** parameter is an object with two properties: **documentItem** and **viewModel**. **documentItem** contains the XML representation of the Technical Document Item in which the Document Element is being created. This object can be used to retrieve information about the Technical Document Item. Likewise, methods like `aras.getItemProperty(inArgs.documentItem, ...)` will work. The **viewModel** object represents the API for the Technical Document Editor.

Warning The **viewModel** object is not part of a supported API. Methods/Properties can be modified without notice in subsequent releases and is to be used at the customer's risk.

i) onAfterInit Method



Figure 30: onAfterInit Method Example

The purpose of the **onAfterInit** Method is solely to validate selected Properties of the Item being created. It is meant to provide a convenient mechanism to ensure consistent data for Items and should execute quickly, without excessive memory or processing. The **onAfterInit** Method is a JavaScript Method that will be passed two parameters – **inDom** and **inArgs** – and must return the **inDom** input object. The context object of the Method is the **Item** being created. Thus '*this*' refers to the Item Object and the

standard IOM Item API applies. The `inDom` parameter represents the XML content of the Item being created also. As such, methods like `aras.getItemProperty(inDom, ...)` will work as well. The `inArgs` parameter is an object with two properties: `documentItem` and `viewModel`. `documentItem` contains the XML representation of the Technical Document Item in which the Document Element is being created. This object can be used to retrieve information about the Technical Document Item. Likewise, methods like `aras.getItemProperty(inArgs.documentItem, ...)` will work. The `viewModel` object represents the API for the Technical Document Editor.

Warning The `viewModel` object is not part of a supported API. Methods/Properties can be modified without notice in subsequent releases and is to be used at the customer's risk.

If there is a validation error, an alert/message should be displayed, and `null` should be returned. When 'base' is used for the **initializerType** in the configuration, returning `null` will prevent the Item from being created.

6 Document Filters

Filters provide a mechanism for Technical Document Authors to associate Characteristics (metadata) with Document Elements and use these characteristics to automatically filter the content. This is useful when technical documents can serve multiple, but related, purposes. For example, a technical document that describes several similar models of the same part. Content that is specific to any model (or models) can have filters associated with it. When an end user wishes to see only the content that is either associated with all models or is particular to a specific model, they can identify the appropriate filters and the Technical Document Editor will display the correct content accordingly.

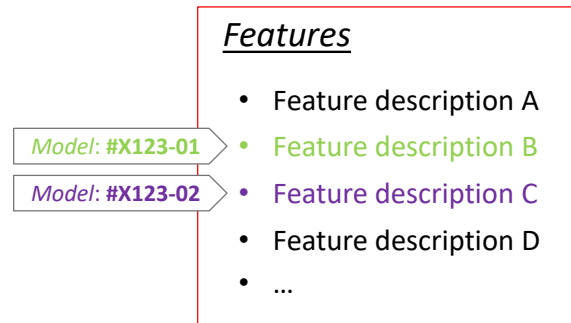


Figure 31: Filter Example

Referring to the figure above, the list of features associated with Model 'X123-01' is A, B, and D. The list of features associated with Model 'X123-02' is A, C, and D. The list of available *Filter Names* and associated *Filter Values* is set in the Method `tp_GetOptionFamilies`.

6.1 Creating the Filter List

The Method `tp_GetOptionFamilies` determines the set of Filters (Options) by returning a JSON-formatted string containing a name/value list where the *name* specifies the Name of the Filter/Option, and the *value* specifies the list of valid values. The default implementation for this Method is as follows:

```
/*
var optionFamilies = {
    "Region": [ "Africa", "Asia", "Central America", "Eastern Europe", "European
Union", "Middle East", "North America", "Oceania", "South America", "The Caribbean"],
    "Zone": [ "A", "B", "C"],
    "Model": [ "X", "Y", "C"]
};
*/
Dictionary<string, List<string>> optionFamilies = new Dictionary<string, List<string>>
();

optionFamilies.Add("Region", new List<String>() { "Africa", "Asia", "Central America",
"Eastern Europe", "European Union", "Middle East", "North America", "Oceania", "South
America", "The Caribbean" } );
optionFamilies.Add("Zone", new List<String>() { "A", "B", "C" } );
optionFamilies.Add("Model", new List<String>() { "X", "Y", "Z" } );

String json = new
System.Web.Script.Serialization.JavaScriptSerializer().Serialize(optionFamilies);

return this.getInnovator().newResult(json);
```

The Method uses a `Dictionary` class to construct the data for the returned JSON data structure. It is possible to return a JSON-formatted string as well using the example pattern described by the comment at the top of the method code. Implementations can use this example Method as a guide for constructing static Filters, stored the list data in custom Items and retrieve the values in this Method, and/or apply whatever custom logic is necessary to construct a dynamic set of data.

7 Appendix

7.1 Content Generator API

This section describes the Document Element classes that represent an Application Programming Interface (API) for implementing a Content Generator. Each subsection describes the class for each Document Element Type along with an example implementation and helpful information for the use of each.

7.1.1 Table Document Element

7.1.1.1 Code Sample

```
// create table with 4 rows and 4 columns
TableDocumentElement tableElement = (TableDocumentElement) this.Factory.NewTable("table", 4, 4);
// or this.Factory.NewTable();
// or this.Factory.NewTable("table");
// merge first cell to the right
tableElement.MergeCells(0, 0, MergeDirection.Right);
// merge third cell in the second row to the left, twice
tableElement.MergeCells(1, 2, MergeDirection.Left);
tableElement.MergeCells(1, 2, 3);
// merge last cell in second row with cell below, twice
tableElement.MergeCells(1, 3, MergeDirection.Down);
tableElement.MergeCells(1, 3, 2);
// unmerge first cell
tableElement.UnmergeCells(0, 0);
// delete last row deletion
tableElement.RemoveRow(tableElement.RowCount - 1);
// add new row at the end
tableElement.AddRow();
// add new row at the beginning
tableElement.AddRow(0);
// add new column at the end
tableElement.AddColumn();
// add new column before the second
tableElement.AddColumn(1);
// remove third column
tableElement.RemoveColumn(2);

targetElement.AddChild(tableElement);
```

7.1.1.2 Additional Information

- If table element Name is not explicitly passed in the `NewTable()` constructor, then first appropriate schema element will be used.
- Factory method `NewTable()` takes as parameters `rowCount` and `columnCount` values. If these parameters are omitted, then table will be created with 2 rows and columns (default value).
- In methods like `AddRow()`, `AddColumn()`, `RemoveRow()` and etc., Index parameter can be omitted. In this case method will be applied to the last row or column. Table must have at least one row and column.

- In method `MergeCells()` parameter `mergeDirection` can take one of the enumeration values: (`MergeDirection.Up`, `MergeDirection.Right`, `MergeDirection.Down`, `MergeDirection.Left`) or integer value (0, 1, 2, 3), numbers correspond to the enumeration.

7.1.2 List Document Element

7.1.2.1 Code Sample

```
ListDocumentElement listElement = (ListDocumentElement) this.Factory.NewList("list1", 3);
// or this.Factory.NewList();
// or this.Factory.NewList("list1");
// set list style
listElement.ListType = ListTypes.Numeric;
// create new list item at the end
listElement.AddItem();
// create new list item at the beginning
listElement.AddItem(0);
// remove second item
listElement.RemoveItem(1);
targetElement.AddChild(listElement);
```

7.1.2.2 Additional Information

- If list element Name isn't passed in `NewList()` constructor, then first appropriate schema element will be used.
- Factory method `NewList()` takes as a second parameter `itemCount` value. If this parameter is omitted and there is no information about default children count in the schema definition, then list will be empty (there is no default value).
- In methods `AddItem()`, `RemoveItem()` parameter `Index` can be omitted. In this case method will be applied to the last list item.
- Property `ListType` can take one of the following enumeration values: `ListTypes.Bullet`, `ListTypes.Numeric`, `ListTypes.Alpha`.

7.1.3 Text Document Element

7.1.3.1 Code Sample

```
// create text element with «Hello World!» content
TextDocumentElement textElement = (TextDocumentElement) this.Factory.NewText("ptxt",
"Initial Text!\n");
// or this.Factory.NewText();
// or this.Factory.NewText("ptxt");
// replace all existing strings with a new one
textElement.Text = "Hello, World!\n";
// add new string
textElement.AddString("How are you, World?\n");
// add new string with «italic» style
textElement.AddString("Bye, World!\n", "bold");
// set style for second string
textElement.Strings[1].SetStyle("italic bold under strike");
// set style for substring
textElement.Strings[2].SetStyle("sub", 3, 7);
// set link on element of another document (root element in this case)
textElement.Strings[4].SetLink("1D93C56E1EBA4F6A9B68D348444C9C17",
"1D93C56E1EBA4F6A9B68D348444C9C17");
```

```
// remove link
textElement.Strings[4].RemoveLink();
targetElement.AddChild(textElement);
```

7.1.3.2 Additional Information

- Factory method `NewText()` has an optional second parameter `initialText` value.
- Text Document Elements contain `StringDocumentElements` as its children. Separate `StringDocumentElements` can be obtained with indexer property `Strings`.
`StringDocumentElement` `stringElement = textElement.Strings[1];`
 - `StringCount` property identifies the number of `StringDocumentElements`.
- Property `Text` returns concatenated content of all strings of the `TextDocumentElement`. If property used as a setter, then all current strings will be replaced by the given string.
- Method `SetStyle()` of `StringDocumentElement` takes text Style string as a parameter. It contains desired styles, which are delimited by the 'space' character. Additionally the `SetStyle()` method can take `startRange` and `endRange` parameters, in this case string may be split on substrings and total number of strings in the `TextDocumentElement` will be increased.

```
// after this command, the number of strings in the text Element will increase to two
textElement.Strings[2].SetStyle("sub", 3, 7);
```

- Method `SetLink()` takes `targetBlockId` (id of existing Technical Document) and `targetElementId` (id of any element in this Document). Currently will work properly for root document element and its children.

7.1.4 Image Document Element

7.1.4.1 Code Sample

```
ImageDocumentElement imageElement = (ImageDocumentElement) this.Factory.NewElement("graphic");
// get tp_Image item from Innovator
Item imageItem = this.Factory.InnovatorInstance.newItem("tp_Image", "get");
imageItem.setID("70CA6215D95C4658ACA68BFDB73E4DEE");
imageItem = imageItem.apply();
// set reference on image item
imageElement.SetImage(imageItem);
targetElement.AddChild(imageElement);
```

7.1.4.2 Additional Information

- Currently there is no explicit constructor for `ImageDocumentElement` in `SchemaElementFactory`. It should be created with base element constructor (`Factory.NewElement`).
- By default, `ImageDocumentElements` are empty. Method `SetImage()` can be used to set the image Item reference and takes an `Item` as a parameter. Currently `ImageDocumentElements` can have references only to `tp_Image` Items.

7.1.5 Item Document Element

7.1.5.1 Code Sample

```
ItemDocumentElement itemElement = (ItemDocumentElement) this.Factory.NewElement("aras-part");
// get item from Innovator
Item itemItem = this.Factory.InnovatorInstance.newItem("Part", "get");
itemItem.setID("CD2B48A0C3C042898AC00254930FC8A1");
itemItem = itemItem.apply();
// set item reference
itemElement.SetItem(itemItem);

targetElement.AddChild(itemElement);
```

7.1.5.2 Additional Information

- Currently there is no explicit constructor for `ItemDocumentElement` in `SchemaElementFactory`. It should be created with base element constructor (`Factory.NewElement`).
- By default, `ItemDocumentElements` are empty. Method `SetItem()` can be used to set the Item reference and takes an `Item` as a parameter. Currently `ItemDocumentElements` can have references only to `tp_Item` Polysource Items.

7.1.6 Item Property Document Element

Note: Item Property Document Elements are also referred to as *Mapped* Property Document Elements.

7.1.6.1 Code Sample

```
Item partItem = bomItem.getPropertyItem("related_id");
ItemDocumentElement itemElement = (ItemDocumentElement)this.Factory.NewElement("PartItemRow");
// set item reference
itemElement.SetItem(partItem);
ItemPropertyDocumentElement nameText = (ItemPropertyDocumentElement)
this.Factory.NewItemProperty("NameProperty", "name");
...
itemElement.AddChild(nameText);
```

7.1.6.2 Additional Information

- This example adds an `ItemPropertyDocumentElement` based on the following Schema Definition:

```
<xs:element name="NameProperty">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:itemProperty">
        <xs:attribute name="property" type="xs:string" fixed="name"/>
        <xs:attribute name="mode" type="aras:itemPropertyModeEnum" fixed="write"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

- Note that in the example above, the property attribute is fixed for the 'name' Property and the mode is fixed to 'write'. These are optional in the schema although it's useful to define default values.
- The example above includes the name of the Property in the `NewItemProperty` method. This is required even though it was defaulted to a value in the schema.

7.2 Configuration Examples

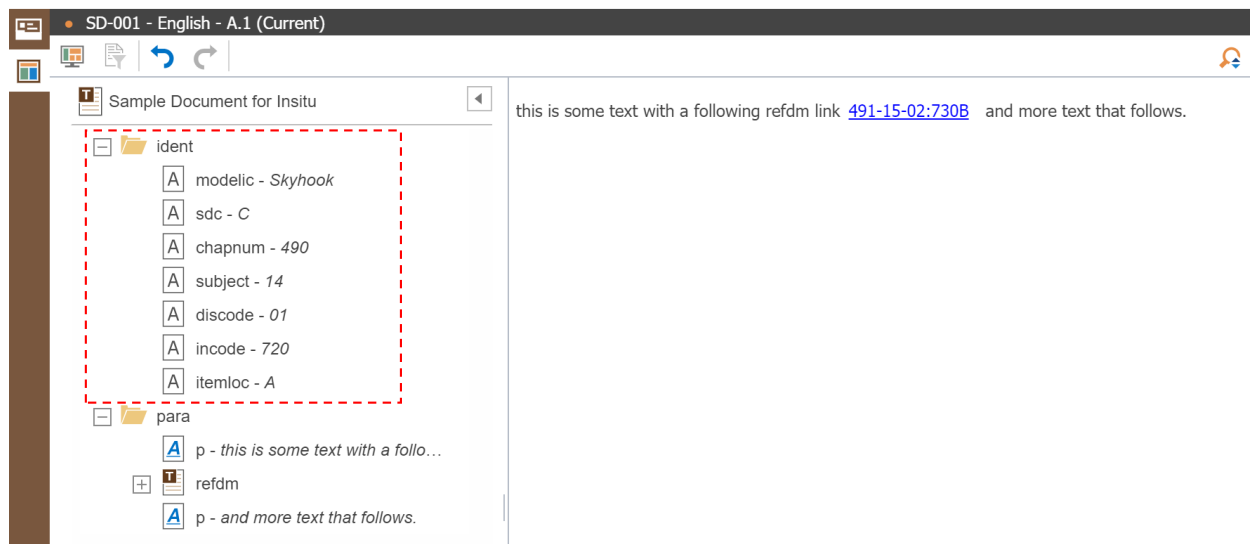
7.2.1 Overview

This section provides examples that describe how to configure a Tech Doc-Enabled Item to achieve a specified purpose. Each sub-section includes an Objective that identifies the purpose of the example.

7.2.2 Example – Auto-creating hidden elements from Tech Doc Item Properties

7.2.2.1 Objective

Show how Document Elements can be automatically created, using values from the active Technical Document instance which then are included, but not displayed in the editor. This example shows how an Administrator can configure the system to generate S1000D *ident*-like information at the beginning of a document by using S1000D Data Module (DM) parameters stored as Properties on the Technical Document. In this case, the DM can use the same 'Ident' information to identify the purpose of the Technical Document to authors who need to search for existing documents. This example uses XML Elements from an S1000D specification, but it can be used for any Properties added to the Tech Doc-Enabled ItemType.



7.2.2.2 Solution

Use a Content Generator for a container 'ident' Document Element that will extract parameter values from the associated Technical Document and add them to the selected node. Use CSS to target nodes within the container and hide the rendering of the corresponding content.

Note: regarding the creation of Content Generator Methods, the first commented line in the code is required - `//MethodTemplateName=CSharp:Aras.TDF.ContentGenerator(Strict);` It is used to identify the proper method template.

7.2.2.3 Content Generator

```
//MethodTemplateName=CSharp:Aras.TDF.ContentGenerator(Strict);
```

```
// a Document Element object representing the Document Element node that was
// placed in the document and associated with this Content Generator is passed
// into this Method call as 'targetElement'. The cast here is actually not
```

```
// necessary since it's the base class type for Document Elements. Container
// nodes will be instantiated as 'DocumentSchemaElement' objects. This is used
// for the 'ident' node.
DocumentSchemaElement targetItem = targetElement as DocumentSchemaElement;

// The Innovator instance is needed so that we can query Properties from the
// Technical Document Item associated with this ident node
Innovator inn = this.Factory.InnovatorInstance;

// The object 'executionContext' is passed into this Method and contains the
// id of the current document. Use it to query for the Tech Doc Properties
Item doc = inn.getItemById( "tp_Block", executionContext.DocumentId );
if(doc == null || doc.isError())
    return;

// The targetElement will be null if the case above failed. In this case, the
// input targetItem is not the Document Element Type expected
if (targetItem != null) {

    // Content Generators can be executed multiple times for a node by the user
    // This call will clear all children of this target node in case this has
    // already been called.
    targetItem.ClearChilds();

    // This should be empty
    if (!targetItem.IsEmpty)
    {
        // Use the doc Item to set properties for non-formatted text nodes
        // Note the call to the constructTextNode method below
        targetItem.AddChild(constructTextNode("modelic", doc.getProperty("modelic")));
        targetItem.AddChild(constructTextNode("sdc", doc.getProperty("sdc")));
        targetItem.AddChild(constructTextNode("chapnum", doc.getProperty("chapnum")));
        targetItem.AddChild(constructTextNode("subject", doc.getProperty("subject")));
        targetItem.AddChild(constructTextNode("discode", doc.getProperty("discode")));
        targetItem.AddChild(constructTextNode("incode", doc.getProperty("incode")));
        targetItem.AddChild(constructTextNode("itemloc", doc.getProperty("itemloc")));
    }
}
} // note the extra closing brace

// This method will construct a node used for non-formatted text in a Technical
// Document. Note that nodes of this type are different than formatted text
// nodes. In this case, the logic within this method is the only way to create
// nodes of this type
DocumentTextNode constructTextNode(string name, string value)
{
    XmlNode xmlNode = this.Factory.ConstructElementOrigin(name);
    xmlNode.InnerText = value;
    DocumentTextNode txtNode = new DocumentTextNode(this.Factory, xmlNode);
    return (txtNode);
}
// Note the absence of a closing node here
```

7.2.2.4 CSS

```
.ident .modelic, .ident .sdc, .ident .chapnum, .ident .subject, .ident .discode,
.ident .incode, .ident .itemloc
{
```

```
display: none;
}
```

All node names in a schema are used as class names in the generated HTML. As such, you can use the class designator – for example: ‘.ident’ – to refer to the corresponding element in a Technical Document. In this case, the child nodes (‘modelic’, ‘sdc’, ‘chapnum’, etc.) are all children of the ‘ident’ element. So, the example CSS rule - `.ident .modelic` – targets any node with a class of ‘modelic’ that is a descendent of an ‘ident’ node. For all these types, the CSS ‘display: none’ will cause the browser to hide the contents of the node. In this case, the text. Also, if a ‘modelic’ (or any of the others) were not included as a descendent of an ‘ident’ node, they *would* display.

7.2.2.5 Schema

```
<xs:element name="ident">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="modelic"/>
      <xs:element ref="sdc"/>
      <xs:element ref="chapnum"/>
      <xs:element ref="subject"/>
      <xs:element ref="discode"/>
      <xs:element ref="incode"/>
      <xs:element ref="itemloc"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

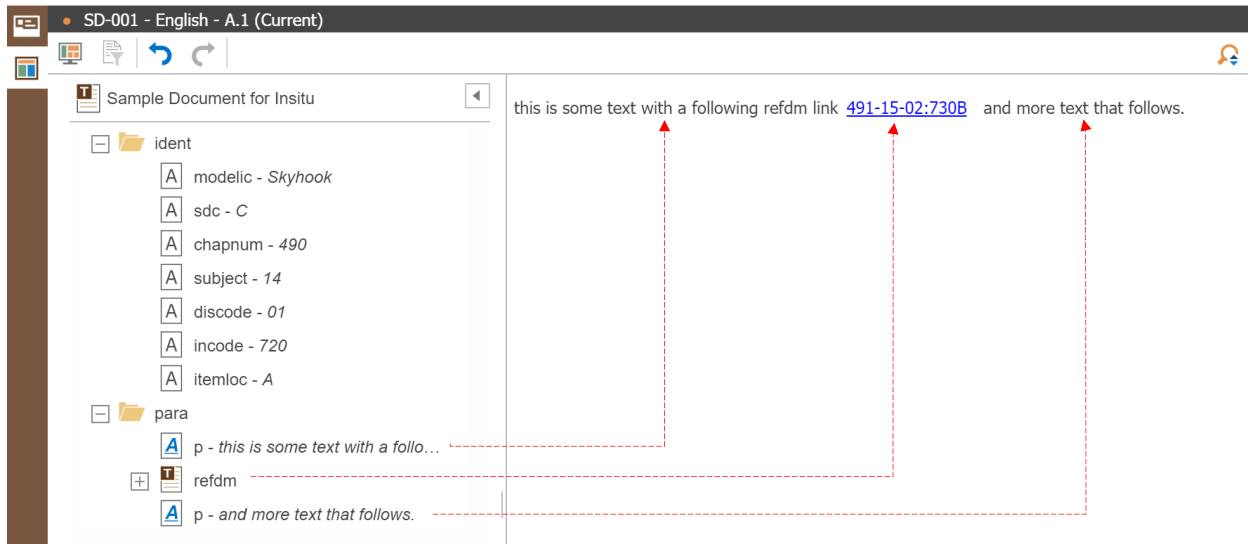
<xs:element name="modelic"><xs:complexType mixed="true"/></xs:element>
<xs:element name="sdc"><xs:complexType mixed="true"/></xs:element>
<xs:element name="chapnum"><xs:complexType mixed="true"/></xs:element>
<xs:element name="subject"><xs:complexType mixed="true"/></xs:element>
<xs:element name="discode"><xs:complexType mixed="true"/></xs:element>
<xs:element name="incode"><xs:complexType mixed="true"/></xs:element>
<xs:element name="itemloc"><xs:complexType mixed="true"/></xs:element>
```

The ‘ident’ node is a container. Each child element uses references (for example `ref="modelic"`) to identify the corresponding node. Note that these nodes do not extend the `aras:text` type and instead are declared as a complex type with ‘mixed=true’. These are treated as non-formatted text nodes in Tech Docs.

7.2.3 Example – Auto-creating inline link and text to external references:

7.2.3.1 Objective

Show how inline text content and a 'refdm' element, that is used to reference a separate Data Module (DM), can be authored within a Technical Document. S1000D content typically contains paragraphs of text with embedded nodes. In XML-speak, that is referred to as 'mixed content'. The Technical Document Framework does not support this behavior and nodes must either contain only child nodes or text; but not both. In addition, although links in Tech Docs can either refer to content within the same document or content within other Technical Documents, there is no out-of-the-box mechanism to automatically generate link text based on the content that is being referenced.



7.2.3.2 Solution

Use an 'Item Document Element' to show how a Document Element within a Technical Document can be used to reference another Technical Document Item in Innovator. Item Document Elements can be configured to always reference a particular ItemType. In this case, use the Technical Document ItemType ('tp_Block'). Use a Content Generator to query content from the referenced Item to automatically generate link text and use the Content Generator API to format the generated link text as a link to the referenced document. This example also necessitated the creation of a separate (and non-S1000D compliant) node – 'p' that will be used for inline text content. Note that the intent here is to use this node solely for inline text content. Use CSS to target p and refdm nodes within the container and force them to render as inline elements when rendered in HTML and PDF.

7.2.3.3 Schema

```
<xs:element name="para">
  <xs:complexType>
    <xs:choice maxOccurs="unbounded">
      <xs:element ref="p" minOccurs="0" maxOccurs="unbounded" />
      <xs:element ref="refdm" minOccurs="0" maxOccurs="unbounded" />
    </xs:choice>
  </xs:complexType>
</xs:element>

<xs:element name="p">
  <xs:complexType>
    <xs:complexContent>
```

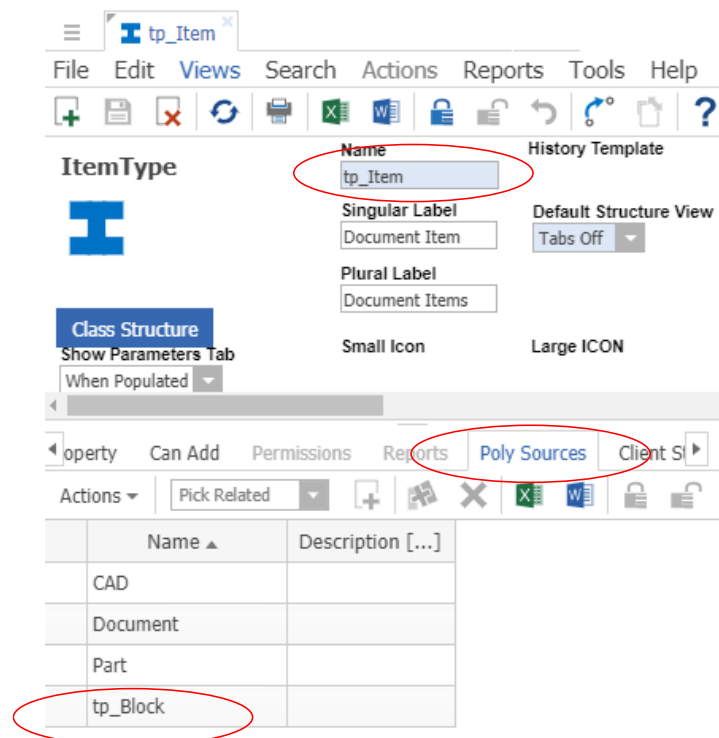
```

        <xs:extension base="aras:text"/>
    </xs:complexContent>
</xs:complexType>
</xs:element>

<xs:element name="refdm">
    <xs:complexType>
        <xs:complexContent>
            <xs:extension base="aras:itemType">
                <xs:choice maxOccurs="1">
                    <xs:element ref="p" minOccurs="1" maxOccurs="1"/>
                </xs:choice>
                <xs:attribute name="typeId" type="xs:string"
                    fixed="ED9F61ADA4334D7D94361F426C081DB5" />
            </xs:extension>
        </xs:complexContent>
    </xs:complexType>
</xs:element>

```

In this schema example, we are treating an S1000D `para` node as a container with child elements of type 'p' and 'refdm'. In this case, p elements can be interleaved with refdm elements without any restrictions on cardinality or order. The refdm node is configured here to extend the `aras:itemType` element. When this is done, Document Elements of this type can reference any ItemType specified in the `tp_Item` PolyItem. For this example, I added the Technical Document ItemType – `tp_block` – to this PolyItem. For example:



Also, in the schema definition, the addition of the `typeId` attribute for the `refdm` element will identify the Item ID for the ItemType to restrict references to. In this case, I added the ItemType ID for the `tp_Block` ItemType (ED9F61ADA4334D7D94361F426C081DB5): thus, targeting references to only a Technical

Document. *This is important because we make assumptions about the referenced Item on the Content Generator logic described below.*

7.2.3.4 CSS

```
.para .p, .para .refdm
{
    display: inline-block;
    padding-right: .5em;
}
```

These CSS rules target `p` and `refdm` classes only when they are descendants of a `para` element. The styling will render the content of these nodes as *inline* (instead of the default *block*). In addition, it adds some right padding to adequately space each `p` and `refdm` content.

7.2.3.5 Content Generator

```
//MethodTemplateName=CSharp:Aras.TDF.ContentGenerator(Strict);

// A Document Element object representing the Document Element node that was
// placed in the document and associated with this Content Generator is passed
// into this Method call as 'targetElement'. The cast here is necessary since
// the base class type for Document Elements is 'DocumentSchemaElement' However,
// this Method uses an Item Document Element since the refdm node is configured
// in the schema as an extension to 'aras:item'. When this Content Generator is
// called, the target Element will include a reference to a Technical Document
// that the user has chosen after they placed the refdm node in the document.
ItemDocumentElement targetItem = targetElement as ItemDocumentElement;

// The targetElement will be null if the case above failed. In this case, the
// input targetItem is not the Document Element Type expected
if (targetItem != null) {

    // Content Generators can be executed multiple times for a node by the user
    // This call will clear all children of this target node in case this has
    // already been called.
    targetItem.ClearChilds();

    // This should be empty
    if (!targetItem.IsEmpty)
    {
        // The target Item, being of type ItemDocumentElement, will have a
        // method (GetItemProperty) to retrieve a Property value of the referenced
        // Item. In this case, we want to build the link text based on
        // added Properties to the Technical Document ItemType (e.g., chapnum,
        // subject, etc.). This code builds that link text
        string linkText = targetItem.GetItemProperty("chapnum", " ") + "-";
        linkText += targetItem.GetItemProperty("subject", " ") + "-";
        linkText += targetItem.GetItemProperty("discode", " ") + ":";
        linkText += targetItem.GetItemProperty("incode", " ");
        linkText += targetItem.GetItemProperty("itemloc", " ");

        // This line will construct a child 'p' node that will be used for the
        // link text.
        TextDocumentElement pElement = (TextDocumentElement) this.Factory.NewText("p",
linkText);

        // This line will format the text content of the p node as a link
```

```
// and have it reference the same Technical Document Item the user has
// selected when they placed the refdm node. The first argument is the
// id of the Technical Document Item, the second is the id of the node
// within the document. Since the schema locked the referenced ItemType
// to a Technical Document, we know what we can create a link to it. The
// root node in a Technical Document has the same id as the document Item
pElement.Strings[0].SetLink(targetItem.ItemId,targetItem.ItemId);

// Add the p node as a child of the given refdm node
targetItem.AddChild(pElement);
}
}
```

7.2.4 Example – Using Attributes to apply styling

7.2.4.1 Objective

Provide a mechanism to allow authors to size images in a technical document. By default, images are placed left justified and sized to fit the maximum space within their container. This is typically the full width of the space minus any margins.

7.2.4.2 Solution

Use attribute values together with CSS selectors to target and apply a set of style rules for a selected image. This technique can be applied to any Document Element and any style configuration, including positioning. This example demonstrates two techniques. The first example will show how to use an attribute to identify a select rule. The second uses attribute values to set specific styles typed by the author. The first option is probably the most useful as it is easy for the end user. The second option can provide for greater flexibility (and unwanted consequences)

7.2.4.3 Option A

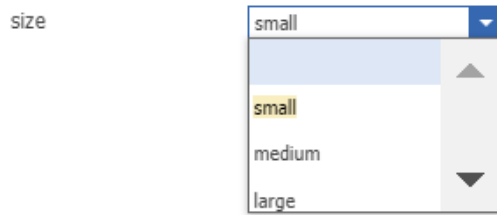
In this solution an attribute is added to the Image Document Element – Graphic. The attribute name is 'size' and there will be three allowed choices: 'small', 'medium', and 'large'. Here is the related schema:

7.2.4.4 Schema

```
<xs:element name="Graphic">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:imageType">
        <xs:attribute name="size" type="ImageSizeAttType"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:simpleType name="ImageSizeAttType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="small"/>
    <xs:enumeration value="medium"/>
    <xs:enumeration value="large"/>
  </xs:restriction>
</xs:simpleType>
```

In this sample, there is an added 'size' attribute to the `Graphic` Document Element and, using a restriction, specified the allowed values of 'small', 'medium', and 'large'. The Attribute editor will display these values in a picklist:



CSS:

```
.Graphic[size='small'] img
{
  max-width: 100px;
}

.Graphic[size='medium'] img
{
  max-width: 300px;
}

.Graphic[size='large'] img
{
  max-width: 600px;
}
```

These style rules target the `img` tag using the parent '`Graphic`' container with a '`size`' attribute of either 'small', 'medium', or 'large'. Of course, other values can be added as well. In addition, other styles can be applied. The example is really showing how the author can use attribute values to assign styling.

7.2.4.5 Option B

In this alternative example, an option is provided to allow the user to set a specific value for size. Here is the related schema:

7.2.4.6 Schema

```
<xs:element name="Graphic">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:imageType">
        <xs:attribute name="style" type="xs:string"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>
```

In this sample, there is an added 'style' attribute to the `Graphic` Document Element. Note the name 'style' is important here. The Attribute editor will display the attribute using a simple string editor:



In this example, we use the value of the attribute to identify a variable – ‘width’ in this case and its value. This is applied to the width style rule in the following CSS:

7.2.4.7 CSS

```
.Graphic
{
    width: var(width);
}
```

Of course, other style rules can be added as well. This example shows an approach that adds flexibility but requires the user to know more about what they’re typing – Use with caution.

7.2.5 Example – Populating PDF header content from attribute data

7.2.5.1 Objective

Automate the creation, placement, and content for a PDF page header using information from the technical document. In this example, the user wants to use information stored in the attributes of a particular Document Element in the PDF Header of a published document.

7.2.5.2 Solution

Use CSS rules and functions that apply to page-based media to create the PDF header and add static and custom content. Note that this example uses Attributes for the source for some of the content, but another element content can be used as well. For example, the text in a *Title* Document Element.

In this example, there is an added ‘chapnum’ and ‘modelic’ attribute to a Document Element named ‘ident’. These values can be populated by the Attribute Editor or added by a Content Generator like:

7.2.5.3 Content Generator

```
// Use Property values from the Technical Documents Item for attribute values
targetItem.SetAttribute("modelic", doc.getProperty("modelic"));
targetItem.SetAttribute("chapnum", doc.getProperty("chapnum"));
```

7.2.5.4 CSS

```
@page
{
    size: 8.5in × 11.0in;
    margin: 40pt 0 40pt 0;
    padding-bottom: 1em;
    padding-top: 1em;

    @top-right
    {
        content: "Test " string(modelic) ", " string(chapnum);
        color: white;
        font-family: Tahoma, Geneva, sans-serif;
        background-color: #8C0827;
        padding-right: 3em;
    }
}

.ident
{
    string-set: modelic attr(modelic), chapnum attr(chapnum);
}
```

}

The Style applied to the PDF output uses @page and the margin identifier @top-right to position the content for the header. In this case, the content is derived from two string sets: 'modelic' and 'chapnum'; which are populated from the attributes of the same name in the '.ident' class DOM elements using the attr() function. Note that only one string-set rule can be used in a group. In this case, we set the value of both string set variables separated by a comma.

7.2.5.5 PDF



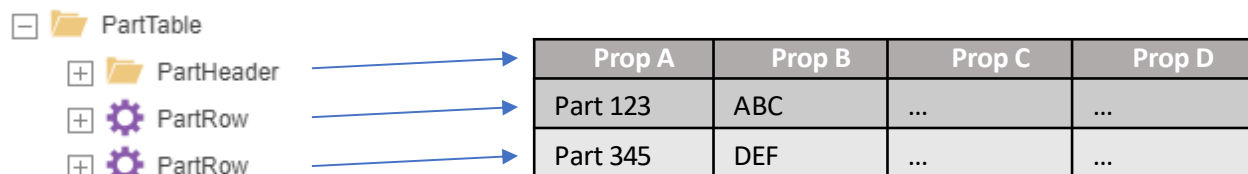
Here is the header portion of a generated PDF document. Note that the Header text is as follows:

Static Text – 'Test', followed by the value of the modelic attribute (in this case 'Skyhook'), followed by a comma and space – ', ', followed by the value of the chapnum attribute (in this case '490').

7.2.6 Example – Creating a Table, in which each row points to individual Parts

7.2.6.1 Objective

Provide a means of auto-generating Table Rows where each Row Object is an Item Document Element, 'pointing to' Part Items and the cells of the Row contain individual Part Item Property data.



7.2.6.2 Solution

For this solution, a typical Table Document Element can't be used because it derives from an aras:table element. The same is true for aras:row and aras:cell elements. Also, the first row of the table – the header row - needs to be auto-created with a cell for each column. Finally, the 'Row' Document Elements need to be 'ItemDocumentElements' since each added row should be associated with a selected Item (Part in this example).

7.2.6.3 Schema

For this structure, the top-level node – PartTable – is the container with a *required* first child of PartHeader, and any number of PartRows. Each PartRow has any number of PartCells; which contain a single Text Document Element. Note the use of default style attributes in this schema. They use 'display: table/table-row/table-cell' values, which when used within <div> style attributes treat the PartTable like a standard HTML table, PartHeader/Rows like table rows, and PartCells like table cells, etc.

```
<xs:element name="PartTable">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="PartHeader" minOccurs="1" maxOccurs="1"/>
      <xs:element ref="PartRow" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="style" default="display:table"/>
  </xs:complexType>
```

```

</xs:element>

<xs:element name="PartHeader">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="PartCell" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="style" default="display:table-row" />
  </xs:complexType>
</xs:element>

<xs:element name="PartRow">
  <xs:complexType>
    <xs:complexContent>
      <xs:extension base="aras:itemType">
        <xs:choice maxOccurs="unbounded">
          <xs:element ref="PartCell" minOccurs="0" maxOccurs="unbounded"/>
        </xs:choice>
        <xs:attribute name="style" default="display:table-row"/>
        <xs:attribute name="typeId" type="xs:string"
fixed="4F1AC04A2B484F3ABA4E20DB63808A88"/>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:element>

<xs:element name="PartCell">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="Text" minOccurs="0" maxOccurs="1"/>
    </xs:sequence>
    <xs:attribute name="style" default="display:table-cell"/>
  </xs:complexType>
</xs:element>

```

7.2.6.4 Content Generator

There needs to be a Content Generator for the `PartHeader` Document Element, which will be called immediately after the `PartTable` Document element is placed. This Method will add the `PartHeader`, `PartCells`, and `Text` for all columns required for the table. Similarly, a Content Generator needs to be created for the `PartRow` Document Elements; which will point to some selected Part (given that they extend `aras:itemType` and the 'typeId' is fixed to the `ItemType` Id of Part. In Innovator 11.0 SP15, the ability to pass JSON-formatted string content to the Content Generator Method was added. We can use this to pass in the names of Columns and Properties to add for the `PartHeader` and `PartRow` Content Generators, respectively. For the `PartHeader` Document Element, the following JSON is used for this example:

Xml Schema Element

Name
PartRow

Renderer
[tp PartRowRenderer](#)

Generator Parameters

```
{
  "props":["name", "item_number",
    "classification"]
}
```

Note the 'Generator Parameters' form element. It uses a JSON-configured editor so that JSON elements can be color-coded as shown. Opening the XML Schema Element exposes this form. In this case, a single array element is used – `props` – with a list of all Properties names to use for the Part Row Contents. The number of array elements determines the number of columns. A similar construct is used for the `PartHeader` Content Generator; but using Column Names as the array elements.

```
//MethodTemplateName=CSharp:Aras.TDF.ContentGenerator(Strict);

// The Target Items should be a PartRow Document Element
ItemDocumentElement targetItem = targetElement as ItemDocumentElement;

if (targetItem != null) {
    targetItem.ClearChilds();

    // if referenced item was set, then
    if (!targetItem.IsEmpty)
    {
        // The configured parameters should contain an array labeled 'props'
        // that contains an array of string values to indicate the properties
        // to use for the associated referenced Item
        if (!this.GenerationParameters.ContainsKey("props"))
        {
            throw new InvalidOperationException("'props' generation parameter was not
passed.");
        }

        // Create cells for each property name configured as parameters
        System.Collections.ArrayList props = (System.Collections.ArrayList)
this.GenerationParameters["props"];
        foreach (String propName in props)
        {
            DocumentSchemaElement cellElement = (DocumentSchemaElement)
this.Factory.NewElement("PartCell");
            cellElement.AddChild(this.Factory.NewText("Text", targetItem.GetItemProperty(propName,
" ")));

            // Add as a child
            targetItem.AddChild(cellElement);
        }
    }
}
```

7.2.6.5 *Renderer*

The following Editor renderer will display the part number for the node text in the Tree View. This is helpful in identifying the referenced Part in the Tree. Note that this example is using data in the content for the part number, so it assumes that the part number (in this case) is stored in the second column. Note that Render Methods are described in Section 2.3.4.

```
return
{
  constructor: function (args)
  {},

  // Set the name of the tree node based on the referenced Part
  GetTreeName: function(/*WrappedObject*/renderObject)
  {
    // Get the of the referenced Part
    // return the content of the second column - Part Item Number
    if(renderObject.origin.childNodes.length > 1)
      return renderObject.origin.childNodes[1].text;
    else
      return "Part";
  }
};
```

7.2.6.6 *CSS*

The CSS targets each Document Element as follows. Note the use of 'border-collapse' for the PartTable class.

```
.PartCell {
  border: 1px solid black;
  padding-left: 1em;
  padding-right: 1em;
  padding-top: 0em;
  padding-bottom: 0em;
}

.PartHeader .PartCell
{
  background: grey;
  text-align: center;
}

.PartHeader .PartCell .Text
{
  font-weight: bold;
  font-size: 1.2em;
  color: white;
}

.PartCell .Text
{
  padding: 0em;
}

.PartTable {
  border-collapse: collapse;
```


}

7.3 Configuring a Table of Contents

The Content Generator enables you to create and modify the Table of Contents for a document using the TOC XML schema element and the `tp_TOCContentGenerator` method. The TOC XML Schema element contains the parameters used to create a Table of Contents as shown in the following screenshot.

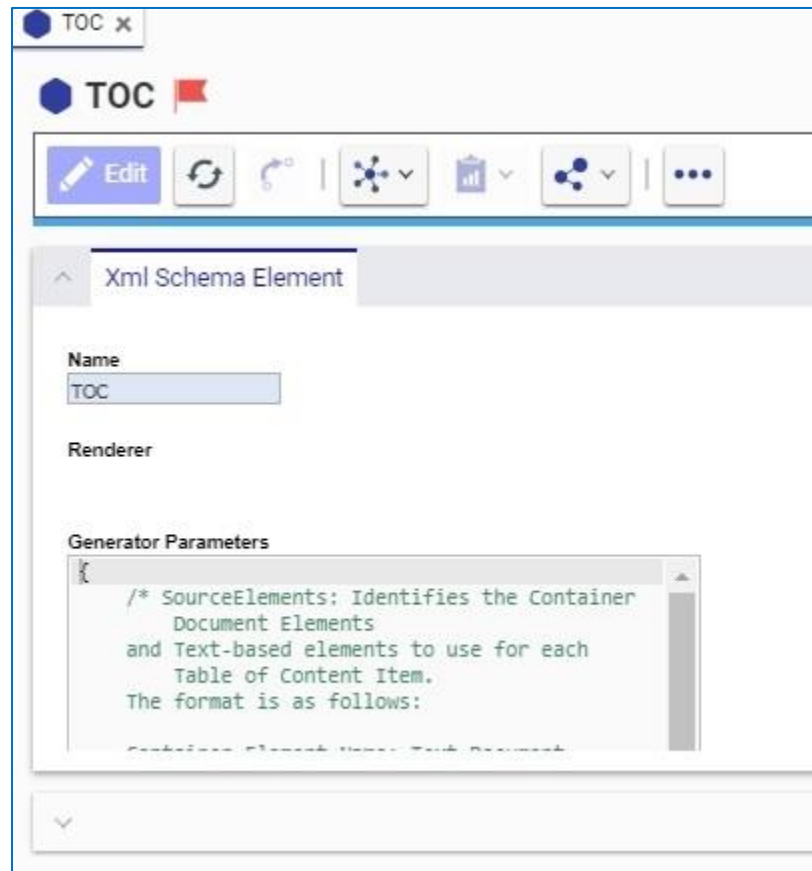


Figure 7.

You can specify the:

- Chapter and Section headings.
- Maximum depth of headings that appear in the Table of Contents.
- Name of the document element to create for each TOC item.

The **SourceElements** JSON Object Parameter identifies the Container Document Elements and Text-based elements to use for each Table of Content Item. The format is as follows:

Container Element Name: Text Document Element Name

For example, the following searches for 'Chapter' and 'Section' Document Elements and will use the content of the first 'Title' Document Element within each for the TOC text.

```
"SourceElements": {
  "Chapter": "Title",
  "Section": "Title"
```

```
},
```

The **MaxLevel** Parameter specifies the maximum depth to use for the Table of Contents. Valid values are equal to or greater than 1. The following example sets the depth to 3; meaning that the TOC Generator will search to a depth level of 3 for child and grandchild Chapter and/or Section Documents for the content of the TOC.

```
"MaxLevel": 3,
```

The **TOCItemName** Parameter specifies the Name of the Document Element to create for each TOC Item generated. The following example will create a 'TOC-Item' Document Element for each TOC Item it creates:

```
"TOCItemName": "TOC-Item"
```

The values you specify should be passed to the `tp_TOCGenerator` method in JSON format. The **Generator Parameters** property accesses the specified values. It's critical that JSON formatting be accurate when setting these properties. The following example show the proper configuration for the example settings discussed above:

```
{
  "SourceElements": {
    "Chapter": "Title",
    "Section": "Title"
  },
  "MaxLevel": 3,
  "TOCItemName": "TOC-Item"
}
```

Note: In the example above, note the use of the outer '{'s and the use of '{'s for the **SourceElements** JSON Object. Also note the use of ','s for separating values and the use of ":" for separating name and value pairs.

Note: JavaScript Object Notation (JSON) is used to serialize JavaScript Object data. It is used in the Technical Document Framework to enable complex configuration data for parameterizing Methods. A description of JSON, including how JSON can be represented via Collection classes in .Net, is outside the scope of this document.

7.4 Managing Languages

The Aras Technical Documentation solution gives you the option of specifying the language used for creating the documents. By default, the language is set to English.

Warning If the default language is changed, external links will disappear from existing Technical Documents. The language should only be changed if there are no Technical Documents in the database.

To edit the language used for creating Technical Documents:

1. Confirm that Language and Locale preferences are set up in the database.

For more information on setting up Language and Locale preferences, see *Aras Innovator - Configuring Internationalization*.

2. Log into Aras Innovator as an administrator.

3. Go to **Administration\Variables\Search** in the TOC and open the **tp_DefaultLanguage** item for editing.

4. Change the **Value** to the desired language.

Note: The **Value** must match the name of the corresponding **Language** item.

7.5 Attributes Validation Support

This appendix describes the data types most used in Technical Documentation.

7.5.1 String Data Types

The following table describes the available string data types. All listed types are processed on the client as “string” types. They are only validated against defined restrictions.

Type Name	Description	Schema validation	Attribute dialog validation
ENTITIES		Yes	No
ENTITY		Yes	No
ID	A string that represents the ID attribute in XML (only used with schema attributes)	Yes	No
IDREF	A string that represents the IDREF attribute in XML (only used with schema attributes)	Yes	No
IDREFS		Yes	No
language	A string that contains a valid language id	Yes	No
Name	A string that contains a valid XML name	Yes	No
NCName		Yes	No
NMTOKEN	A string that represents the NMTOKEN attribute in XML (only used with schema attributes)	Yes	No
NMTOKENS		No	No
normalizedString	A string that does not contain line feeds, carriage returns, or tabs	No	No

QName		Yes	No
string	A string	Yes	Yes
token	A string that does not contain line feeds, carriage returns, tabs, leading or trailing spaces, or multiple spaces	No	No
anyURI		Yes	Yes
base64Binary		Yes	No
hexBinary		Yes	No
NOTATION		Yes	No

7.5.2 Date and Time Data Types

Type Name	Description	Schema validation	Attribute dialog validation
date	Defines a date value	Yes	Yes. Works like dateTime.
dateTime	Defines a date and time value	Yes	Yes
duration	Defines a time interval	Yes	No
gDay	Defines a part of a date - the day (DD)	Yes	No
gMonth	Defines a part of a date - the month (MM)	Yes	No
gMonthDay	Defines a part of a date - the month and day (MM-DD)	Yes	No
gYear	Defines a part of a date - the year (YYYY)	Yes	No
gYearMonth	Defines a part of a date - the year and month (YYYY-MM)	Yes	No
time	Defines a time value	Yes	No

7.5.3 Numeric Data Types

- [1] Numeric types are processed as a 'long' on the client and not validated against type boundaries.
- [2] There is a limitation within the .Net processing logic for 'default/fixed' values validation. If the `min/maxExclusive` restriction is specified and it has a value that exceeds the type of boundary by one ('1'), then system will show a validation error. For example:

byte has a range of values -128 .. 127. If the `maxExclusive` restriction is given a value of '128' then a validation error will be shown.
- [3] The Javascript `parseInt/Float()` method is used to process strings for client-side validation within the Attributes Dialog. Strings that contain invalid characters for a numeric value will not show as invalid (warning icon).
- [4] Restriction 'totalDigits' is not processed for integer, long, int, short, and byte types.

Type Name	Description	Schema validation	Attribute dialog validation
byte	A signed 8-bit integer	Yes [2]	Yes [1]
unsignedByte	An unsigned 8-bit integer	Yes [2]	No
short	A signed 16-bit integer	Yes [2]	Yes [1,3]
unsignedShort	An unsigned 16-bit integer	Yes [2]	No
decimal	A decimal value	Yes [2]	Yes [3]
int	A signed 32-bit integer	Yes [2]	Yes [1,3]
unsignedInt	An unsigned 32-bit integer	Yes [2]	No
integer	An integer value	Yes [2]	Yes [1,3]
long	A signed 64-bit integer	Yes [2]	Yes [3]
unsignedLong	An unsigned 64-bit integer	Yes [2]	No
negativeInteger	An integer containing only negative values (...,-2,-1)	Yes [2]	No
nonNegativeInteger	An integer containing only non-negative values (0,1,2,...)	Yes [2]	No
nonPositiveInteger	An integer containing only non-positive values (...,-2,-1,0)	Yes [2]	No
positiveInteger	An integer containing only positive values (1,2,...)	Yes [2]	No

Type Name	Description	Schema validation	Attribute dialog validation
double		Yes	Yes [3]
float		Yes	Yes [3]

7.5.4 Miscellaneous Data Types

[1] Value can be represented as '0'-'1' or 'true'-'false' pairs in the XML Schema. However, since the editor field on the client is represented as a checkbox we support only 'true'-'false' pair.

Type Name	Description	Schema validation	Attribute dialog validation
boolean		Yes	Yes [1]

7.6 File Import API

7.6.1 Overview

The File Import API provides a framework for creating technical document content programmatically by extracting data from a selected file. The process works as follows:

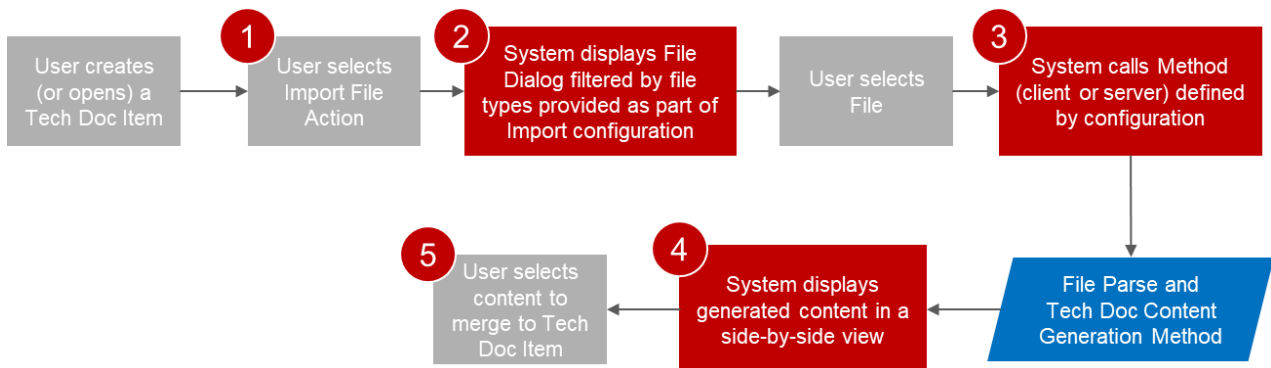


Figure 8.

1. Using an existing or a new Technical Document, the User initiates the File Import operation by selecting an Action. Note that this Action is only enabled when the Technical Document is in Edit mode.

2. The system prompts the user with a File Dialog, filtered based on configured Document Types (extensions)
3. After the User chooses a single file, the system executes a custom Method that parses the data from the selected file and generates Document Element content
4. The system displays the generated content in a side-by-side view (right side) where the content is read-only
5. The user can copy/merge all generated content with a single operation or can selectively drag and drop individual Document Elements from the view to the opened Technical Document
6. Closing the right-side view discards the generated content

A similar description is provided in the following diagram:

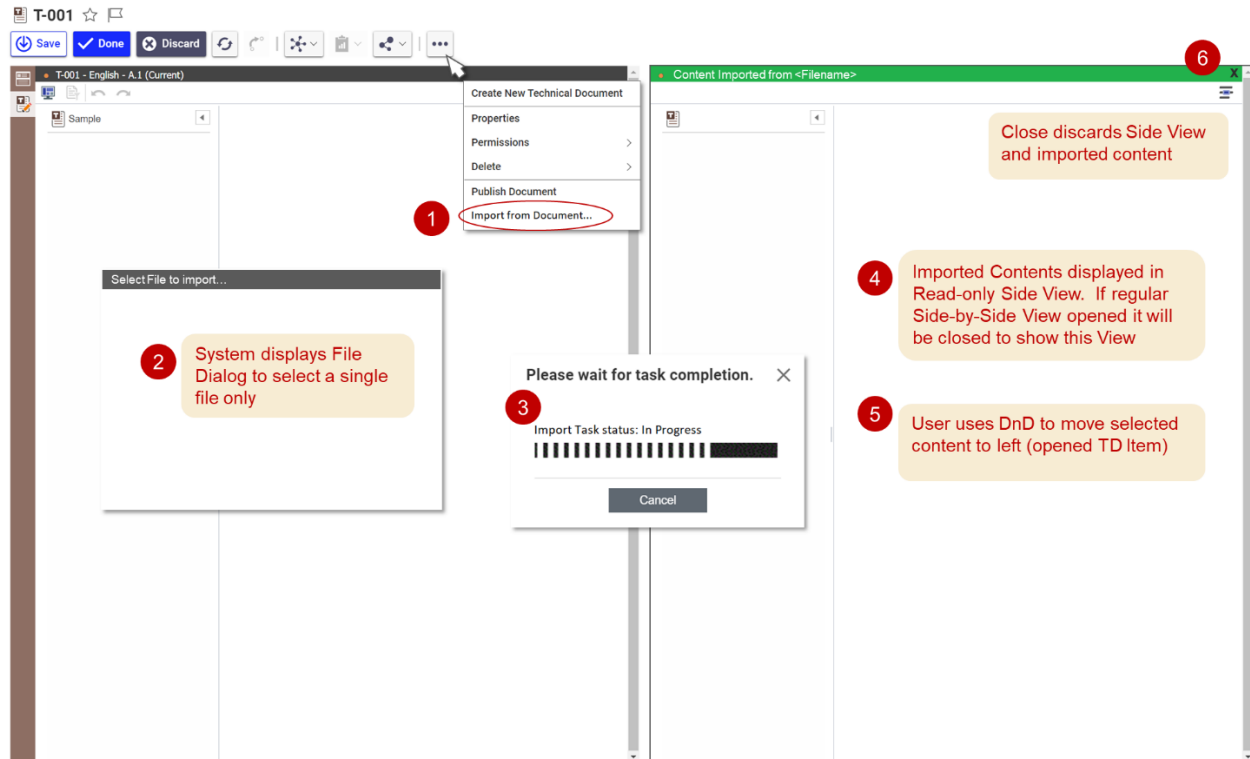


Figure 9.

The custom Method (7.6.2.4) is responsible for reading the content from the file and use the Content Generator API (7.1) to programmatically generate Document Element content based on the parsed file data. Thus, custom implementations need only provide the logic to read the specific file format content and use the associated Content Generator Classes/Methods to create document content.

7.6.2 Creating a File Importer

The following example shows how to set up File Import for MS Word (.docx) files.

Note: Before creating a File Import Method it's important to understand the Document Schema being used for the generated content. Generated Document Elements use a specified Document Type. System Administrators should understand the structure and limitations of the configured schema before implementing the logic to generate content for that schema.

7.6.2.1 Initializing the Methods for the Import File Action

System Administrators provide the following as part of File Import Configuration:

1. Supported Document type(s). This is specified using file name extensions. The system will use this information to filter the files displayed in the File Dialog.
2. Identify whether the system should provide the custom Method with file content in ASCII text format or as binary Base64 format. The latter is required for non-ASCII files.
3. Identify whether the system should store the selected file in the vault. Imported files can first be uploaded and stored in the vault. End-user customizations are responsible for managing these File Items.
4. Identify the Method that will be executed to parse the file data and return the generated Document Elements

7.6.2.2 Update Method `cui_tdf_ivicb_fileimport_init`

Method `cui_tdf_ivicb_fileimport_init` is used to initialize the Configurable UI (CUI) action for File Import. Within this Method is the JavaScript software to initialize the `ImportManager` object. The `configs` array parameter for the `ImportManager` identifies the configuration for each supported File Type. It needs to be populated. As an example, the following sample code initializes the Import Manager with two file types: JSON, and MS Word.

```
if (!importManager) {
  Promise.resolve(window.TDF || import('../jsBundles/tdf.es.js')).then(() => {
    importManager = additionalData.importManager =
      new window.TDF.import.managers.FileImportManager({
        aras: window.aras,
        configs: [{
          extensions: ['.json'],
          processing: {
            methods: ['fi_tdf_importcontent_json'],
          },
        },
        {
          extensions: ['.docx'],
          processing: {
            methods: ['td_WordImporter'],
          },
        }
      ]
    });
}
```

The `configs` property must be defined to include one or more JSON objects with the following properties:

`extensions`: array of strings for all related file extensions. Each string identifies the file extension starting with a ‘.’

`processing`: Identifies one or Method names that will be called in succession after the User has chosen a File for import

`reading`: Optional (see below)

`contextData`: Optional (see below)

7.6.2.3 Update Method `cui_tdf_ivicb_fileimport_click`

Method `cui_tdf_ivicb_fileimport_click` is called when the Import File Action is executed.

Within this Method is the JavaScript software that extends the `ImportManager` configuration with data used for calling the configured Methods. The `reading` parameter for the `ImportManager` identifies how the configured Method should be called. For example, the following sample code configures the Import Manager to read the selected File data, provide it as Base64 content to each configured Method, and *not* save the file to the vault.

```
if (isEditorActive) {
  const { tdfSettings } = tabController.shareData;
  const xmlSchema = window.aras.getItemProperty(window.item, 'xml_schema');
  const { importManager } = additionalData;

  importManager
    ?.import(null, {
      reading: {
        readContent: true,
        outputFormat: 'base64',
        saveFile: false,
      },
      contextData: {
        xmlSchema: xmlSchema,
        contentBuilder: tdfSettings.contentBuilderMethod,
      },
    })
    .then((result) => {
      if (result) {
        const editorWindow =
          tabController.views['editor_container'].domNode
            .contentWindow;

        editorWindow.openImportPanel(result);
      }
    });
}
```

The `reading` property must be defined to include one or more JSON objects with the following properties:

readContent: If `'true'`, the file contents are read from the selected file and provided as input to the configured Method(s).

outputFormat: Either `'text'` or `'base64'`(default). This determines the format of the input data provided for the configured Method(s).

saveFile: If `'true'`, the selected file is saved in the Vault. If `'false'`, it won't.

contextData: Contains configurable options for the schema and Content Builder Methods. In this example, the `xmlSchema` is retrieved from the opened Technical Document and the `contentBuilder` uses the default. It is best to leave these values as provided in the sample.

Note: It is possible to define both of these parameters when modifying the Method `cui_tdf_ivicb_fileimport_init` (see above). In this case, separate `reading` and `contextData` can be defined for each supported file type.

7.6.2.4 Define the Import Method

The following example shows a server-side Method used to read data from a MS Word (docx) file and generate Document Elements. Note that the details related to parsing/extracting content from the input file are not included. The code is used to describe how to access the input data and how to use the Schema Factory and Content Generator API to create Document Elements and return the results in the expected format. Much of the code can be copied and used as is.

```
return Execute(this, RequestState);
}

internal static Item Execute(Item sourceItem, Aras.Server.Core.IContextState requestState)
{
    Innovator innovator = sourceItem.getInnovator();
    string languageCode = innovator.getI18NSessionContext().GetDefaultLanguageCode();

    var executionContextData =
    Aras.TDF.Base.ComplexObjectJsonDeserializer.Deserialize(sourceItem.getProperty("executioncontext"));
    var contextData = executionContextData["context"] as Dictionary<string, object>;

    string schemaId = contextData["xmlSchema"] as string;
    string contentBuilder = contextData["contentBuilder"] as string ?? "tp_DocumentGet";

    var importData = executionContextData["input"] as Dictionary<string, object>;
    var fileContent = Convert.FromBase64String(importData["content"] as string);
    var memStr = new MemoryStream(fileContent);

    Aras.TDF.Base.SchemaElementFactory factory = CreateFactory(sourceItem, innovator,
    schemaId, contentBuilder);
    factory.CustomContentEnabled = true;
    factory.DefaultLanguageCode = languageCode;

    using (WordprocessingDocument wordprocessingDocument =
    WordprocessingDocument.Open(memStr, true))
    {
        mainDocPart = wordprocessingDocument.MainDocumentPart;
        Body body = mainDocPart.Document.Body;
        var rootElement = factory.NewBlock() as Aras.TDF.Base.DocumentSchemaElement;

        ...
    }
    var importedData = factory.ExportElementToXml(rootElement);
    return innovator.newResult(importedData);
}

public static Aras.TDF.Base.SchemaElementFactory CreateFactory(Item thisItem, Innovator
innovator, string schemaId, string contentBuilderMethod)
{
    if (thisItem == null)
    {
        throw new ArgumentException("Processing error for File Import");
    }

    var executionContext = new Aras.TDF.Base.SchemaElementExecutionContext(thisItem.getID(),
    thisItem.getType(), contentBuilderMethod);
```

```
return new Aras.TDF.Base.SchemaElementFactory(innovator, schemaId, executionContext);
```

1. The context of the Method (`this`) is the Document Item opened or created as part of the Use Case to initiate the File Import Action (see above). `RequestState` is not used or needed for this implementation. Note the missing initial curly brace – it is based on the Method template chosen.
2. Input for this Method is passed as a JSON-encoded string. This line de-serializes the data into Objects for 'input' and 'context'.
3. `context` data is a series of name/value pairs that were defined in the client Action Method (see above). In this case, it includes the XML Schema (`DocumentType`) and the Content Builder. For the purposes of File Import, this data is only required when instantiating the Schema Factory (#6 below) used for creating Document Elements.
4. `input` data describes the file being imported. This includes the file name without the path ('name'), file extension string ('extension'), file size ('size'), MIME Type ('mimeType'), and file content ('content').
5. Because the file data was passed as a Base64 stream (see above), it needs to be converted back to the format as it was defined in the file.
6. The embedded method – `CreateFactory()` – shows the code necessary for creating the Schema Factory used to create Document Elements. This code can be used as is.
7. The Factory object is used as a mechanism to create individual Document Elements. The root element must be a `Block` as shown on this line.
8. The Factory object is used to convert the entire hierarchy of Document Element Objects into the XML representation needed for the expected return of this Method. **Note: The XML for the Technical Document content should be generated using the Schema Factor only to ensure compatibility with new and future versions of the Technical Document Editor.**
9. The generated XML is included in the returned result. This is used for display in the right-side view within the Document Editor.

7.6.2.5 Enable the Menu Item

The **Import from Document...** Menu Item needs to be enabled using the Configurable User Interface configuration. The menus and toolbar buttons for the Technical Document Editor are configured within the **TechDoc** Presentation Configuration.

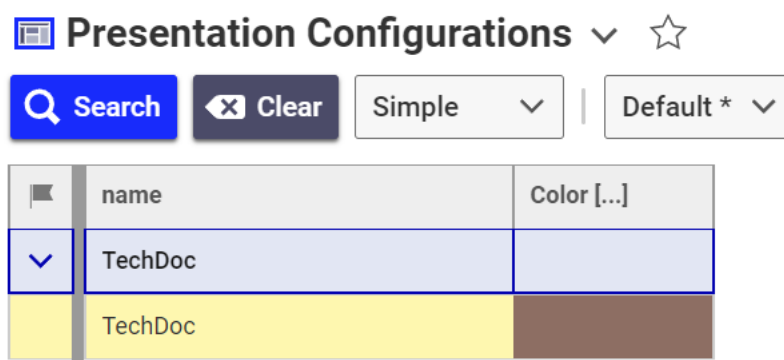
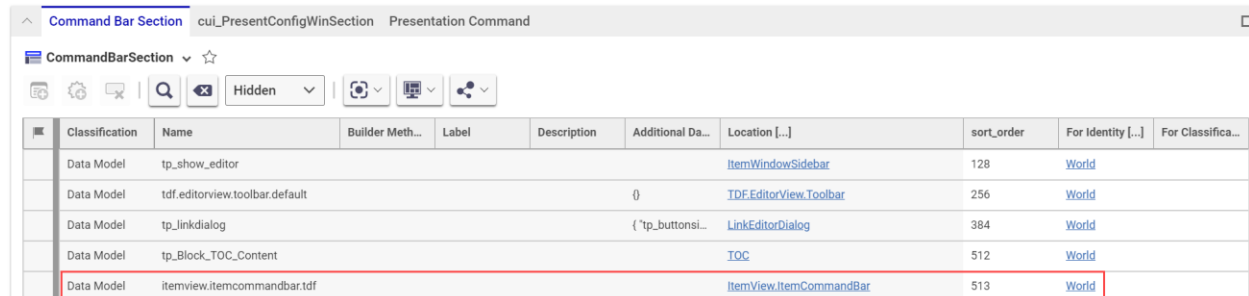


Figure 10.

Note: The Presentation Configuration ItemType is not added to the Innovator Table of Contents (TOC) by default. To add it, so that a search for the **TechDoc** Presentation Configuration can be opened for Edit, use the TOC Editor as described in the Configurable User Interface Administrator Guide.

Edit the **TechDoc** Presentation Item to add the **itemview.itemcommandbar.tdf** Command Bar Section.



Classification	Name	Builder Meth...	Label	Description	Additional Da...	Location [...]	sort_order	For Identity [...]	For Classifica...
Data Model	tp_show_editor					ItemWindowSidebar	128	World	
Data Model	tdf.editorview.toolbar.default				()	TDF.EditorView.Toolbar	256	World	
Data Model	tp_linkdialog				{ 'tp_buttonsl...	LinkEditorDialog	384	World	
Data Model	tp_Block_TOC_Content					TOC	512	World	
Data Model	itemview.itemcommandbar.tdf					ItemView.ItemCommandBar	513	World	

Figure 11.

Select the **Add CommandBarSection** button, search for and add the **itemview.itemcommandbar.tdf** Command Bar as shown, setting the Sort Order and For Identity as required.

7.7 Client API

7.7.1 Overview

The Technical Document Editor is implemented using JavaScript classes/methods that provide the ability to, for example, add new Document Elements, update styling, link content to selected Items, manipulate content, etc. The Technical Documentation Client Application Programming Interface (Client API) exposes the same underlying functionality in a simplified interface so that it can be used to add custom features within the Editor. This API is made available in user-developed Methods attached to new toolbar items, menus, or key processing events to customize and extend the operation of the Editor. Toolbars, menus, and key events are configured using the Configurable User Interface (CUI) (see the Configurable User Interface Administrator Guide for more information on CUI and how to use this framework to custom the User Interface within Aras Innovator).

7.7.2 Scope

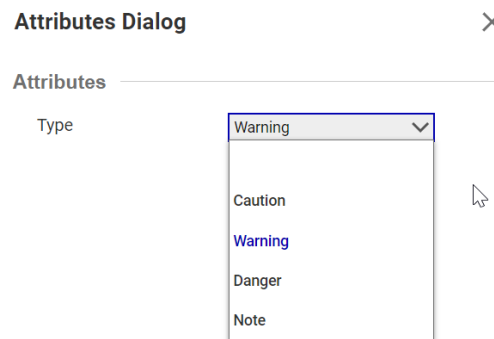
The Client API consists of 3 main classes: Document, Schema, and View (see Figure 17). The **Document** class provides access to all the existing content within an open Technical Document, provides the ability create new Document Elements, and remove (or move) existing Document Elements. The **Schema** class provides the ability to query the associated Document Schema to check or validate content operations against the user-defined XML Schema. The **View** class provides access to selected Elements. When used along with custom logic implemented with JavaScript Methods, the breadth of customization options is extensive.

7.7.3 Customization Examples

To get a sense of the possibilities of using the Client API, this section provides some samples that illustrate the ways in which the API can be leveraged for specific Use Cases.

7.7.3.1 Setting Document Element Attributes

Document Elements can have one or more Attributes that can be used to characterize them, or otherwise provide additional semantic information. Attributes can also be used to distinguish styling (see Section 7.2.4). In the following example, a *Note* Document Element has 4 Attributes; the values of which help identify its *Type* and are used to isolate unique Styling so that each Note Type can be rendered appropriately. These enumerations are displayed in the Attributes dialog as valid selections for this Attribute:



This example adds a dropdown list control to the Technical Document Editor toolbar that is enabled when the user selects a Note Document Element. This can be used in lieu of the Attributes Dialog as a more convenient mechanism for settings these values.

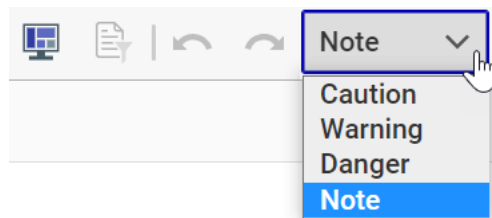


Figure 12. Note Type Selection Dropdown

A sample of the rendered Note Document Element types are as follows:

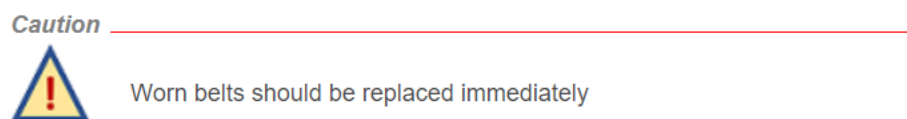


Figure 13. 'Caution' Type example



Figure 14. 'Note' Type Example

Schema

```
<xs:element name="Note">
  <xs:complexType>
    <xs:sequence>
```

```

        <xs:element ref="Text" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="type" type="NoteAttType" default="Note"/>
</xs:complexType>
</xs:element>

<xs:simpleType name="NoteAttType">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Caution"/>
        <xs:enumeration value="Warning"/>
        <xs:enumeration value="Danger"/>
        <xs:enumeration value="Note"/>
    </xs:restriction>
</xs:simpleType>

```

CSS

```

...
.Note[type='Note'] { border-top: 1px solid blue; border-bottom: 1px solid blue; }
.Note[type='Caution'] { border-top: 1px solid red; border-bottom: 1px solid red; }
...

```

The Method logic to make the change is as follows:

```

const { document, view } = options.contextData;
const selectedElements = view.selection.get();

// Set the 'type' Attribute for the selected 'Note' Document Element
// based on the chosen value from the dropdown box
selectedElements[0]?.setAttribute('type', target.value);

```

In general, the selected Element is identified, and the Attribute is set based on the user's choice. Thus, with two lines of JavaScript, and some CSS, a convenient and data-driven mechanism is provided to the Technical Author to automatically characterize and render Note information in a Technical Document.

7.7.3.2 Manipulating Document Elements

When authoring Technical Document content, there may be a set of common manual operations that would be more convenient to automate to increase productivity and provide an increased level of consistency. The following Use Case shows an example of *List Item Indentation* whereby users can increase (or decrease) indentation by inserting (or removing) *List Items* to provide a sub-*List* for selected Text.

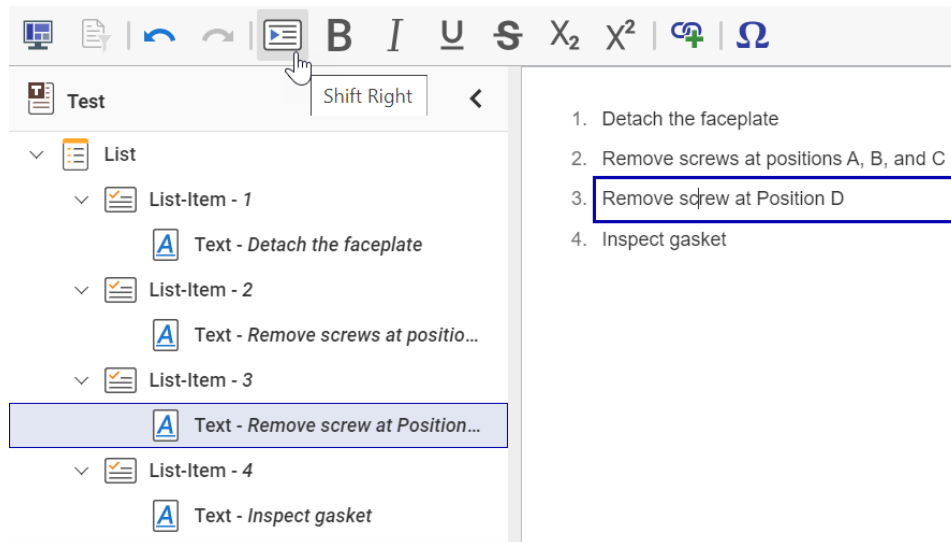


Figure 15. Shift Right Custom Function

When using List Document Elements (Section 7.1.2), indentation is a matter of creating 'sub-lists': Lists that are children of a List Item. For example, in Figure 15 there is a single List with 4 List Items; each with a single Text Document Element. To indent a List Item, a new child List needs to be added with content added as it's List Items and so on.

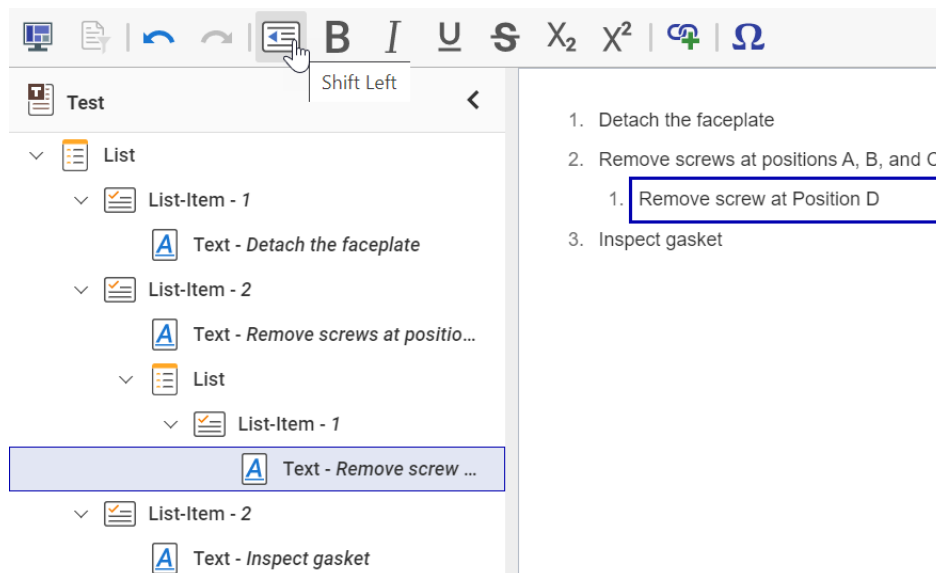


Figure 16. Shift Left Custom Function

Figure 15 and Figure 16 also show a customized **Shift Right** and **Shift Left** button that has been added to the Editor toolbar. Toolbar buttons can be configured to be displayed and/or enabled based on the state of the Document (whether it is opened for Edit) and/or depending on what the user has selected. In this example, when the user selects any Document Element that is a List Item or a child of a List Item either the **Shift Left** or **Shift Right** Button is displayed.

The logic to enable this capability (not shown):

1. Checks to see if the selected Document Element is either a List Item or is a direct descendant of one *and* that the List Item is not the first in its List
2. If so, the entire List Item Document Element is removed from its parent List
3. A new List is created and appended to the previous List Item
4. The selected List Item is added to the newly created List

The Client API provides the ability to perform each of these steps (and more). Specifically for this example, it uses the **View** class to ensure that a single Document Item is selected. It uses the **Document** class to create a new List Document Element, and it uses the selected List **Element** (explained later) to remove the selected List Item and reposition it in a new List.

7.7.4 Client API Definition

7.7.4.1 Overview

This section provides a detailed description of the Technical Documentation Client API. The Client API consists of several JavaScript classes; a definition of the available properties and methods of each are included.

Warning This API description will serve as the authoritative source for classes and related functionality for the Technical Documentation Client API. The classes/methods/properties included here are guaranteed to be available unless officially deprecated through an explicit notice. Any use of JavaScript functionality for Technical Document Editor customization not described in this section will be at the risk of the implementor since that capability may change without notice.

Each Method used with a CUI Item is passed several input parameters. This is standard design for Methods used with CUI. One of them – `options` – contains a `contextData` object that is used to access the Technical Document **Document** and **View** objects. **Document** and **View** are core objects defining the Client API (see Figure 17). For example, the following code fragment will create local `document` and `view` variables to be used to access the **Document** and **View** respectively:

```
const { document, view } = options.contextData;
```

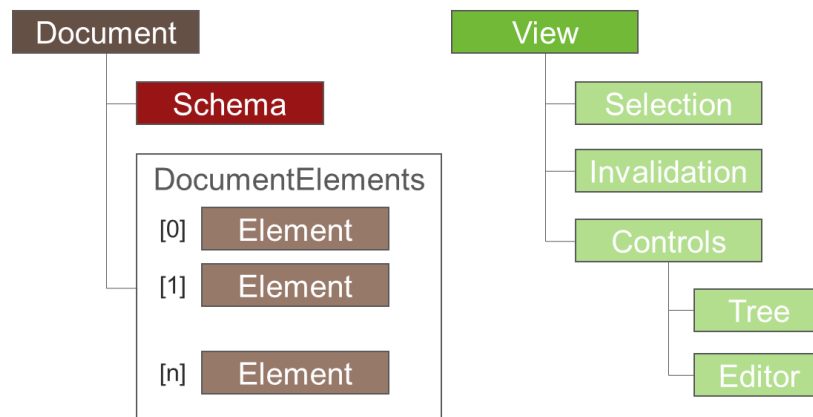


Figure 17. Document / View / Schema Classes

7.7.4.2 Document

The **Document** class is the access point for document content as well as information about the Technical Document Item in which the document content is contained.

Document Class Properties		
Name	Write	Description
item	F	An XML Node representing the Technical Document Item (e.g., <code>tp_Block</code>). This object can be used to access Properties or related Items. For example: <code>docName = aras.getItemProperty(document.item, 'name');</code>
schema	F	Object handle to the Schema class defined later in this section

language	F	String representing the language used for the current document. For example: 'en'
editable	F	Boolean that identifies whether the current Technical Document is opened for editing. This is useful when enabling CUI controls that modify Document content. For example: <pre>if (document.editable) { // logic to update document...</pre>
elements	F	Handle to DocumentElements class (defined later in this section) that provides access to, and creation of, Document Elements. For example: <pre>const newList = document.elements.create('List'); or const firstElem = document.elements.get(1);</pre>

Document Class Methods		
Name	Input	Description
createElement		Used to create a Document Element. Document Elements are created based on the Element name provided in the Document Schema (e.g., 'List', 'ItemInfo', 'Note', etc.). The returned Element object can then be used/added. For example: <pre>document.createElement('ItemInfo', {item, partItem}); or document.createElement('Note');</pre>
		Note: An Element is not added to a document when it is created. Rather it must be explicitly added as a child of some existing Element
	name	Document Element Name
	parameters	(Optional) Element Object Properties. The Properties of the object used are based on the type of Document Element being created. Creating instances of Element types are described later in the document
clear		Removes all content from the document. Use with caution!

7.7.4.3 DocumentElements

The **DocumentElements** class is a container for Document Elements. Note that this class manages Document Element content as a flat list. However, children of specific Document Elements can be accessed using the **ElementChildren** class defined later.

DocumentElements Class Properties		
Name	Write	Description
root	F	Handle to the top-most Element in a Technical Document. Note that all Technical Documents have a single, root Element – <code>block</code> – that contains all the content. Note also that root is the same Element as returned by <code>get(0)</code> – see below.
count	F	Number of Document Elements, including the root block, in the document.

DocumentElements Class Methods		
Name	Input	Description
get	Used to retrieve a Document Element. Document Elements can be accessed based on an index – a sequential number starting from 0 and progressing in the order the Elements are displayed in the Tree View, and id – internal id value for each Element, or uid – a unique 32 character identifier that persists with the Element. For example: <pre>const blockElem = document.elements.get(0); const fourthElement = document.elements.get(4, 'index'); const listElement = document.elements.get('24DKR82LFMN5927DJ4MNSHR7693KR821', 'uid');</pre>	
	index	Positive integer starting from 0 or a 32 character string based on criteria. Note that the first ('0') Element is the root block, which contains all Document Elements.
	criteria	(Optional) – Either 'index' (default), 'id', or 'uid'. If provided the given index parameter is interpreted accordingly.
index	Returns the index number of the given Element. You can check an Element's position within the document by calling this method. For example: <pre>let contentPosition = document.elements.index(someElement);</pre>	
	element	Element to check
create	Used to create an Element. Document Elements or each type (see Section 2.3.2) can be created using this method. The optional parameters are used to include type-specific data. Note that an Element is not included in a document until it is explicitly added as a child of some existing Element, including the root block. For example: <pre>let newTable = document.elements.create('Table', {rowCount: 5, columnCount: 3}); let itemProperty = document.elements.create('ItemProperty', {property: 'name', mode: 'write'}); let text = document.elements.create('Text', {text: 'some text'});</pre>	
	name	Name of the Document Element as defined in the Document schema (e.g., 'List', 'ItemInfo', 'Text', etc.).
	parameters	(Optional) Object containing name:value pairs used to construct an Element. The Document Element type determines the valid values. See details for each Element Type below.
toArray		Returns a newly generated array of all Elements in the document. Use with caution with large documents.

7.7.4.4 Element

The **Element** class is the base class for all Element Types: List, Table, Item, etc.

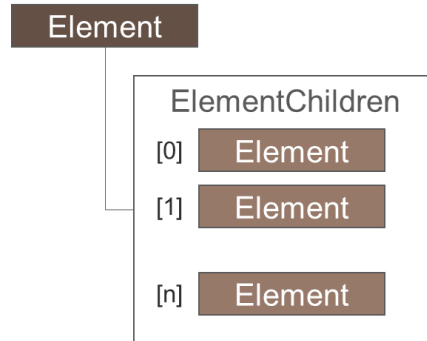


Figure 18. Element Classes

Element Class Properties		
Name	Write	Description
id	F	Internal integer ID assigned sequentially as document content is read/created. Note that an Element's <i>index</i> is different from its <i>ID</i> .
uid	F	Unique, alphanumeric 32-char identifier that persists with the element content.
name	F	Element name as assigned in the Document Schema
parent	F	Handle to an elements direct parent Element
document	F	Handle to the Document object
dynamic	F	Boolean. True, if the Element is associated with a Content Generator that is dynamic
external	F	Boolean. True if the Element is a block that is referenced within the opened Technical Document. Note: Element children of an external block will have an external value of <i>False</i> even though the content is included in a referenced document.
editable	F	Boolean. True if the Element can be modified
children	F	Handle to ElementChildren class. See below
descendants	F	Generated array of a full recursive/deep hierarchy of all content under the Element. For example: <pre>// Get the full set of Document Elements const select = document.elements.root.descendants; // Restrict to a subset of only Text Elements allTxt = select.filter((element) => element.is('Text'));</pre>

Element Class Methods		
Name	Input	Description

contains	Returns True if the given Element is a descendant of the Element on which this method is called. A descendant is any Element child, grandchild, etc that exists <i>under</i> an Element.	
	element	Element to check.
getAttribute	Returns the Attribute value with the given name. Null if the Attribute does not exist.	
	name	Attribute name to check
setAttribute	Sets the Attribute with the given name to the given value.	
	Note: It is possible to set an Attribute for an Element that is not defined in the Document Schema and this Attribute/Value will persist with the document.	
	name	Attribute name
	value	Attribute value
is	Returns True is the Element <i>is</i> the given Type. For example: <pre>if (selectedElement.is('Note')) {</pre>	
	type	Type name, as defined in the Document Schema, to check
clone	Returns a deep copy of the Element	
remove	Removes the Element from the document	
	Note: To remove the Element only from its parent, use <code>DocumentElements.remove()</code> or <code>ElementChildren.remove()</code> .	

7.7.4.5 ElementChildren

The **ElementChildren** class is a container for Documents Elements that are direct children of an associated Element. Similar to an array, the Element children are managed as a sequence representing the order in which content is displayed in the document.

ElementChildren Class Properties		
Name	Write	Description
count	F	Number of direct children. Returns -1 if the associated Element does not contain any child Elements

ElementChildren Class Methods		
Name	Input	Description
get	Returns the Element at the given index.	
	index	Index value (starting from 0) of the Element to retrieve. Similar to an array, the index is the ordered position of the Element.
index	Returns the index number (starting from 0) of the given Element. Returns -1 if the given Element is not a child.	
	element	Element to check.

insert	Inserts the given element at the optional given position. For example: <code>targetList.children.insert(listItem, 2);</code>	
	Note: Document Element are added to a document only by inserting/appending to either a Document or an Element.	
	element	Element to add.
	position	(Optional) Position (starting at 0) to insert the given Element. If the given value is greater than or equal to the <code>count</code> , the new Element will be appended. If the given value is less than the count, the given Element will be inserted at that location, effectively shifting all other children accordingly.
append	Inserts the given element at the end of the list. For example: <code>firstCellRow2.children.append(cellText);</code>	
	Note: Document Element are added to a document only by inserting/appending to either a Document or an Element.	
	element	Element to add.
clear	Removes all child Elements.	
toArray	Returns a newly generated array of all child Elements. Use with caution with large /deep Element hierarchies.	
remove	Removes the given Element from its parent	
	Note: To remove the Element completely, use <code>Element.remove()</code> .	
	element	

7.7.4.6 TextElement

TextElement extends the **Element** class by providing content specifically for Text Document Elements. Text Document Elements are composed of one or more string fragments (`<aras:emph>`) that are used to specify distinct formatting. For example, the following string:

This is a sentence with **bold**, *italics*, and underlined content.

...would be decomposed into the following string fragments:

```
<Text>
  <aras:emph>This is a sentence with </aras:emph>
  <aras:emph bold="true">bold</aras:emph>
  <aras:emph>, </aras:emph>
  <aras:emph italic="true">italics</aras:emph>
  <aras:emph>, and </aras:emph>
  <aras:emph under="true">underlined </aras:emph>
  <aras:emph>content.</aras:emph>
</Text>
```

The terms 'string fragment' and 'emph' are used interchangeably in this section.

Note: Technical Documentation does not store interleaved (mixed) XML/text. Instead, it uses individual XML elements (`<emph>`) to differentiate styling or other distinct metadata

TextElement objects manage the text content as individual objects that can be created, retrieved, and/or updated using the Client API.

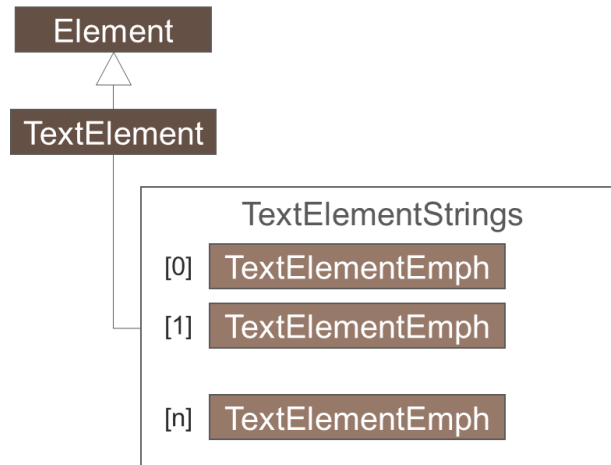


Figure 19. Text Element Classes

The JavaScript to create the sample sentence above using the Client API:

```

let formattedText = document.createElement('Text');
formattedText.strings.add({text: 'This is a sentence with '});
formattedText.strings.add({text: 'bold', style: {bold: true}});
formattedText.strings.add({text: ', '});
formattedText.strings.add({text: 'italics', style: {italic: true}});
...
document.elements.root.children.insert(formattedText);

```

TextElements are created using either the **Document.createElement()** or **DocumentElements.create()** methods. For **TextElement** objects, the second parameter is optional and can include the text Property as specified in the **TextElementEmph** class. For example, the following code sample demonstrates an alternative way of creating a new Text Document Element by including the text:

```

// Create a new Text Element with a single String Fragment
const text = document.elements.create('Text', {text: 'test text'});

```

Note: Non-formatted Text Elements (see Section 2.3.2.3) do not consist of String fragments and only contain a single text property. Elements can be set using the code sample above.

Like all dynamically created **Elements**, they must be added to the children list of some existing **Element** to be included in the document.

The remainder of this section includes a description of the **TextElement**, **TextElementStrings**, and **TextElementEmph** classes.

The **TextElement** class represents a formatted Text Document Element with all its *emph* content.

TextElement Class Properties		
Name	Write	Description
text	T	Generated String that concatenates the text (without formatting) from all the constituent string fragments. This Property is writable. Setting it will overwrite all text content with a single, non-formatted string fragment.
strings	F	Handle to the TextElementStrings class. See below.

The **TextElementStrings** class is a container for the formatted string fragments – *emphs*.

TextElementStrings Class Properties		
Name	Write	Description
count	F	Number of string fragments

TextElementStrings Class Methods		
Name	Input	Description
get	Returns the string fragment at the given index.	
	index	Index value (starting from 0) of the string fragment to retrieve. Similar to an array, the index is the ordered position of the Element.
add	Appends the given String or String fragment object to the set.	
	element	String (text) or TextElementEmph object
remove	Removes the string fragment provided or at the given position. For example: <pre>formattedText.strings.remove(2); // remove 3rd fragment</pre> or <pre>let thirdEmph = formattedText.strings.get(2); formattedText.strings.remove(thirdEmph);</pre>	
	value	Either an index of a TextElementEmph to remove.

The **TextElementEmph** class manages each formatted string fragment – *emph*.

TextElementStrings Class Properties		
Name	Write	Description
text	T	Text content of the string fragment
link	T	String URL if this string fragment is a link
style	T	Object containing name:value pairs for styling information. The following style properties relate to the standard set provided in the Editor: - bold - italic - under - super - sub - strike By default, these are false. Setting any combination of these style parameters as True will result in the corresponding style being applied to the text in the string fragment. For example: <pre>formattedText.strings.add({text: '...', style: {bold: true}});</pre>

		It is possible to set any style property that's defined in CSS using this parameter and the results will persist with the document content. For example: <code>formattedText.strings.get(0).style = {fontcolor: 'red', fontsize: '24px'};</code> Overriding Style settings as defined in the Document Type (see Section 2.4) is discouraged given the design intent of Technical Documentation to separate content from the way it's presented
--	--	--

TextElementStrings Class Methods		
Name	Input	Description
divide		Splits the string fragment at the given text position (0 – beginning of string fragment) and returns a newly generated string fragment containing the remainder of the text. The new string fragment is added automatically to the TextElement. For example: <pre>let splitEmph = formattedText.strings.get(0).divide(7); splitEmph.style = {under: true};</pre> Null is returned if the given position is not within the bounds of the string fragment.
	position	Character position (starting from 0) of the string fragment at which to split the string.
clone		Returns a copy of String fragment object.
remove		Removes the string fragment

7.7.4.7 TableElement

TableElement extends the **Element** class by providing content specifically for Table Document Elements and its related Table Row, Column, and Cell classes. The row and column content of a Table Document Element is managed by the Client API to ensure that all tables are 'rectangular'. That is, that all rows have the same number of columns, and all columns have the same number of rows. Thus, Cells should not be added/removed independent of their respective rows and columns.

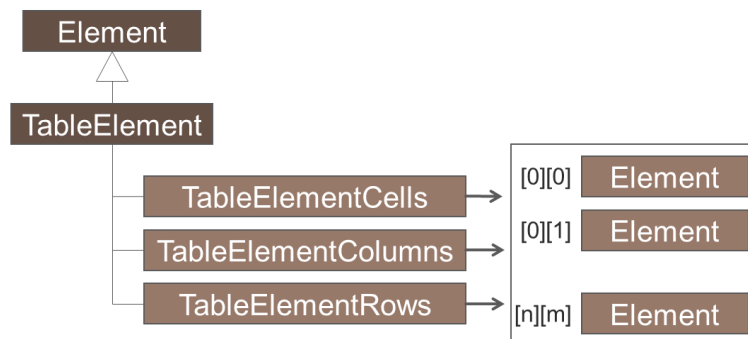


Figure 20. Table Element Classes

All Table Document Elements must be constructed with a given row and column count > 0. After, new rows and/or columns can be added and removed using the API. The content hierarchy of a Table Document Element is as defined in the schema. Table Document Elements (`aras:table`) contain Row Document Elements (`aras:tr`), which contain Cell Document Elements (`aras:td`) (see Section 2.3.2.6). Thus, when a Table Element is constructed, it's children should consist entirely of Row Document Elements. Row Document Elements should consist of Cell Document Elements. The content

of a Table is stored as children of Cell Document Elements. The Client API will construct/remove/update the appropriate Row and Cell Document Elements based on the associated Table function applied.

The following code sample shows a newly created table with row additions, added content, and cell merging.

```
// Create a new 3X4 Table
const table = document.elements.create('Table', {rowCount: 3, columnCount: 4});
// Add it to the beginning of the document
document.elements.root.children.insert(table);
// Get the first cell in the second row
const firstCellRow2 = table.cells.get(1,0);
// Insert a new Text Element
const cellText = document.createElement('Text', {text: 'Cell text'});
firstCellRow2.children.append(cellText);
// Add a new second row
table.rows.add(1);
// Adjust the column width percentages
table.columns.widths = ['20%', '20%', '20%', '40%'];
// Merge the cell with added Text with its right neighbor
table.cells.merge(2, 0, 'right');
```

The resulting table:

Cell text			

Figure 21. Client API-generated Table

The remainder of this section includes a description of the **TableElement**, **TableElementColumns**, **TableElementRows**, and **TableElementCells** classes.

The **TableElement** class represents a Table Document Element and is the top-most reference object for updating and accessing all content within a Table. All actions on a table are provided by one of the row, column, or cell classes. Each are accessible through the corresponding Property.

TableElement Class Properties		
Name	Write	Description
rows	F	Handle to TableElementRows class (see below).
columns	F	Handle to TableElementColumns class (see below).
cells	F	Handle to TableElementCells class (see below).

TableElements are created using either the **Document.createElement()** or **DocumentElements.create()** methods. For **TableElement** objects, the second parameter to these methods must be included and identify the number of columns and rows using 'columnCount' and 'rowCount' respectively. For example, the following code sample demonstrates the only valid way of creating a new Table:

```
// Create a new 3X4 Table
const table = document.elements.create('Table', {rowCount: 3, columnCount: 4});
```

Like all dynamically created **Elements**, they must be added to the children list of some existing Element to be included in the document.

7.7.4.8 TableElementRow

The **TableElementRows** class is an accessor for table Row Elements and provides the ability to add/remove rows. Note that the Row Elements are types of **Elements** and thus extend the same functionality as an **Element** object. For example, the children of a Row Element are the set of Cell Document Elements.

Warning Cell Document Elements should not be added to a Row Element, because doing so would make the table non-rectangular. Instead, use the **TableElementColumns** class to add an entire column – with Cells added automatically to each Row.

TableElementRows Class Properties		
Name	Write	Description
count	F	Number of Rows

TableElementRows Class Methods		
Name	Input	Description
get		Returns the Row Element at the given index.
	index	Index value (starting from 0) of the row to retrieve. Similar to an array, the index is the ordered position of the Element.
add		Inserts a new Row into the Table. Each Row will have the appropriate number of Cell Elements based on the current number of Columns. The Row at the given index will be shifted if not appending to the end and all rows will be shifted accordingly.
	index	Position of the row (starting from 0) of the new Row.
remove		Removes the Row at the given position. For example: <code>Table.rows.remove(2); // remove 3rd row</code>
	row	Either an index or a Row Element to remove.

7.7.4.9 TableElementColumns

The **TableElementColumns** class provides the ability to add and remove Columns. The Client API will automatically add or remove the corresponding Cell Elements (and their content) respectively.

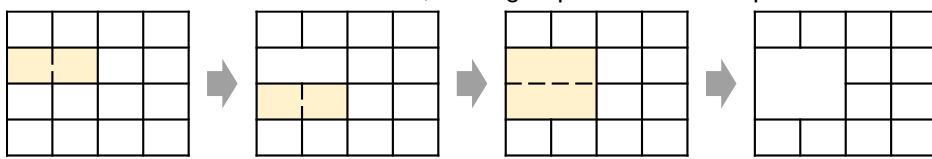
TableElementColumns Class Properties		
Name	Write	Description
count	F	Number of columns
widths	T	String array containing the percentage widths of all columns. This can be modified by setting with a string array with the size equaling the number columns. The number of strings in the array should equal the column count. For example: <code>// Adjust the column width percentages</code> <code>table.columns.widths = ['20', '20', '20', '40'];</code>

--	--	--

TableElementColumns Class Methods		
Name	Input	Description
add		Inserts a new Column into the Table. Each Column will have the appropriate number of Cell Elements based on the current number of Rows. If not appending to the end, the Column at the given index will be shifted accordingly.
	index	Position of the Column (starting from 0) of the new Column.
remove		Removes the Column at the given position. For example: <code>Table.columns.remove(2); // remove 3rd column</code>
	index	Position of the Column (starting from 0) to remove.

7.7.4.10 TableElementCells

The **TableElementCells** class is an accessor for Table Cell Elements and provides the ability to retrieve and merge/unmerge Table Cells. Note that the Cell Elements are types of **Elements** and thus extend the same functionality as an **Element** object. For example, the children of a Cell Element are the set of allowed Document Elements as defined by the schema.

TableElementCells Class Methods		
Name	Input	Description
get		Returns the cell at the given index. The given row and cell numbers must be ≥ 0 and less than the count of rows/cell respectively.
	row	Index value (starting from 0) of the Cell row
	column	Index value (starting from 0) of the Cell Column
merge		Merges the cell at the given coordinates based on the given <i>direction</i> . Cells can be merged with other cells of similar spans. This mimics the functionality within the Technical Document Editor. For example, to merge 4 cells (2 rows and 2 columns) the cells in each row or column must be first merged before these merged cells are merged with the other column or row. Thus, 3 merge operations are required: 
	row	Index value (starting from 0) of the Cell row
	column	Index value (starting from 0) of the Cell column
	direction	String representing the direction of the cell to merge with. Must be one or 'left', 'right', 'top', 'bot'.
unmerge		Un-Merges the cell at the given coordinates. Any cell within a cluster of merged cells can be given as a coordinate. All merged cells will be unmerged with this operation.
	row	Index value (starting from 0) of the Cell row

	column	Index value (starting from 0) of the Cell column
--	--------	--

7.7.4.11 ItemElement

The **ItemElement** class extends **Element** with capability for setting and retrieving Item data. **ItemElement** objects represent Item Document Elements (see Section 2.3.2.2), which are containers with a direct link to a referenced Item. This link is maintained as a Relationship between the encompassing Document Item and the referenced Item. Item Document Elements typically have a configured Content Generator that, upon creation, executes custom logic to automatically populate the child content; presumably with data from the referenced Item and/or its related content and so on. The Client API extends the ability of a Content Generator to update existing Item Document Elements.

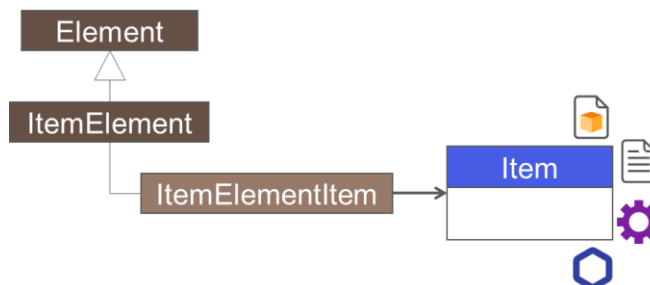


Figure 22. ItemElement Classes

The following code sample shows the process of creating an **ItemElement** and setting one of the referenced Item's Properties:

```

// Get the reference Item to use
const partItem = window.aras.getItemByKeyedName('Part', 'P-Tmpl1')
// Create an Item Document Element and add to the beginning of the document
const itemElement = document.createElement('ItemInfo', {item: partItem});
document.elements.root.children.insert(itemElement);
// Set the referenced Part's Make/Buy Property - changes set on Save
itemElement.item.setProperty('make_buy', 'Buy');

```

ItemElement extends **Element** and contains a handle to the **ItemElementItem** class, which is a wrapper for the referenced Item and provides accessors/mutators for Property data on the referenced Item. This section includes a description of these classes

ItemElement Class Properties		
Name	Write	Description
empty	F	True if there's no related Item set
item	F	Handle to the ItemElementItem object (see below).

7.7.4.12 ItemElementItem

ItemElementItem Class Properties		
Name	Write	Description
node	T	Referenced Item Object. The object handle is equivalent to that which is returned by <code>Innovator.getItemBy*()</code> methods. This can be changed by the Client API by setting this node. Changing the node Item changes the reference to that Item.

		Note: If setting, it's necessary to ensure that the Item used is of an ItemType included in the <code>tp_Item</code> Polysource.
id	F	ID of the referenced Item
type	F	ItemType name of the reference Item. For example, 'Part', 'Document'.
editable	F	True if changes to the referenced Item are allowed.
modified	F	True, if any Property was set prior to being saved.

ItemElementItem Class Methods		
Name	Input	Description
getProperty		Returns the value of the referenced Item Property with the given name
	name	Property name
setProperty		Sets the value of the referenced Item Property with the given name to the given value.
		Note: Setting a Property on a referenced Item is not saved to that Item until the Technical Document is saved. Authors will see a pencil icon (✎), signifying that the Property of the referenced Item has not been updated, next to the Item Document Element in the Tree View. Upon saving the pencil icon will be hidden.
	name	Property name
	value	Property value
dropChanges		Removes pending Property changes (established by the <code>setProperty()</code> method).

7.7.4.13 ItemPropertyElement

The **ItemPropertyElement** class extends **Element** with capability for setting and retrieving referenced Item Property data. **ItemPropertyElement** objects represent Mapped Property Document Elements (see Section 2.3.2.5), which are unformatted Text Document Items that *link* to a specific Property on an Item referenced by some Item Document Element in its parent lineage. This link is loose in that there's no established Relationship generated. Property mappings are by configured by Property name only. Changing the associated Item Document Element will cause changes to all descendant Mapped Properties regardless of whether the referenced Items were the same ItemType.

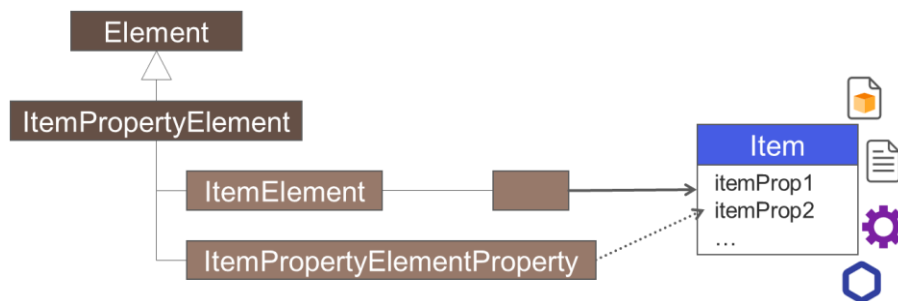


Figure 23. ItemPropertyElement Classes

The **ItemPropertyElement** is an accessor to both the **ItemPropertyElementProperty** Element – that represents the actual mapped Property in the document – and the **ItemElement** - that is a handle to the associated Item Document Element. Mapped Properties are either read-only, in which case the ItemPropertyElement provides a view of the actual Property data, or they are writable. Writable Mapped Properties provide a *proxy editing mechanism* using the Technical Document Editor. The Client API caches updates to writable Mapped Properties until the Technical Document is saved and the referenced Items are updated/synched.

The following code sample shows the process of creating an **ItemPropertyElement**:

```
// Create a Mapped Property set to refer to the 'tmp' Property
const cellMappedProp = document.createElement('ItemProperty');
cellMappedProp.property.name = 'tmp';
// Add it to the document
firstCellRow2.children.append(cellMappedProp);
```

When rendered in the document, the generated Mapped Property will display the Property value of whatever Item Element precedes it in the element ancestry lineage if one exists. Otherwise, the content will be empty.

ItemPropertyElement extends **Element** and contains a handle to the **ItemPropertyElementProperty** class, which is a wrapper for the referenced Mapped Property and provides accessors/mutators for Property data on the referenced Item. This section includes a description of these classes

ItemPropertyElement Class Properties		
Name	Write	Description
property	F	Handle to ItemPropertyElementProperty (see below)
sourceElement	F	Handle to the ItemElement object

7.7.4.14 ItemPropertyElementProperty

ItemPropertyElementProperty Class Properties		
Name	Write	Description
name	T	Mapped Property name
mode	T	Mode – either 'read' or 'write'
localValue	F	Current value of the Mapped Property
modified	F	True, if the Property was set prior to being saved.

7.7.4.15 ImageElement

The **ImageElement** class extends **Element** with capability for setting and retrieving referenced image Items (tp_Image). The 2D Graphic data must come from a tp_Image Item.

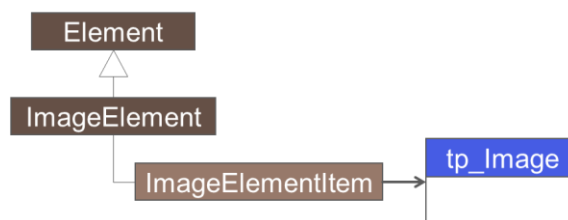


Figure 24. ImageElement Classes

The following code sample shows the process of creating an **ImageElement**:

```
// Get the tp_Image Item to use for the image data
const imgItem = window.aras.getItemByKeyedName('tp_Image', 'G-00016');
// Create the 'Graphic' Image Element and set the tp_Image
const imageElement = document.createElement('Graphic');
imageElement.image.node = imgItem;
// Add it to the document
document.elements.root.children.insert(imageElement);
```

This section includes a description of these classes.

ImageElement Class Properties		
Name	Write	Description
empty	F	True if there's no associated Image Item attached. Image Elements without an image will display with a 'placeholder' image in the viewer.
image	F	Handle to the ImageElementItem object (see below)

7.7.4.16 ImageElementItem

ImageElementItem Class Properties		
Name	Write	Description
node	T	Item (of type tp_Image) for setting only. Setting this property on an existing Image Element will cause the Image Element to update with the new image data. Note: This property is for setting only. To get the associated tp_Image Item, use the id Property, and retrieve using IOM.
id	F	ID of the associated tp_Image Item
src	F	URL to vault file used for the image

7.7.4.17 View

View provides access to the Technical Document Editor selection, scroll, and invalidation capability. This section provides a description of each of these functions and how they can be applied.

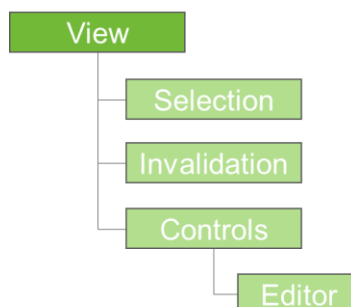


Figure 25. View Classes

View is an accessor to each of the Classes that provide for the functions mentioned above. Its only content (for the Client API) are handles to the Selection, Invalidation, and Editor Controls defined in this Section.

View Class Properties		
Name	Write	Description
selection	F	Handle to the Selection object (see below)
invalidation	F	Handle to the Invalidation object (see below)
controls	F	Handle to the Controls object (see below)

7.7.4.18 Selection

The **Selection** class provides the ability to get a list of selected **Elements** or set this list. This is useful for identifying the Element (or Elements) to apply a custom function to or direct the user to an Element (or Elements) by programmatically selecting them.

The following code sample shows the process of selecting a specific Text Element:

```
// Get the full set of Document Elements
const select = document.elements.root.descendants;
// Restrict to a subset of only Text Elements
const selectText = select.filter((element) => element.is('Text'));
if (selectText.length >= 1) {
  const lastTextElement = selectText[selectText.length-1];
  view.selection.set(lastTextElement);
}
```

Selection Class Methods		
Name	Input	Description
set	Sets the Elements selected in the viewer.	
	Note: It is possible to select multiple non-contiguous Elements. However, editing functions typically can only be applied within the editor to those that are contiguous.	
	Element(s)	Either a single Element or Element array
get	Returns an array of selected Elements. This array is empty if there's no current selections.	

7.7.4.19 Invalidation

The **Invalidation** class provides the ability to suspend the updates made visible in the editor so that multiple functions can be applied atomically. Within the Client API, when an Element is added, removed, or modified the UI is simultaneously updated. This may be time-consuming and/or result in unnecessary synchronization operations if there are many updates. In addition, the undo buffer is updated with each of these individual function calls. The Invalidation class provides a `suspend` and `resume` method to be used to wrap multiple operations so that they are applied together.

The following code sample will create a table like Figure 26:

Cell text			

Figure 26. Sample Generated Table

```
// Temporarily suspend Editor UI operations
view.invalidation.suspend();
// Create a new 3X4 Table
const table = document.elements.create('Table', {rowCount: 3, columnCount: 4});
// Add it to the beginning of the document
document.elements.root.children.insert(table);
// Get the first cell in the second row
const firstCellRow2 = table.cells.get(1,0);
// Insert a new Text Element
const cellText = document.createElement('Text', {text: 'Cell text'});
firstCellRow2.children.append(cellText);
// Add a new second row
table.rows.add(1);
// Merge the cell with added Text with its right neighbor
table.cells.merge(2, 0, 'right');
// Resume to apply changes
view.invalidation.resume();
```

The suspend and resume method calls must be applied in pairs, or the results are undefined. In this example, there are several operations that would cause an update to the UI. These include the insertion of the newly created Table, the insertion of the newly created Text, the additional Row, and the Merge of two Cells. By wrapping these operations within suspend/resume calls, the entire set of updates are treated as one.

Invalidation Class Methods		
Name	Input	Description
suspend		Suspends updates to the viewer.
		Note: This must be followed by a call to <code>resume</code> before the CUI Method returns.
resume		Resumes updates to the viewer.
		Note: This must be preceded by a call to <code>suspend</code> .

7.7.4.20 Editor Controls

The **Controls** class provides access to the **Editor** class for use in programmatically scrolling the View. This is useful when directing the user to a newly selected Element.

The following code sample shows the process of selecting and scrolling to a specific Text Element:

```
// Get the full set of Document Elements
const select = document.elements.root.descendants;
```

```
// Restrict to a subset of only Text Elements
const selectText = select.filter((element) => element.is('Text'));
if (selectText.length >= 1) {
  const lastTextElement = selectText[selectText.length-1];
  // Make sure the selected Element is in view
  view.controls.editor.scrollTo(lastTextElement);
  view.selection.set(lastTextElement);
}
```

Note that the **Editor** class is accessed through a single Property of the **Controls** class as shown above.

Editor Class Methods		
Name	Input	Description
scrollTo	Scrolls the view to the given Element in the viewer.	
	Note: The given Element must exist in the document. That is, it must be contained in the children list of some Element.	
	Element	Element within the document to scroll to

7.7.4.21 Schema

The **Schema** class provides the ability to access Document Schema information and validate operations on content within a document against the limitations and restrictions specified by the Document Schema. Schemas define restrictions with respect to the type of Document Elements that can be used, allowed Attributes, and cardinality.

The following code sample shows how the schema can be used to check for the addition of an **Element**.

```
const { document, view } = options.contextData;
const schema = document.schema;
...
const lastTextElement = selectText[selectText.length-1];
// Check to ensure that a new Text Element can be added to the document
const allowed = schema.checkElementAction('append', lastTextElement, 'Text');
if(allowed)
{
  // Code to append new content...
}
```

The logic in the sample retrieves the last Text Element in the document and checks the schema as to whether appending another Text Element is allowed. Actions of 'append' and 'insert' are permitted. Programmatically validating operations to a document in this manner ensures that the structure of a technical document conforms with the rules designated by the associated schema.

Schema Class Properties		
Name	Write	Description
item	F	Handle to the Document Type corresponding to the opened Technical Document.

elements	F	Object with Name/Object pairs for each Document Element defined in the document schema. Note: Each object in <code>elements</code> contains information about the XML schema element defined in the schema using the <code>declaration</code> property. The data within <code>declaration</code> is defined by functionality outside the Client API and is thus not directly controlled by it. In addition, the actual properties and population of property data is dependent on the content in the XML Schema. However, the property – <code>declaration.elementAttributes</code> is used with the Technical Document editor's Attribute dialog and is useful in other Use Cases associated with Technical Documentation. It is explained below for these purposes. As such the content in <code>elements.declaration.elementAttributes</code> is the only valid functionality to be used within the Client API for this Property. See example below.
----------	---	--

The `elements` parameter is dynamically generated based on the XML Elements defined in the XML Schema (see Section 2.3.1). Data for each XML Element is accessed using the Element's name as an index. For example, in the sample below to access data about the XML Element 'Note' use `elements['Note']`. The most useful part of this information is the attributes list, which contain the permitted values as defined in the schema if they are defined there. Element attributes provide metadata, which can be used to characterize or classify element instances. The following example shows how you can process the schema example in Section 7.7.3.1 to extract the defined/permitted Attribute values for the Note XML Element:

```
// Retrieve the Note Document Element Schema Item
const noteElem = schema.elements['Note'];
// Get the Attributes definition for this Element
const attrs = noteElem.declaration.elementAttributes;

// Search for the 'type' attribute to get the defined/allowed values
for(const attr of attrs)
{
    if(attr.Name === 'type')
    {
        var restrictions = attr.Restriction !== null ? attr.Restriction : [];
        // Use the attribute value setting from the schema
        resultSettings.options = restrictions.map(function(restVal) {
            return {
                label: restVal.Value,
                value: restVal.Value
            };
        });
    }
}
```

Each XML attribute object in `elementAttributes` has a `Name` and a `Restriction` Property array if restrictions were defined in the XML schema. The `Restriction` Property contains Type/Value pairs.

Schema Class Methods		
Name	Input	Description

checkElementAction	Returns True is the specified action on the given Element with the new/target Element is allowed. False otherwise.	
	Note: The given Element must exist in the document. That is, it must be contained in the children list of some Element.	
	action	Either 'insert' or 'append'. Designates the intended action to check.
	source	Element that the specified Action is to be performed on. Since the accepted actions only apply to the addition of content, they are checked relative to an existing Element.
	target	Type name (String) representing the Document Type to check to see if an append or insertion if allowed.
getElementNames	Returns a String array of all the Document Element Names in the schema	